Business Process Technologies and Management
2. Conceptual Process Modeling

Prof. Dr. Stefanie Rinderle-Ma

Dr. Jürgen Mangler

Technical University of Munich

Department of Informatics

Chair of Information Systems

and Business Process Management

stefanie.rinderle-ma@tum.de

juergen.mangler@tum.de

Literature and links

- Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures, Third Edition. Springer 2019, ISBN 978-3-662-59431-5, pp. 1-417 2019
- Marlon Dumas, Marcello La Rosa, Jan Mendling, Hajo A. Reijers: Fundamentals of Business Process Management, Second Edition. Springer 2018, ISBN 978-3-662-56508-7, pp. 1-527
- https://www.signavio.com/bpm-academic-initiative/
- https://cpee.org/

Process modeling levels

→ **Capturing the process logic, communication**

- Semantic or conceptual process modeling
  - Event-driven process chains (EPC), ARIS https://www.ariscommunity.com/aris-express
  - Business Process Modeling and Notation (BPMN), Standard, www.bpmn.org
- Logic, formal languages:
  - Petri nets, Workflow Nets
  - Refined Process Structure Tree (RPST) → J. Vanhatalo, H. Völzer, J. Koehler: The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)
- Execution languages: graphical programming
  - Examples: BPEL, WSM-Nets, CPEE-Trees
- Transformation between levels is possible, but has to be done carefully!

→ **Verification, Analysis**

→ **Process implementation and execution**

Process modeling levels

→ **Capturing the process logic, communication**

- Semantic or conceptual process modeling
  - Event-driven process chains (EPC), ARIS https://www.ariscommunity.com/aris-express
  - **Business Process Modeling and Notation (BPMN)**, Standard, www.bpmn.org

→ **Verification, Analysis**

- Logic, formal languages:
  - **Petri nets, Workflow Nets**
  - **Refined Process Structure Tree (RPST)** → J. Vanhatalo, H. Völzer, J. Koehler: The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)

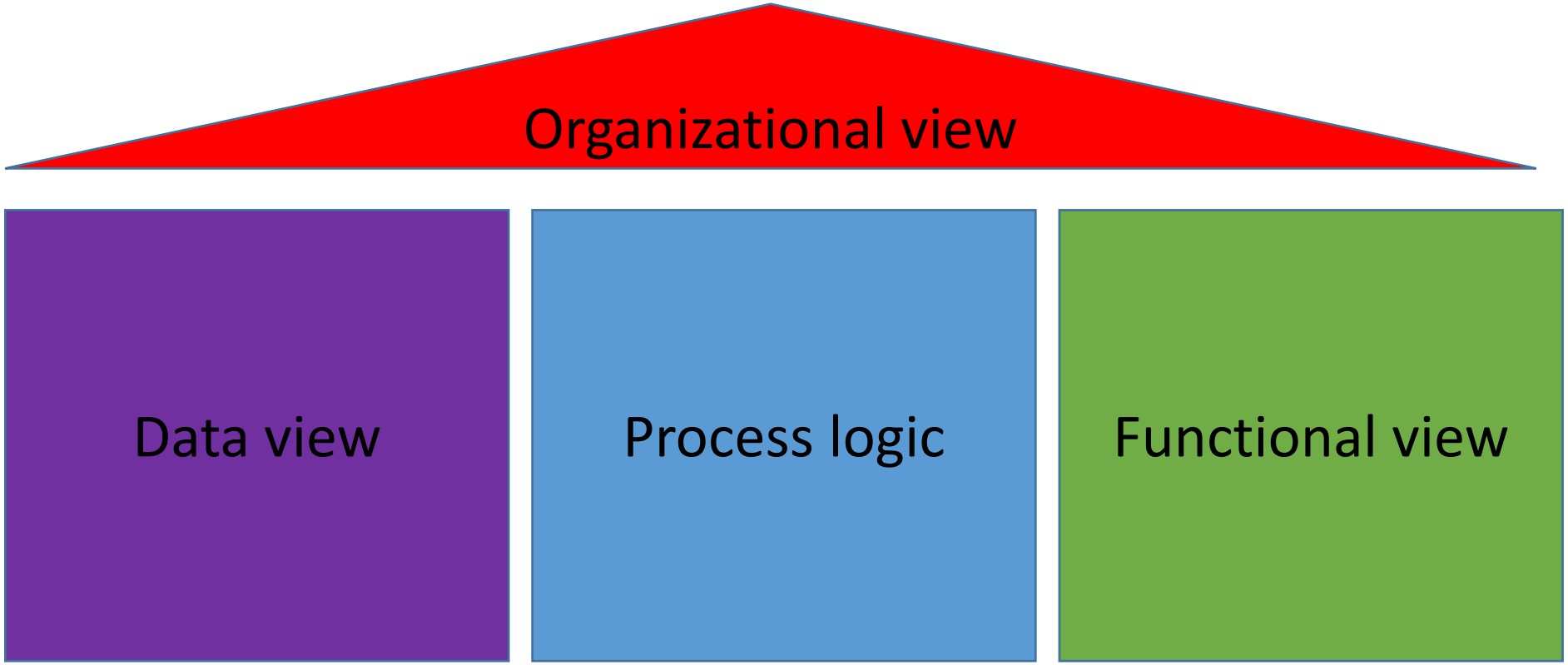
→ **Process implementation and execution**

- Execution languages: graphical programming
  - Examples: BPEL, WSM-Nets, **CPEE-Trees**
- Transformation between levels is possible, but has to be done carefully!

Example: Customer complaint

Customers state their complaint via an online form. The form contains the insurance number, the amount of damage, and a description of the damage (mandatory). Optionally, customers can upload pictures for further documentation. If the amount of damage is ≤ 100 Euro, an automatic plausibility check is conducted and the complaint is approved. The customer is informed via email and the money transfer is initiated. If the damage amount is above 100 Euro, a clerk is checking the complaint. The customer is informed on the decision via email. If the complaint is rejected, the customer is offered an optional meeting.

Process perspectives



Example: Customer complaint

red:
green:
blue:
purple:

Customers state their complaint via an online form. The form contains the insurance number, the amount of damage, and a description of the damage (mandatory). Optionally, customers can upload pictures for further documentation. If the amount of damage is ≤ 100 Euro, an automatic plausibility check is conducted and the complaint is approved. The customer is informed via email and the money transfer is initiated. If the damage amount is above 100 Euro, a clerk checks the complainnt. The customer is informed on the decision via email. If the complaint is rejected, the customer is offered an optional meeting.

Process modeling levels

→ **Capturing the process logic, communication**

- Semantic or conceptual process modeling
  - Event-driven process chains (EPC), ARIS https://www.ariscommunity.com/aris-express
  - **Business Process Modeling and Notation (BPMN)**, Standard, www.bpmn.org
- Logic, formal languages:
  - **Petri nets, Workflow Nets**
  - **Refined Process Structure Tree (RPST)** → J. Vanhatalo, H. Völzer, J. Koehler: The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)
- Execution languages: graphical programming
  - Examples: BPEL, WSM-Nets, **CPEE-Trees**
- Transformation between levels is possible, but has to be done carefully!

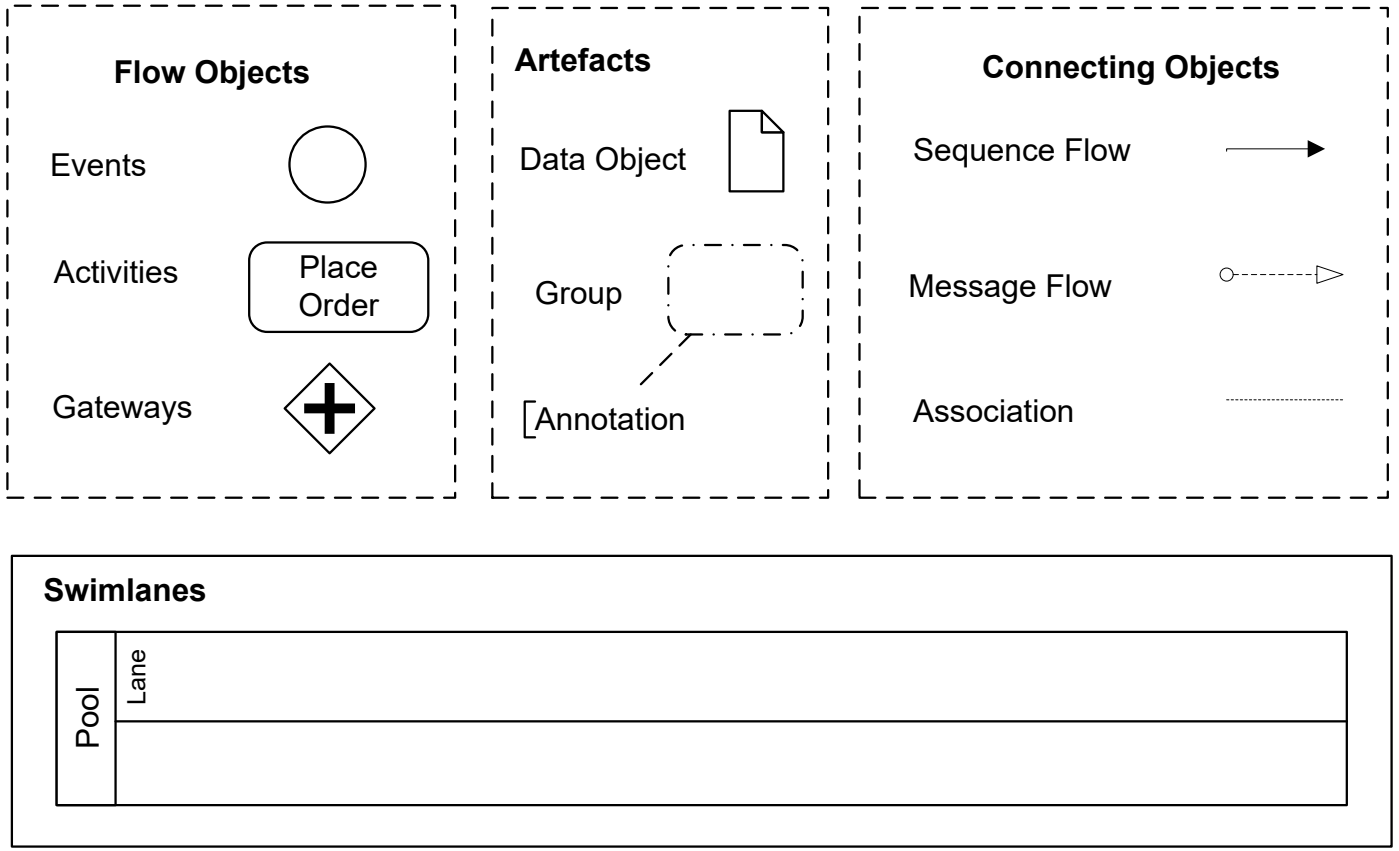
→ **Verification, Analysis**

→ **Process implementation and execution**

BPMN (recap)

- Current process modeling standard: https://www.bpmn.org/
- Web-based modeling with Signavio: https://academic.signavio.com/p/login
- Many other tools available
- Selection of specific design goals:
  - Modeling of interorganizational processes → communication between partners via message exchanges
  - Previously: Mapping to process execution language WS-BPEL (Business Process Execution Language for Web Services), deprecated
  - Mapping to execution languages is necessary as formal semantics is only partly available
- In this course: mapping of BPMN to to Petri Nets for formal analysis
- And to CPEE trees (https://cpee.org/) for process automation and execution

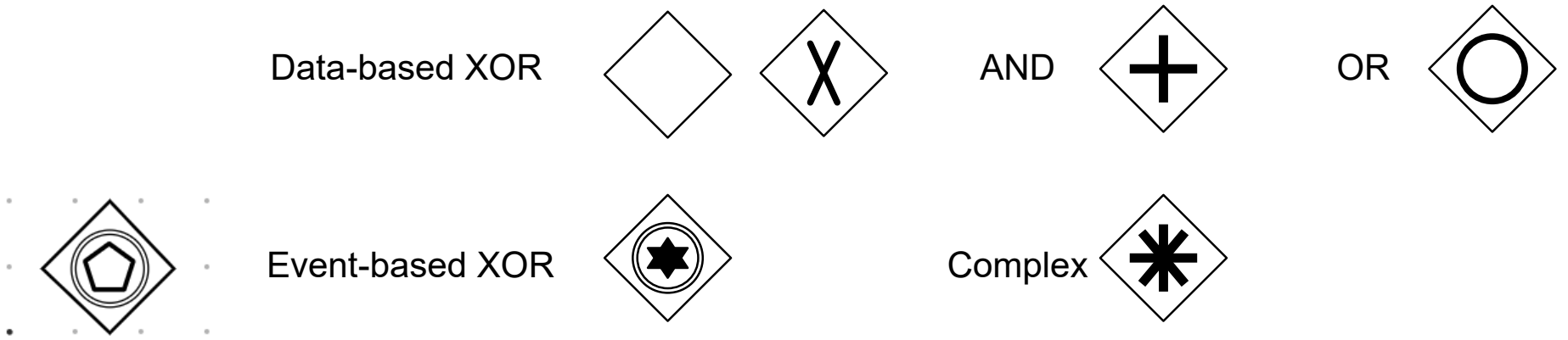
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.78. Business Process Modeling Notation: categories of elements

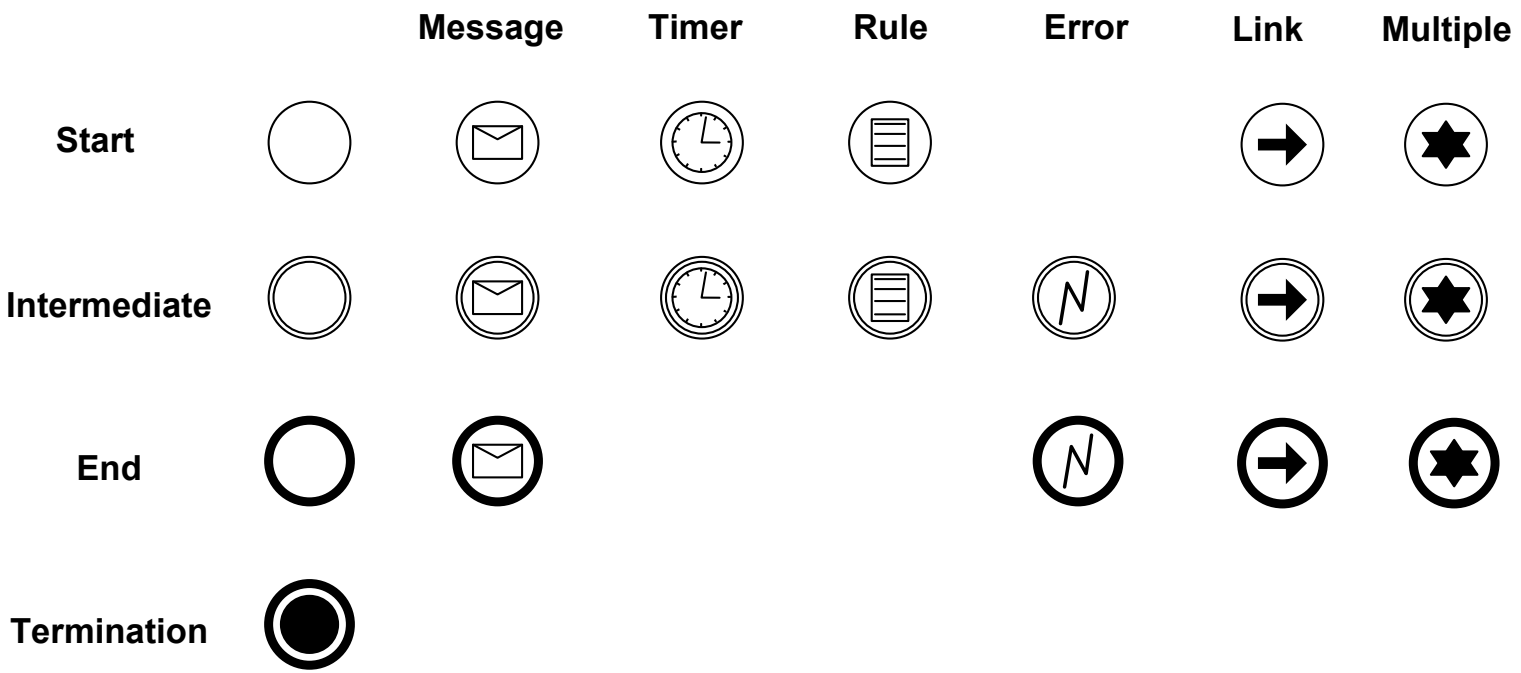
BPMN (recap)



Gateway types in the BPMN, Object Management Group (2006)

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

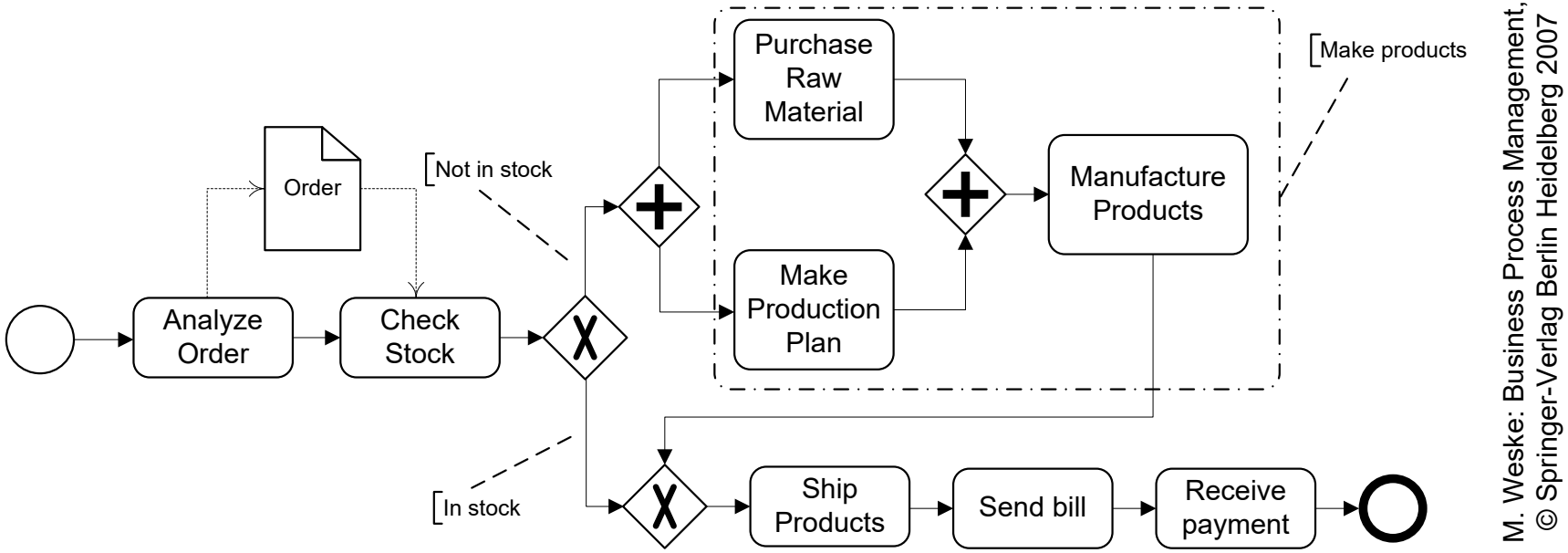
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.80. Event types in the BPMN, Object Management Group (2006)

BPMN (recap)



M. Weske: Business Process Management, © Springer-Verlag Berlin Heidelberg 2007

Fig 4.77. Business process diagram expressed in BPMN

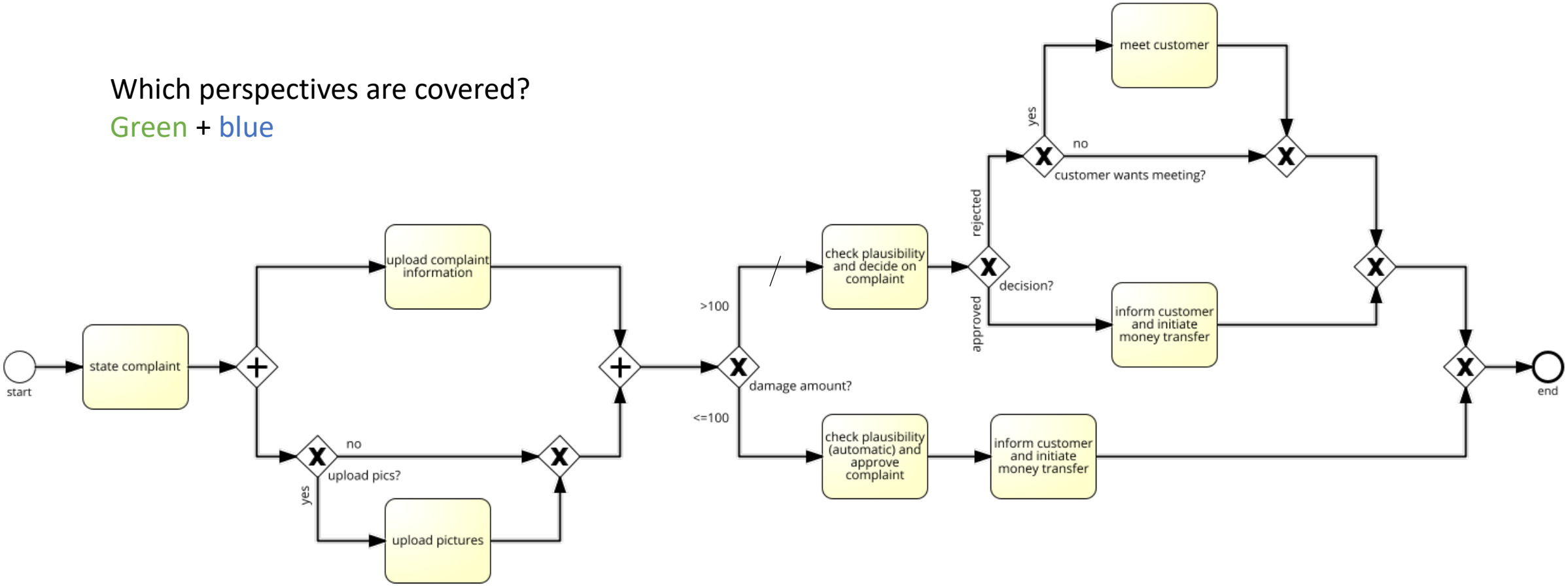
Example: Customer complaint

red:
green:
blue:
purple:

Customers state their complaint via an online form. The form contains the insurance number, the amount of damage, and a description of the damage (mandatory). Optionally, customers can upload pictures for further documentation. If the amount of damage is ≤ 100 Euro, an automatic plausibility check is conducted and the complaint is approved. The customer is informed via email and the money transfer is initiated. If the damage amount is above 100 Euro, a clerk checks the complaint. The customer is informed on the decision via email. If the complaint is rejected, the customer is offered an optional meeting.

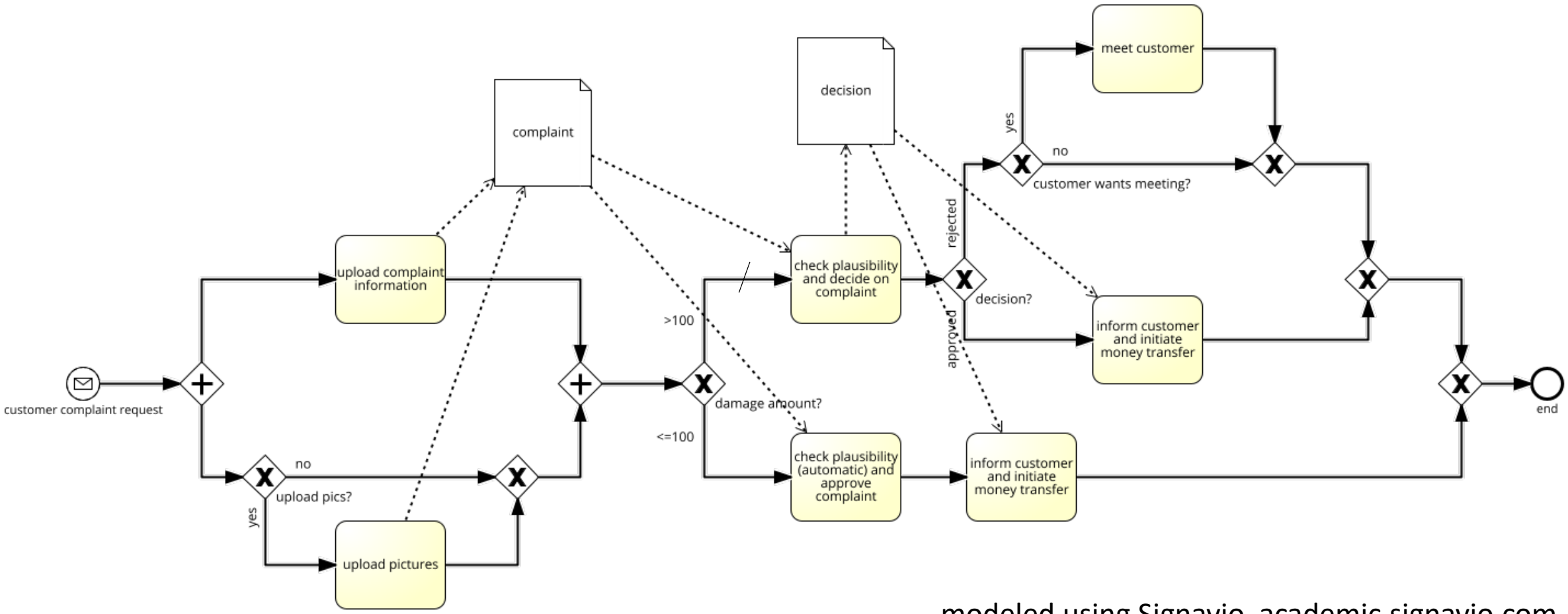
BPMN process model – control flow only

Which perspectives are covered?
Green + blue



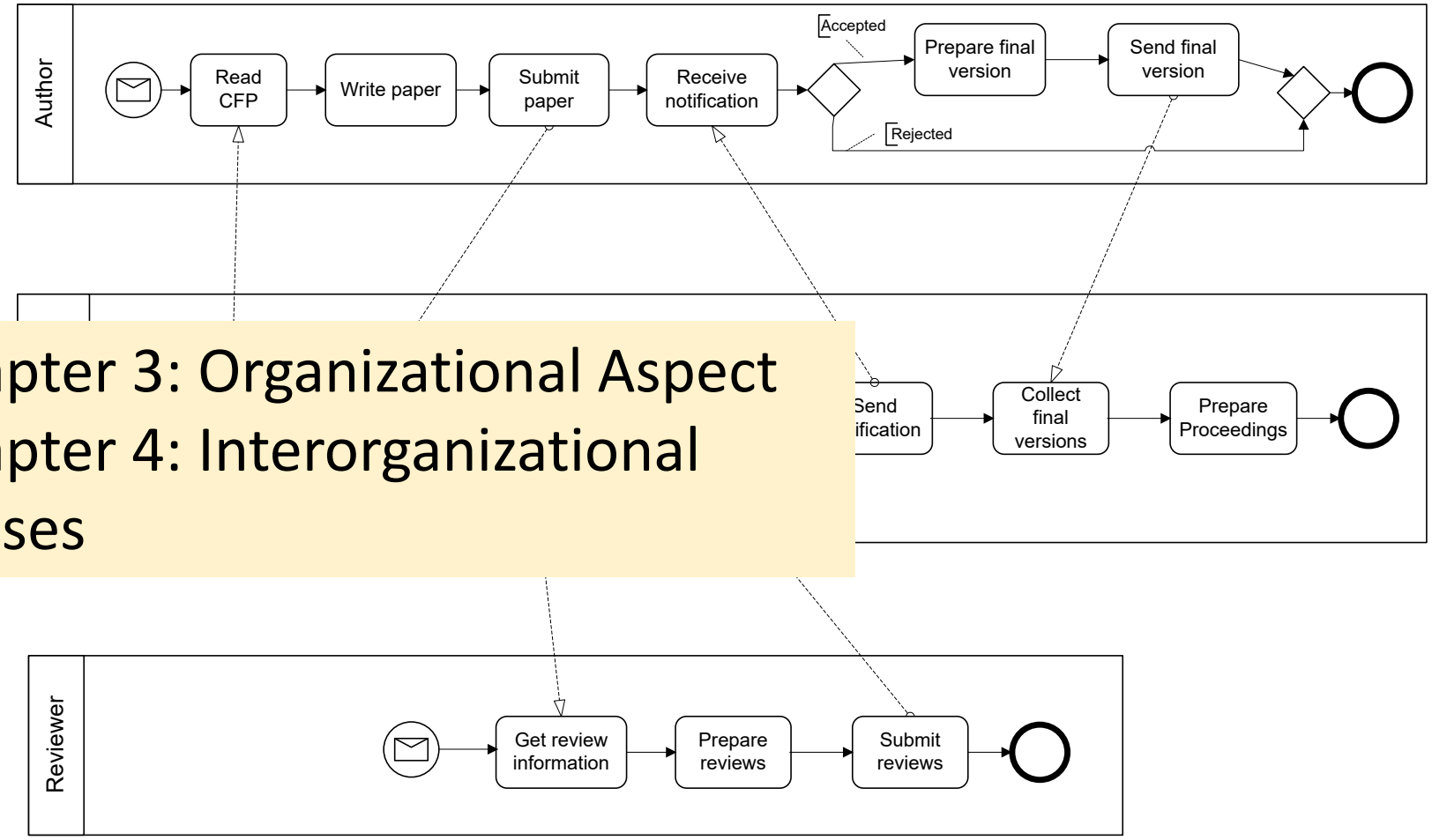
modeled using Signavio, academic.signavio.com

BPMN process model: + data



modeled using Signavio, academic.signavio.com

BPMN (recap)



→ Chapter 3: Organizational Aspect
→ Chapter 4: Interorganizational Processes

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.79. Business process diagram of a scientific conference review process

BPMN (recap)

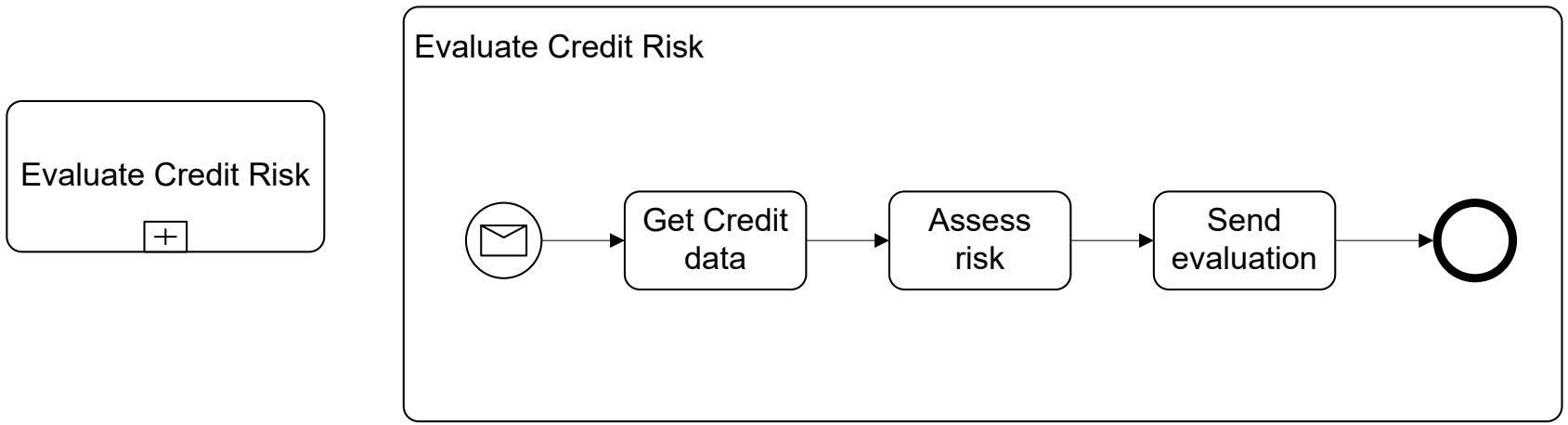
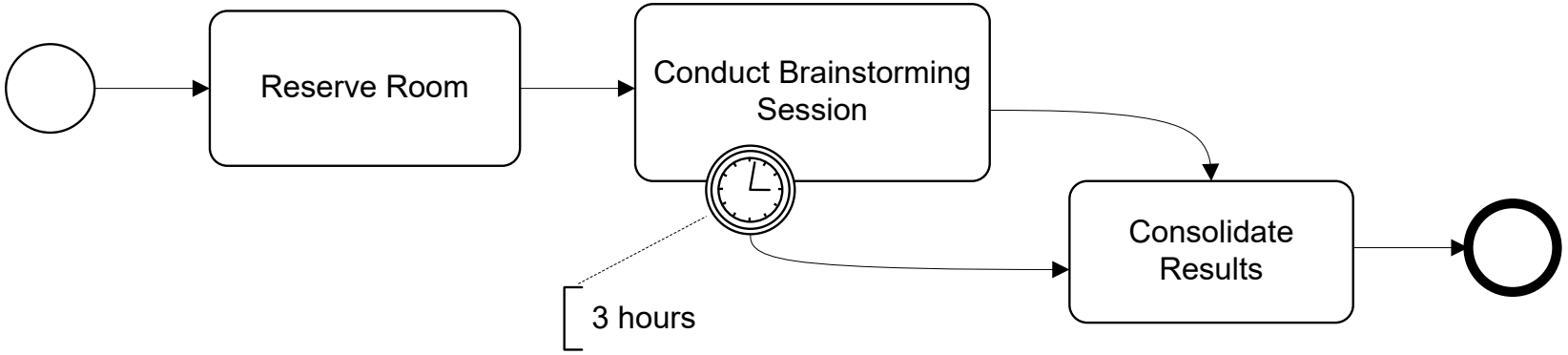


Fig 4.81. Collapsed and expanded subprocess

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

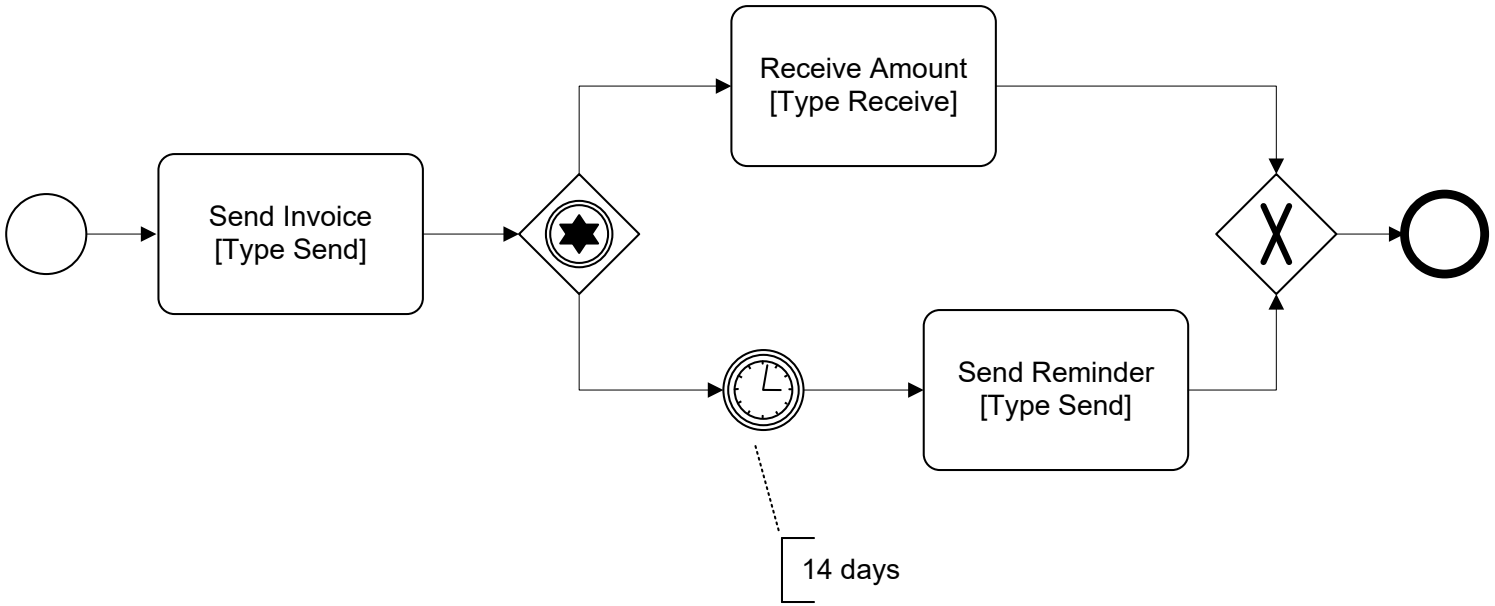
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.83. Exception flow, triggered by intermediate timer event

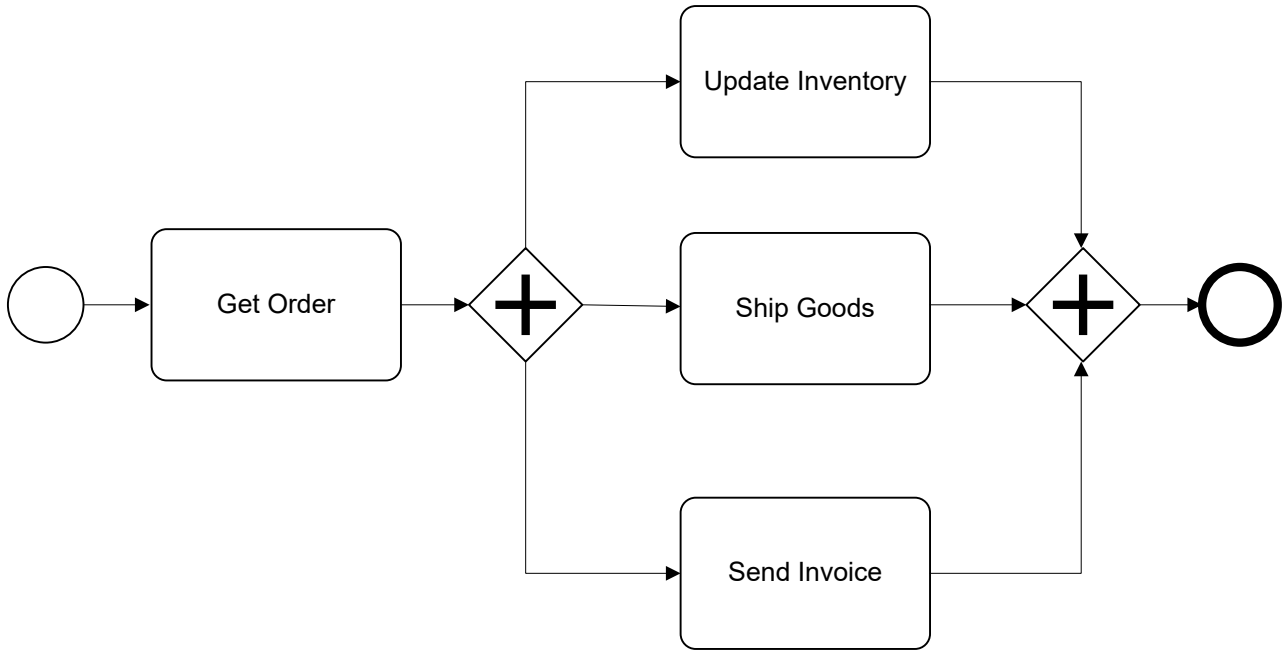
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.85. Example of an *event-based exclusive or* gateway

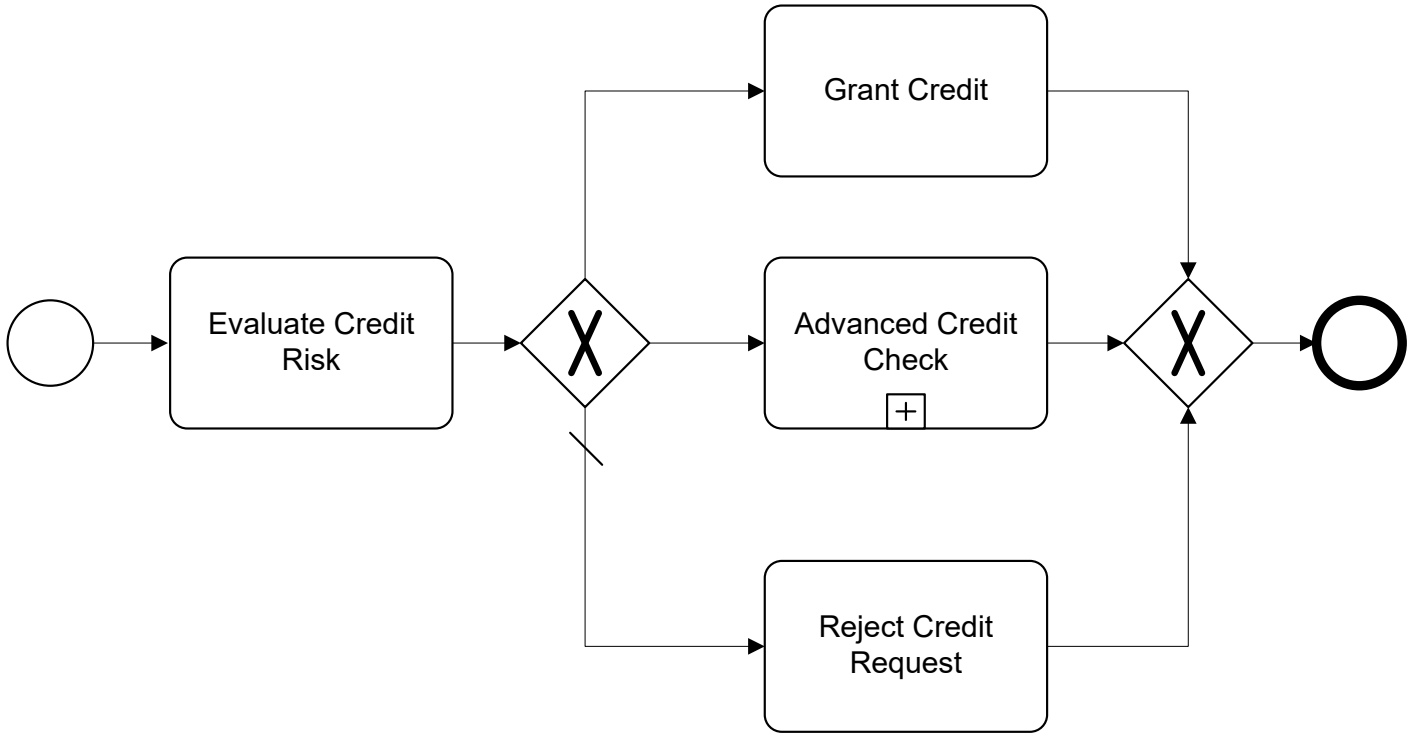
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.87. Example of an *and split* and *and join* gateway

BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.88. Sample business process with sequence flow and default sequence flow

Process modeling levels

→ **Capturing the process logic, communication**

- Semantic or conceptual process modeling
  - Event-driven process chains (EPC), ARIS https://www.ariscommunity.com/aris-express
  - **Business Process Modeling and Notation (BPMN)**, Standard, www.bpmn.org
- Logic, formal languages:
  - **Petri nets, Workflow Nets**
  - **Refined Process Structure Tree (RPST)** → J. Vanhatalo, H. Völzer, J. Koehler: The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)
- Execution languages: graphical programming
  - Examples: BPEL, WSM-Nets, **CPEE-Trees**
- Transformation between levels is possible, but has to be done carefully!

→ **Verification, Analysis**

→ **Process implementation and execution**

Petri Nets

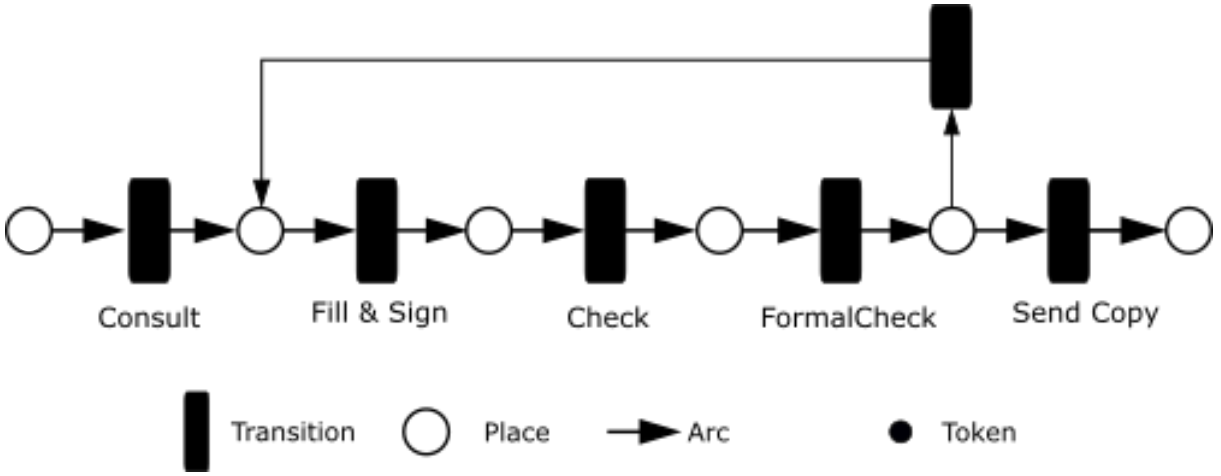
- Different classes with different expressiveness
- Syntax and formal semantics
- bipartite graph with transitions (active elements) and places (passive elements); transitions and places are arranged in an alternative way
- For process modeling: Workflow Nets -> later

DEFINITION 2.1 (Petri Net) A *(Petri-)Net* is a triple N = (P, T, F) with

$$P \cap T = \emptyset \text{ and } F \subseteq (P \times T) \cup (T \times P)$$

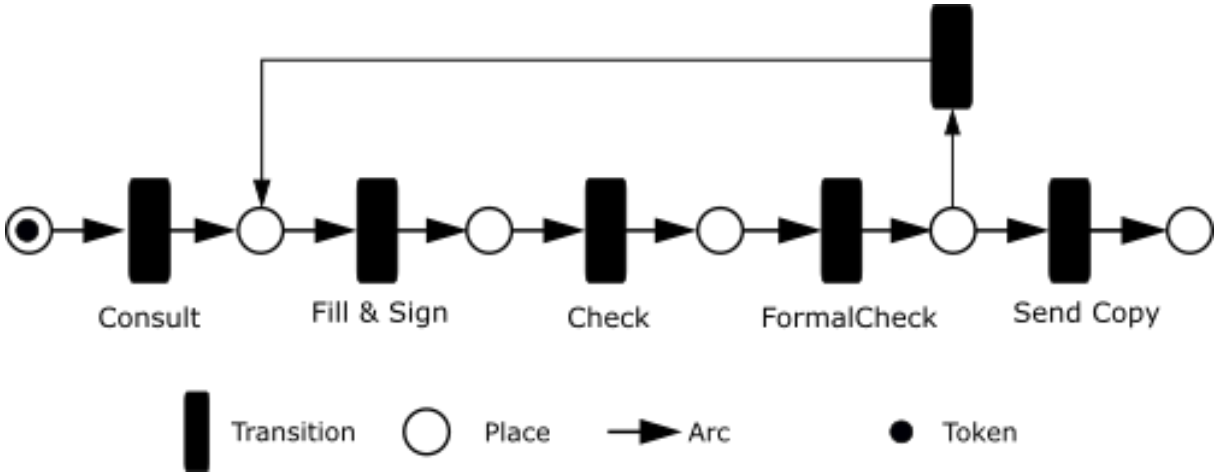
Petri Nets

- Pros:
  - formal analysis -> reachability
  - automation basically possible
- Cons:
  - understandability
  - data flow has to modeled in a separate manner; “higher” classes of Petri Nets, e.g., DB-Nets (M. Montali, A. Rivkin: DB-Nets: On the Marriage of Colored Petri Nets and Relational Databases. Trans. Petri Nets Other Model. Concurr. 12: 91-118 (2017))



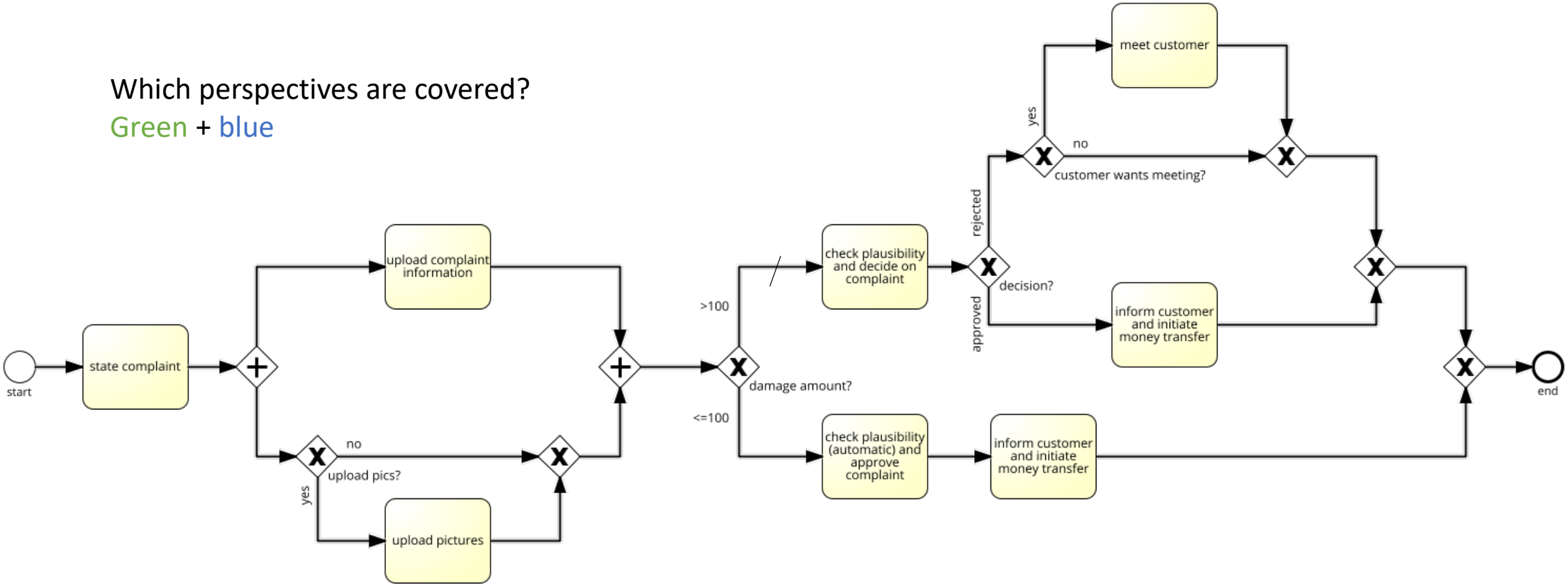
Petri Nets

- Dynamic behavior is expressed by tokens
- Tokens mark places
- **Event/Condition Nets (EC Nets)**: max. 1 token per place
- Interpretation of places as pre-conditions: if a place carries a token, the precondition is fulfilled



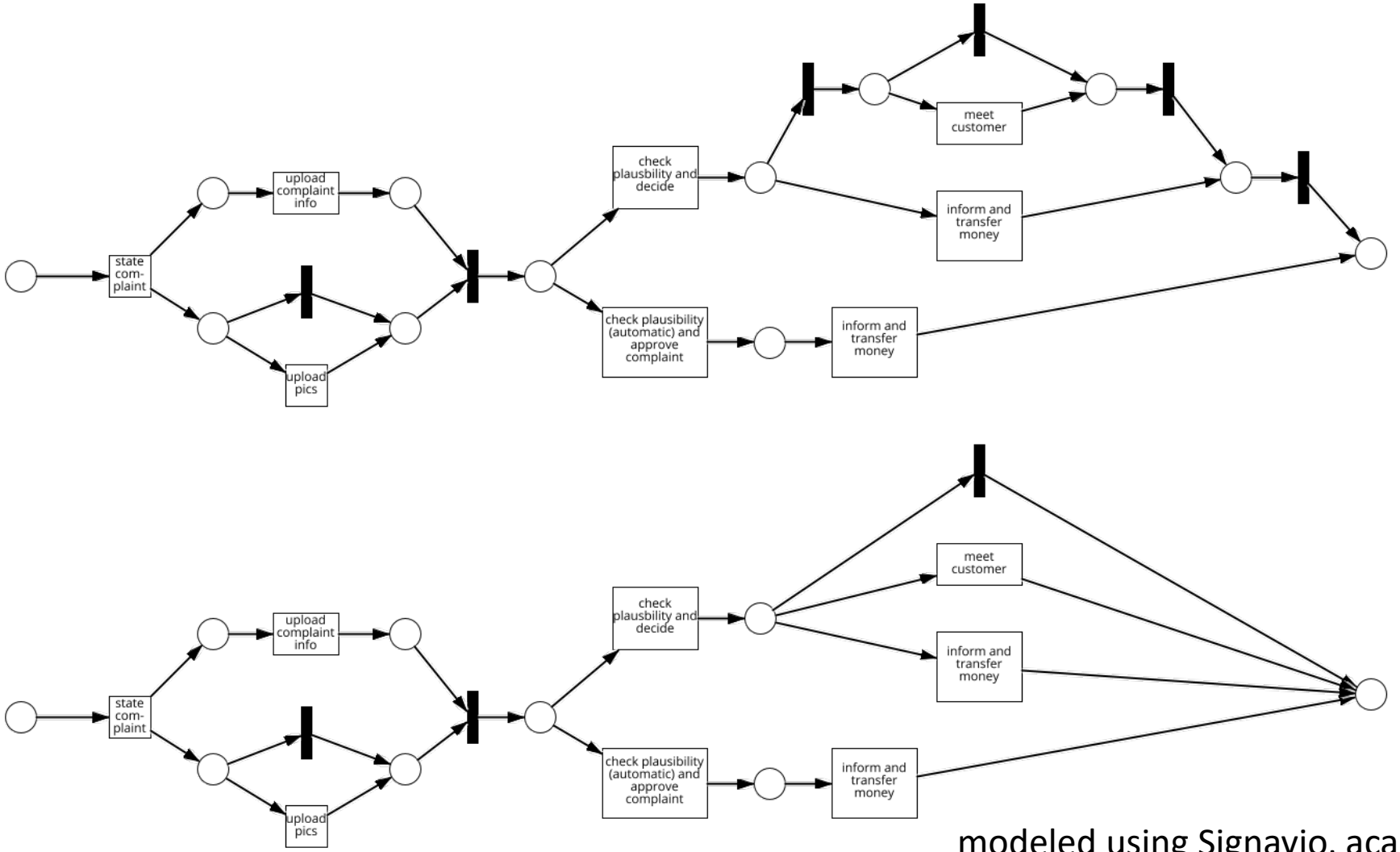
BPMN process model – control flow only

Which perspectives are covered?
Green + blue



modeled using Signavio, academic.signavio.com

Complaint process control Flow: EC Net



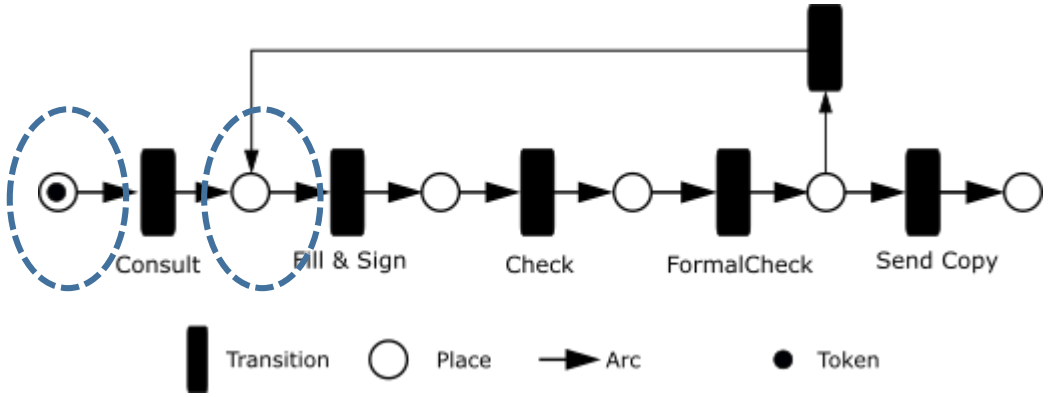
modeled using Signavio, academic.signavio.com

Petri Nets

DEFINITION 2.2 (Marking): A marking function M of an E/C-Net PN = (P, T, F) is a mapping M: P → {0,1} or M: P → {True, False} respectively.

- **M(s) = True: Place s carries one token, i.e., the condition described by s is satisfied**
- **M(s) = False: Place s carries no token, i.e., the condition described by s is dissatisfied**

For marking M the set of marked places is given by c := { p ∈ P | M(p) = 1 }. c is denoted as *case*.

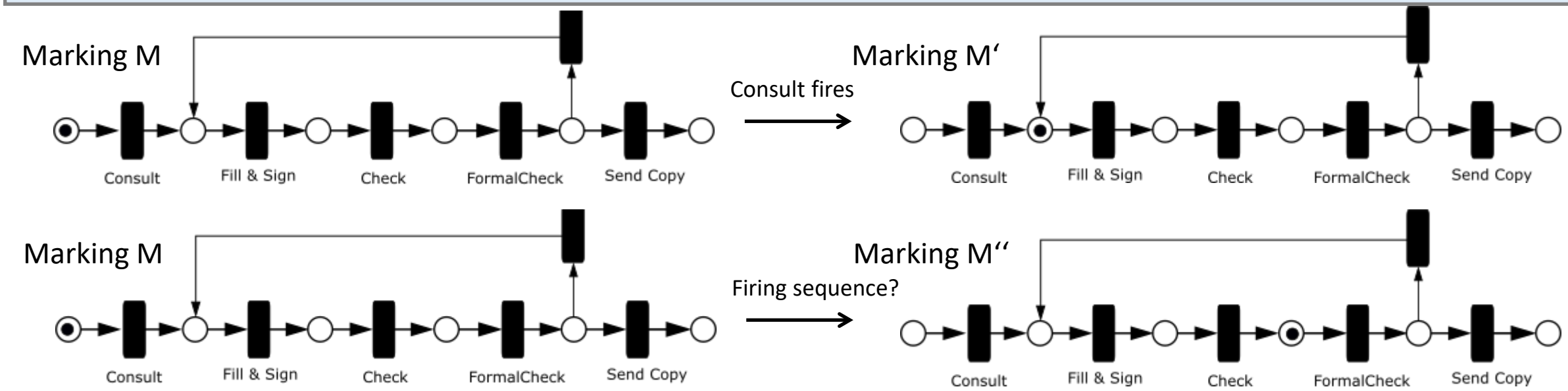


For $t \in T$, $\bullet t = \{p \mid (p,t) \in F\}$ defines the set of input places for t and $t\bullet = \{p \mid (t,p) \in F\}$ defines the set of output places.

Petri Nets

DEFINITION 2.3 (Marking/Firing): Let M: P → {F, T} be marking function of an E/C-Net PN = (P, T, F).

- **$M \xrightarrow{t} M'$: by firing t, the state of PN changes from M to M'**
- **$M \longrightarrow M'$: ∃ t ∈T with $M \xrightarrow{t} M'$**
- **$M_1 \xrightarrow{*} M_n$: ∃ firing sequence $t_1, t_2, \ldots, t_{n-1}$ such that $M_i \xrightarrow{ti} M_{i+1}$ for i = 1, …, n**
- **M' is reachable from M ⇔ $M \xrightarrow{*} M'$**



Petri Nets

- A transition t of an E/C Net will become *enabled* if all input places of t and no ouput place of t carry a token, formally:

$$\forall p \in \bullet t : M(p) = True \wedge \forall p \in t\bullet : M(p) = False$$

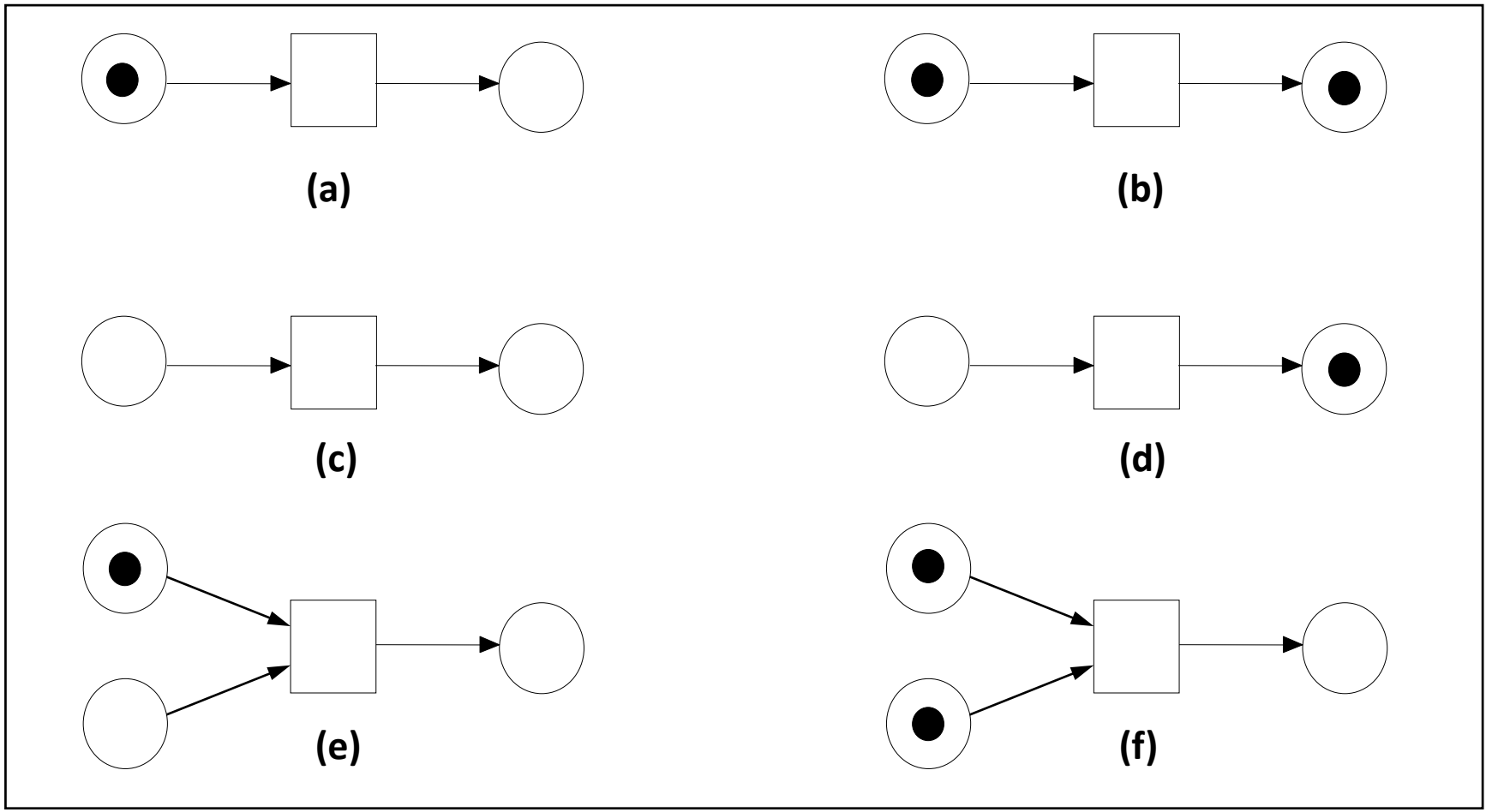
- Only activated transitions may *fire* (i.e., be executed).
- When firing transition t all tokens from its input places are removed and one new token is added to each outplace of t, formally:

$$M \xrightarrow{t} M' \text{ with } M'(s) = \begin{cases} \text{False} & p \in \bullet t \setminus t\bullet \\ \text{True} & p \in t\bullet \setminus \bullet t \\ \text{True} & p \in \bullet t \cap t\bullet \\ M(p) & p \notin \bullet t \cup t\bullet \end{cases}$$

Petri Nets

- **Marking Rule for E/C-Nets explained**: When firing a transition, all tokens from the input places are removed and on a token is put on every output place.
- By firing all transitions „through“ the net, a process instance (case) is executed.
- Recall: process instances describe specific cases for e.g., a patient or a product.
- Modeling of the process, i.e., the process templace, happens during *design time*.
- Executing the process instances happens during *runtime*.

Petri Nets

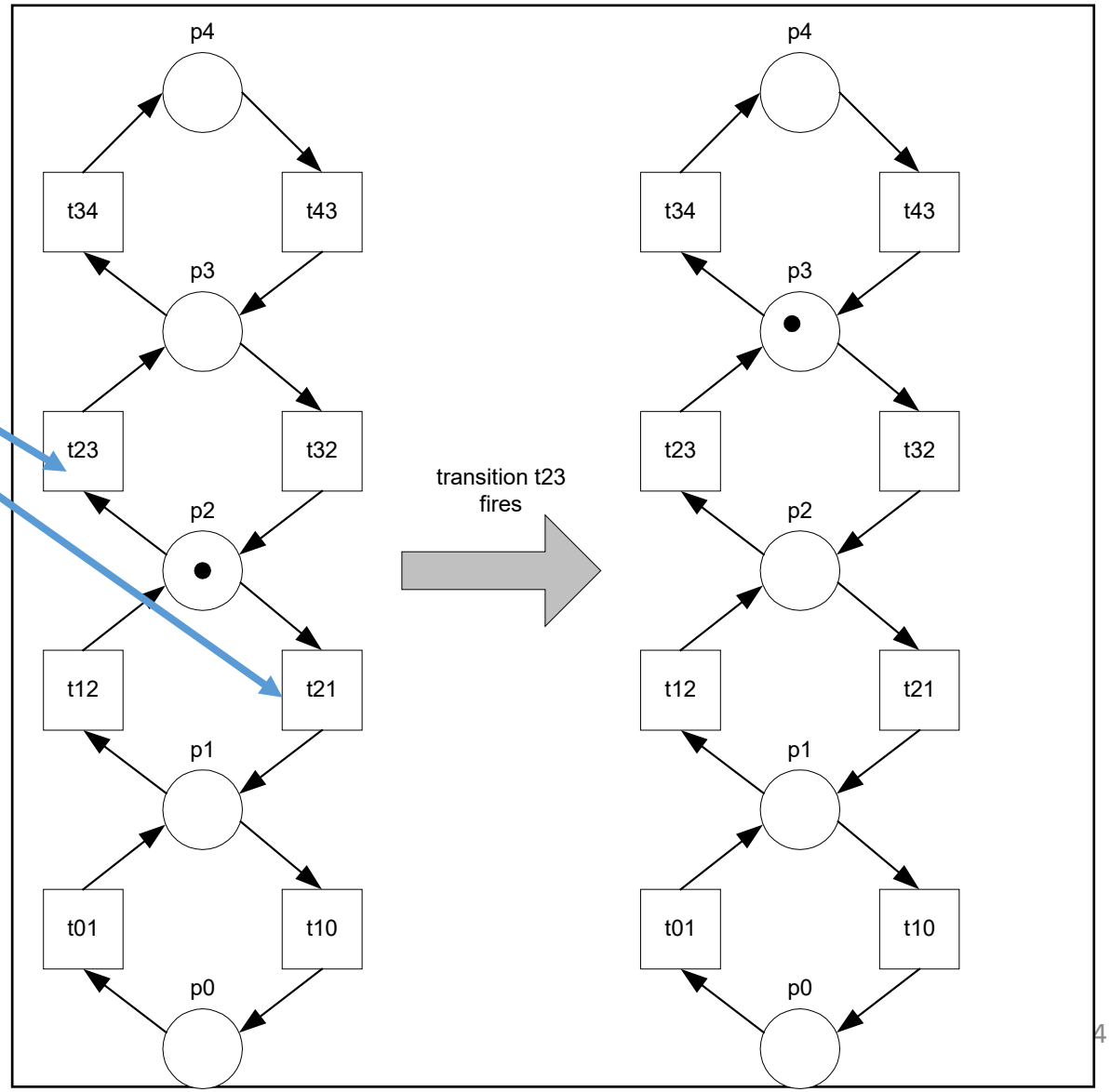


Exercise: Which transitions are enabled?

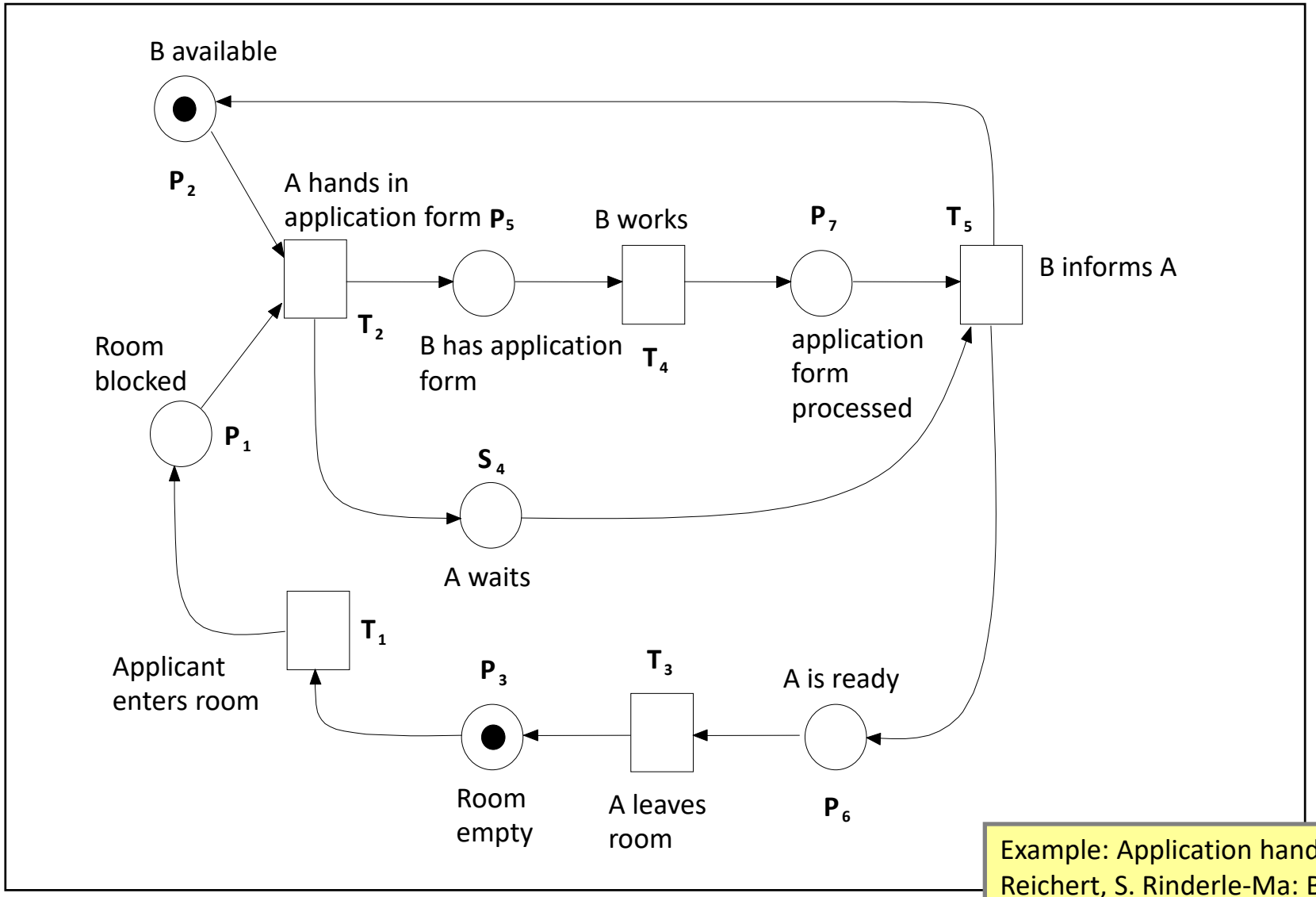
Petri Nets

Two transitions are enabled, but only one can fire

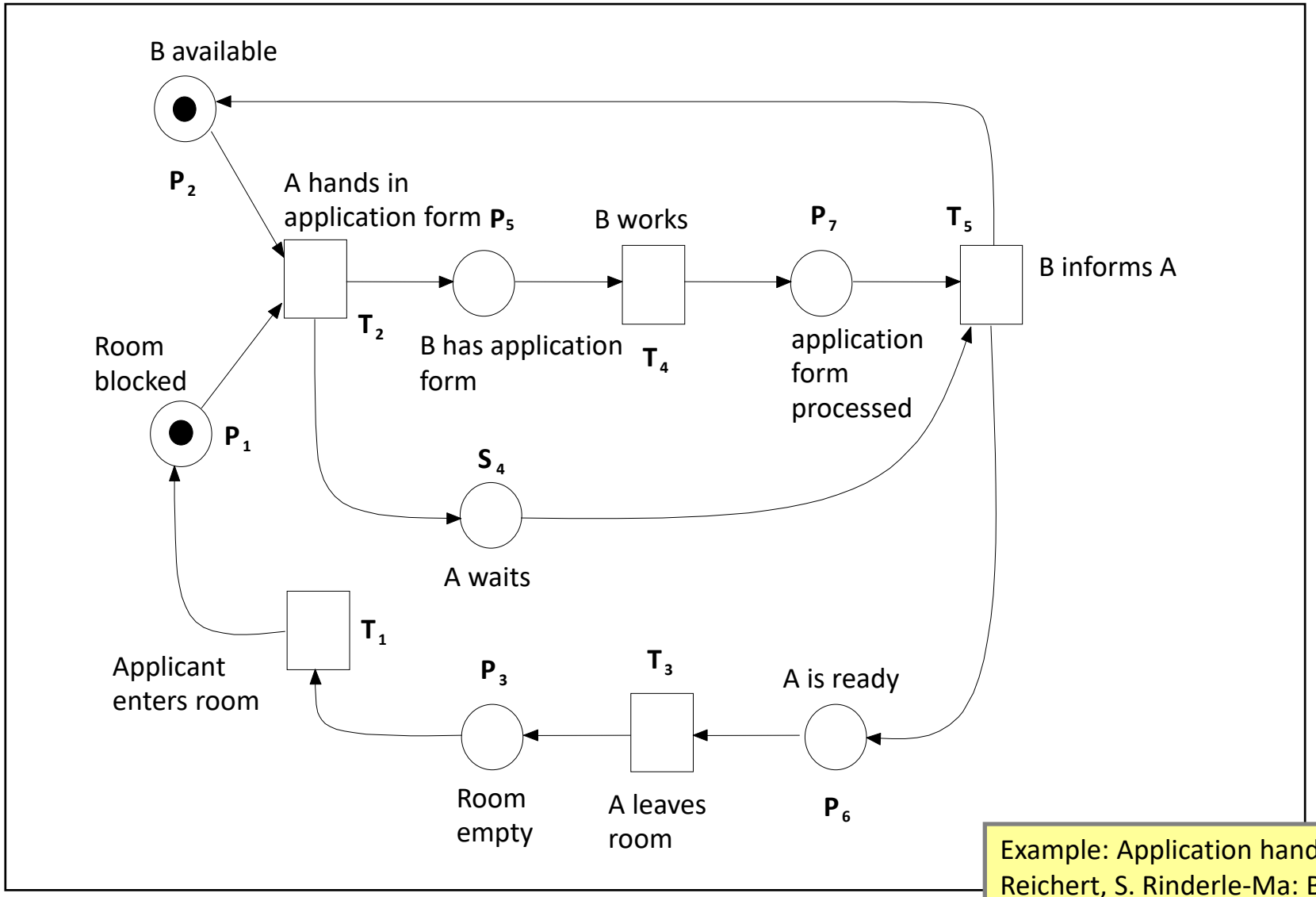
Non-determinism



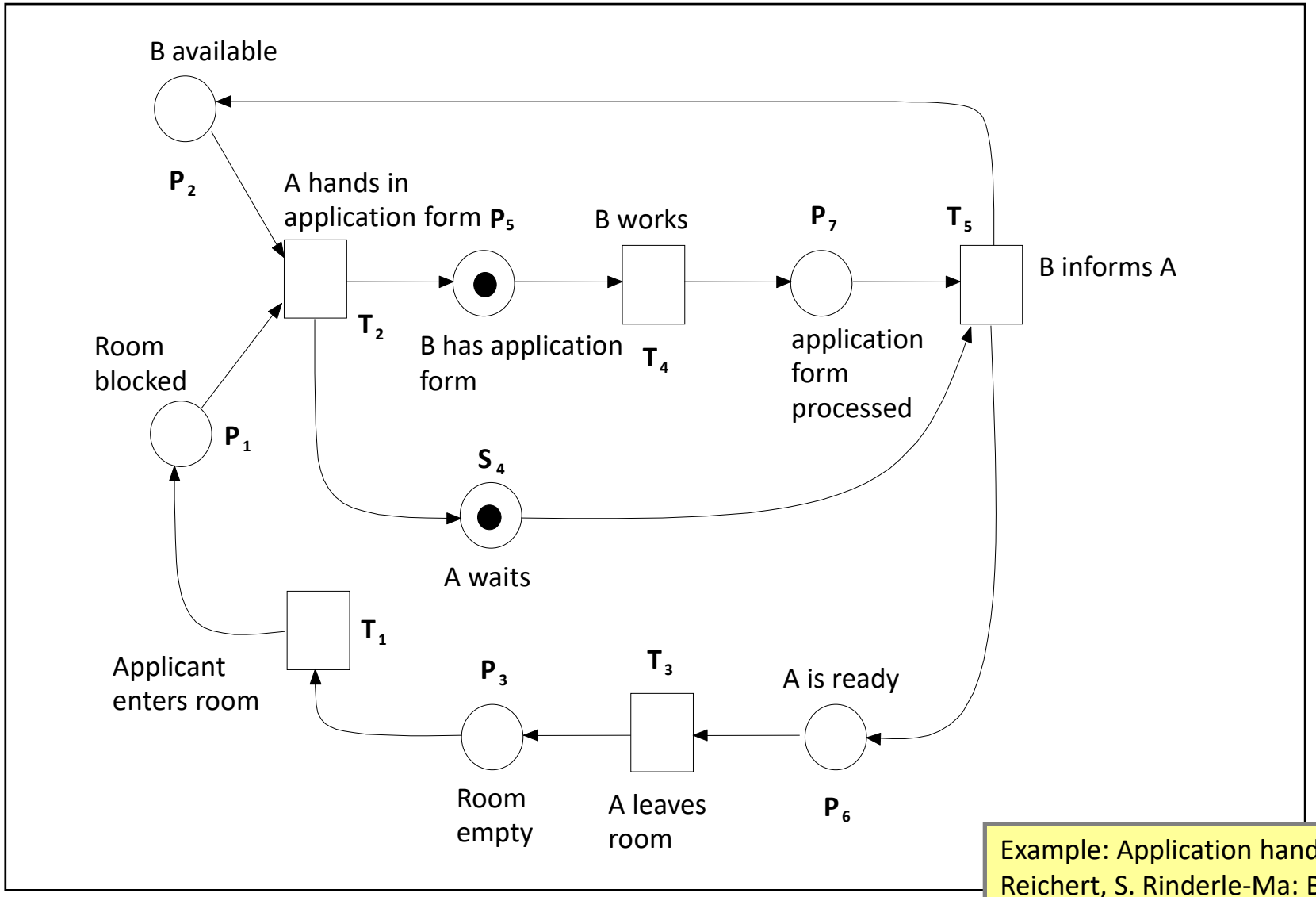
Example taken from
www.workflowcourse.com
(W.M.P. van der Aalst)



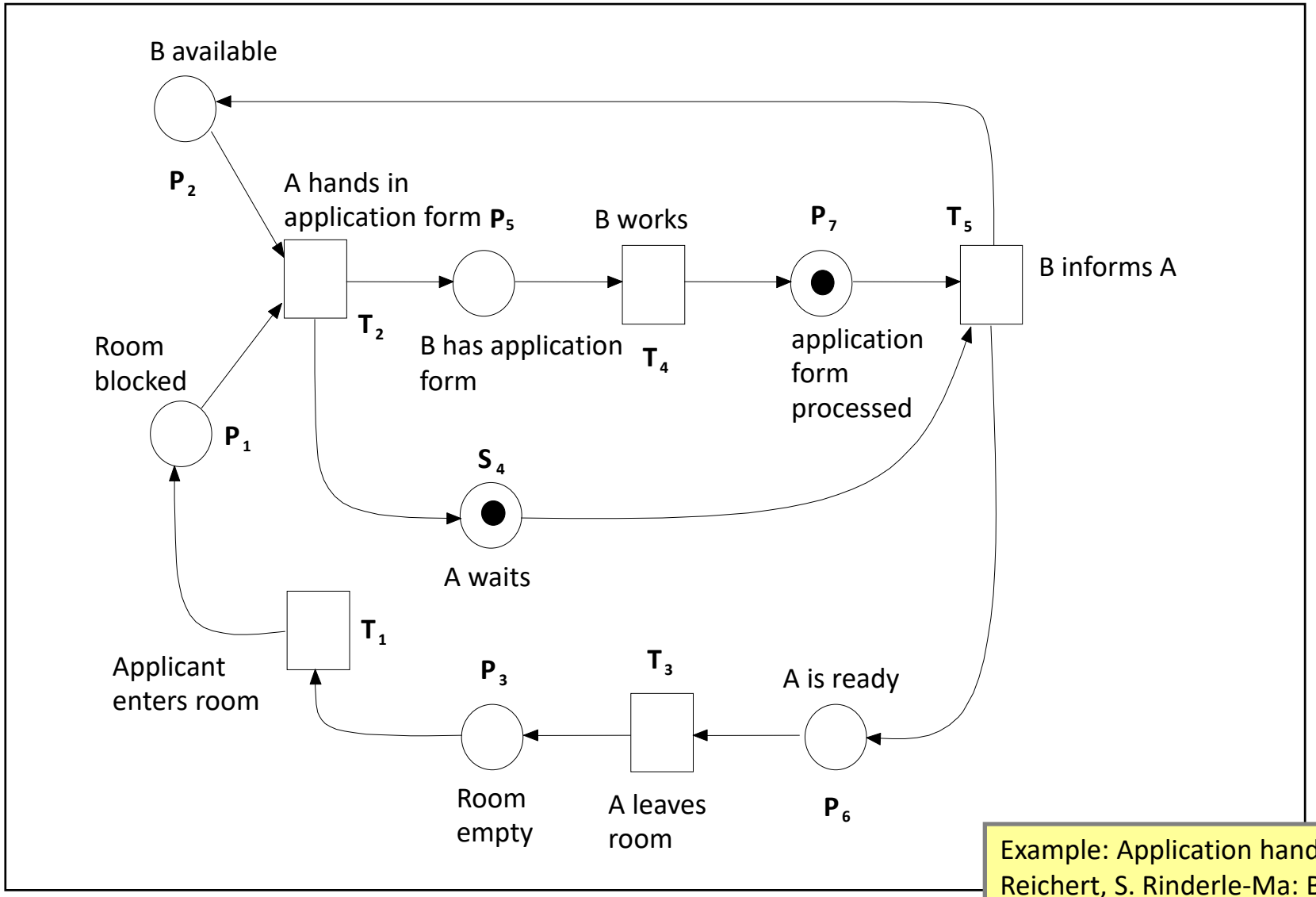
Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



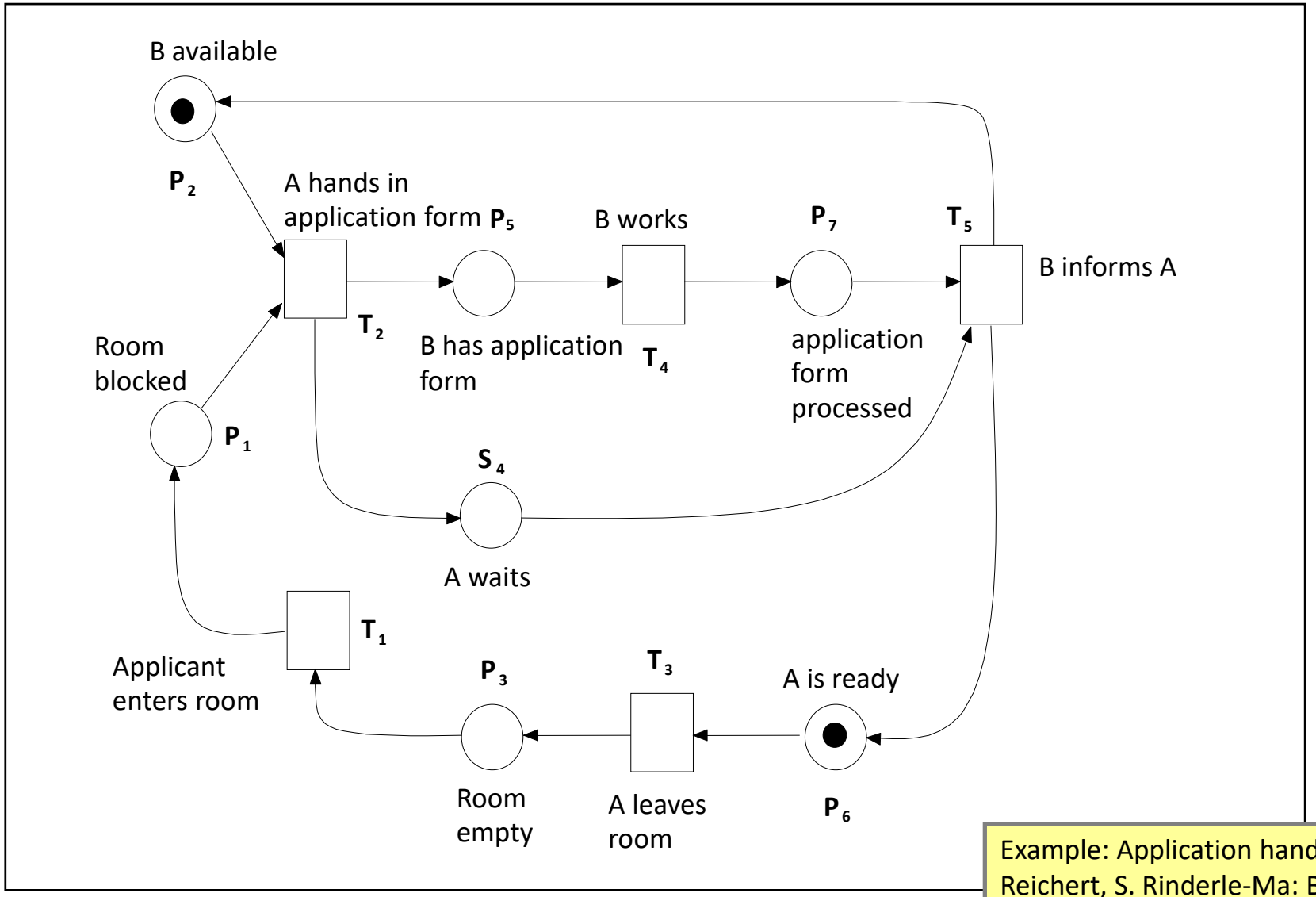
Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



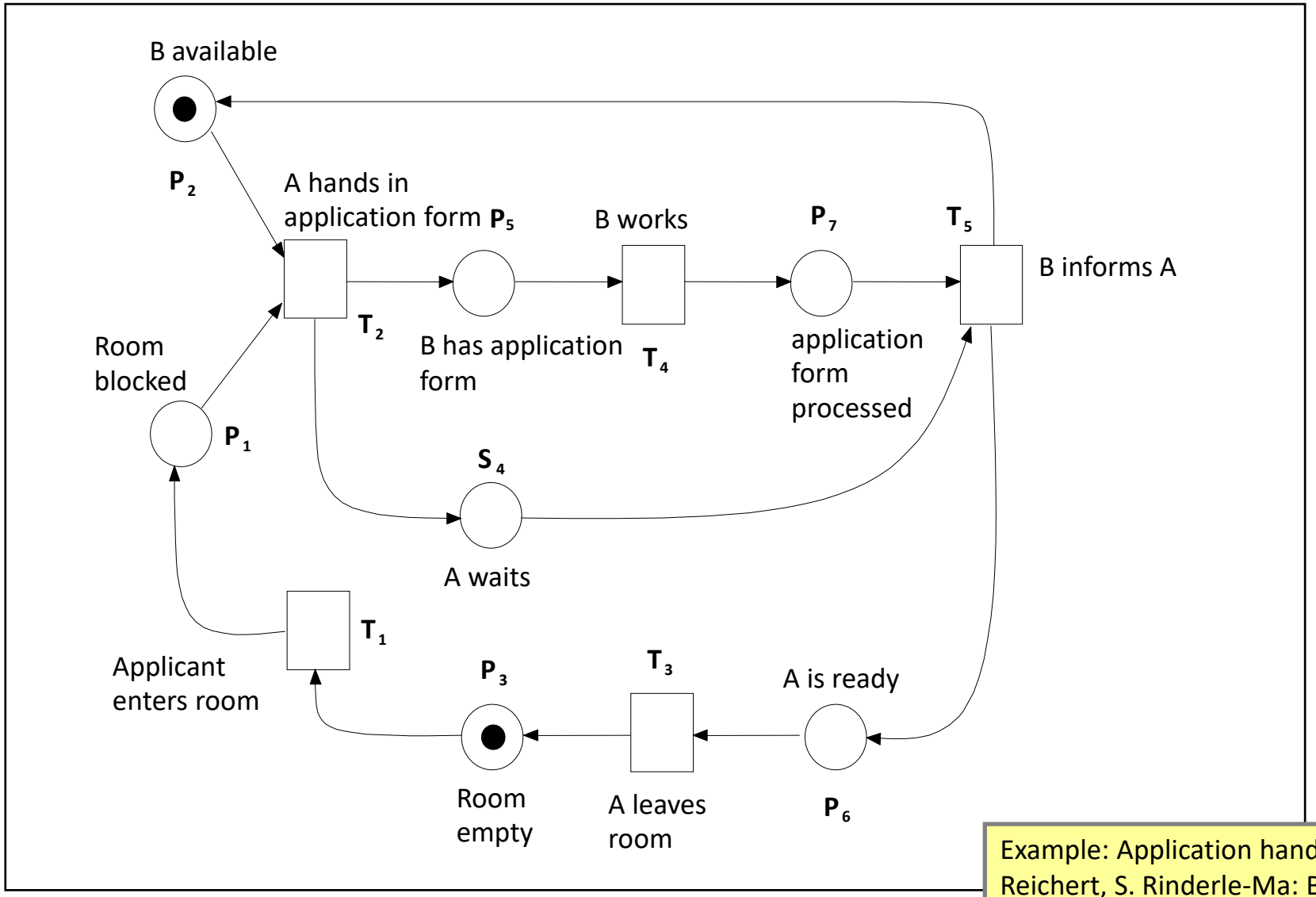
Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)

Structured analysis

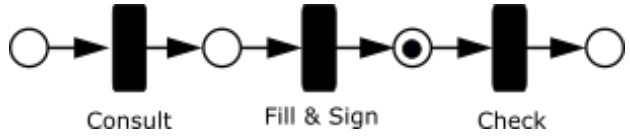
- Reachable execution states

Time	Places							Transitions				
	P1	P2	P3	S4	P5	P6	P7	T1	T2	T3	T4	T5
0		X	X					X				
1	X	X							X			
2				X	X						X	
3				X			X					X
4		X				X				X		
5		X	X									

Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)

Petri net based logging

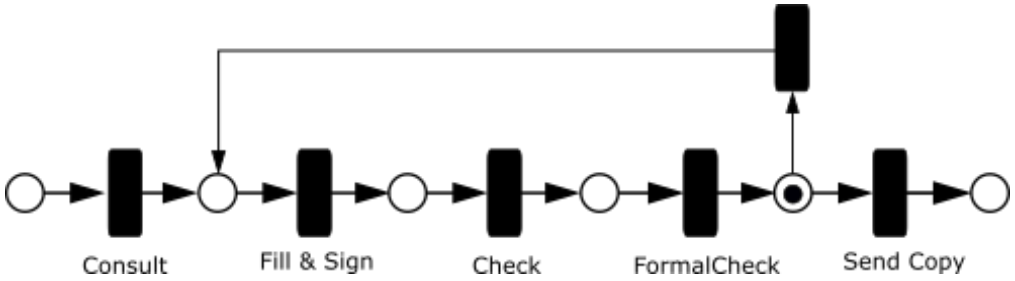
- The marking, i.e., the placement of tokens on the Petri net, signifies an execution state of a process instance.
- In which execution state is the Petri net ?



- Which transitions/activities have been executed is stored into a (process) execution log.
- Example execution log: `<Consult, Fill & Sign>`
- Execution logs enable to restore the execution state of process instances
- Execution logs provide the input for ***Process Mining***.

Petri net based logging

- Determine the execution state of the Petri net below:



- Can you determine a process execution log for this instance in a unique way?
- Which execution logs are possible?

Place/Transition Nets

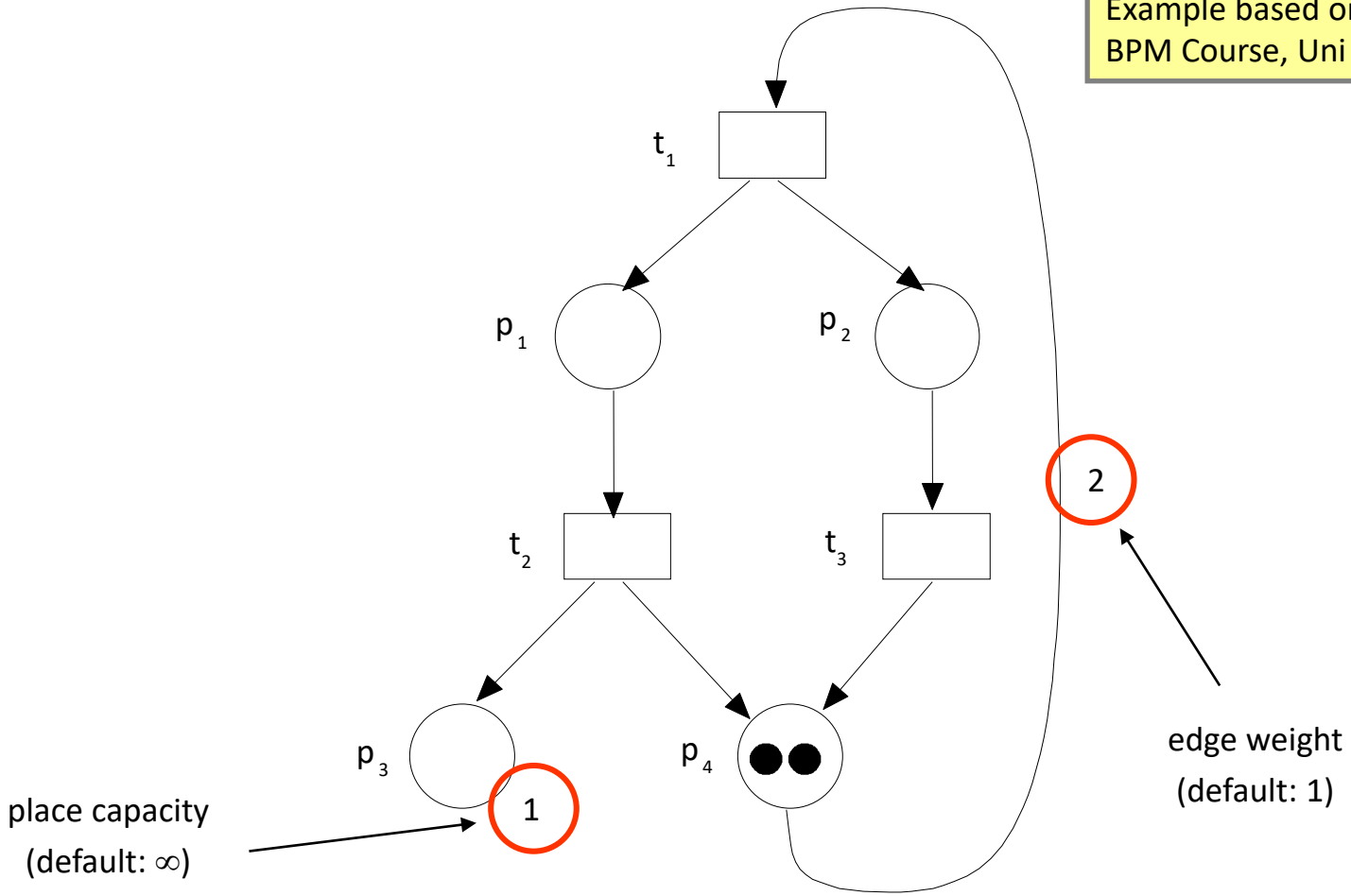
- E/C Nets are now extended by edge weights and place capacities
- Places can carry more than one token.
- Tokens remain anonymous.
- Useful to describe systems and processes with limited resources

DEFINITION 2.4 (Place/Transition Net): A tuple $Y = (P, T, F, C, W, M_0)$ is denoted as Place/Transition-Net (P/T-Net) if

- **(P, T, F) is a Net**
- **$C: P \rightarrow \mathbb{N} \cup \{\infty\}$ assigns capacities to places**
- **$W: F \rightarrow \mathbb{N}$ assigns weights to edges**
- **$M_0: P \rightarrow \mathbb{N}_0$ denotes the initial marking with $\forall\, p \in P : M_0(p) \leq C(p)$.**

Place/Transition Nets

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm



Place/Transition Nets

DEFINITION 2.5 (Firing Rules for P/T Nets)

- **A transition t of an P/T-Net is activated under a marking M (notation: M[t⟩) if**
  - **$\forall$ p $\in$ •t : M(p) $\geq$ W(p,t)**
  - **$\forall$ p $\in$ t• : M(p) + W(t,p) $\leq$ C(p)**
- **$M \xrightarrow{t} M'$ with:**

$$M'(p) = \begin{cases} M(p) - W(p,t), & \text{falls } p \in \bullet t \setminus t\bullet \\ M(p) + W(t,p), & \text{falls } p \in t\bullet \setminus \bullet t \\ M(p) - W(p,t) + W(t,p), & \text{falls } p \in t\bullet \cap \bullet t \\ M(p), & \text{sonst} \end{cases}$$

Place/Transition Nets: Reachability

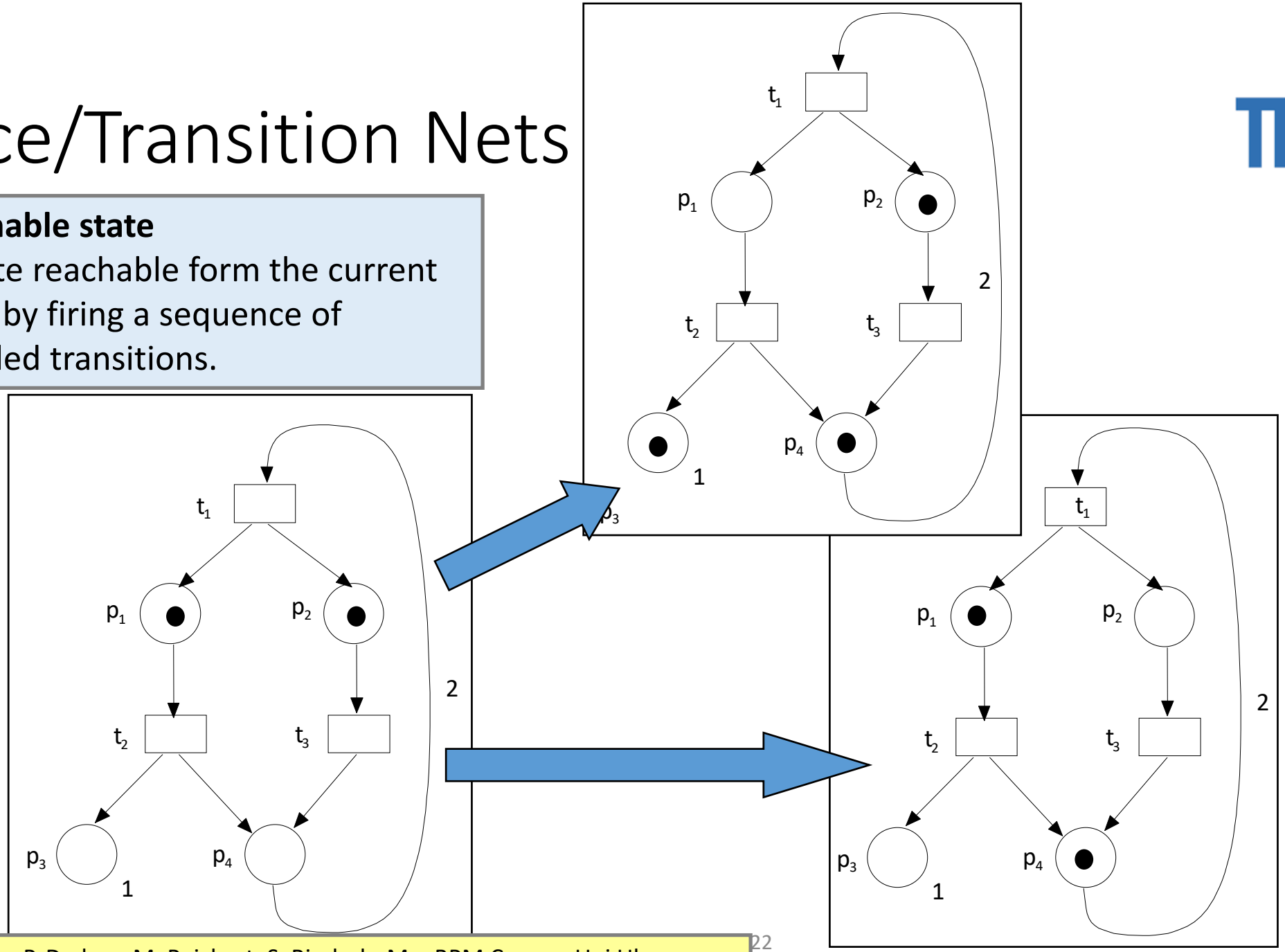
- Analysis of dynamic properties by P/T-Net-example
- General target: Harvesting knowledge on execution behavior
  - Deadlocks
  - Unbounded places
  - Non-reachable, yet desired states
- Very important for process-oriented applications: Knowing possible problems in advance, not when they are already happening!!

DEFINITION 2.6 (Reachability Set): Let M_0 be an initial marking. Then $[M_0\rangle$ denotes the set of all possible markings which can be reached starting from M_0 using the firing rules as defined before. $[M_0\rangle$ is denoted as reachability set of a P/T-Net.

For any marking M_i, $[M_i\rangle$ denotes the set of markings which can be reached from M_i.

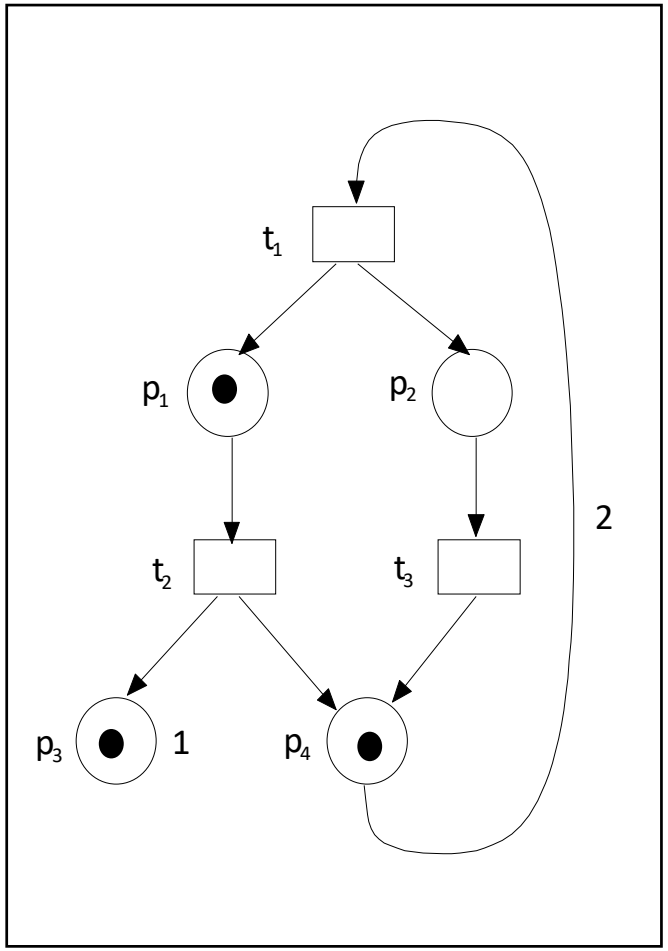
Place/Transition Nets

reachable state
A state reachable form the current state by firing a sequence of enabled transitions.



Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

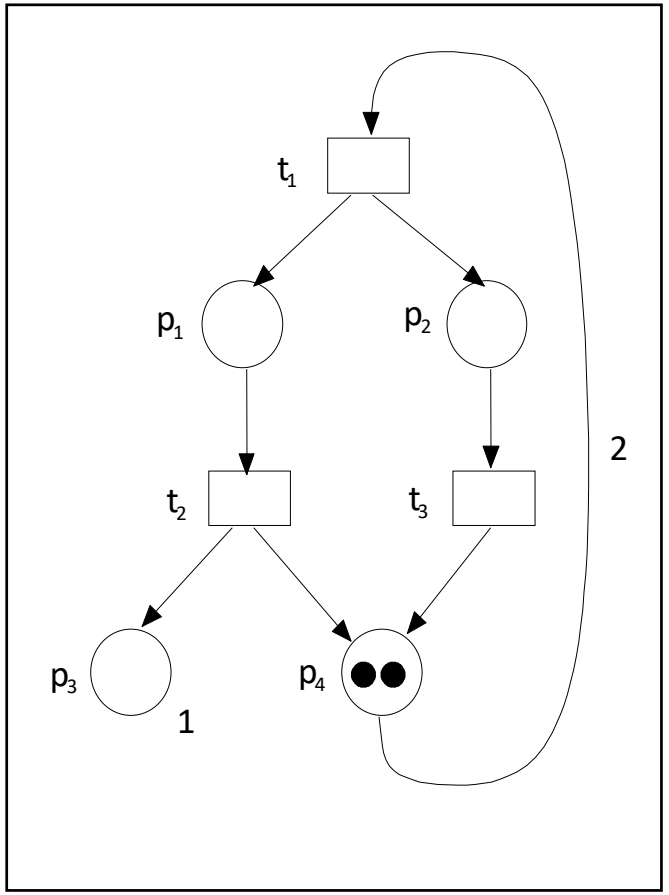
Place/Transition Nets



dead state
A state where no transition is enabled.

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

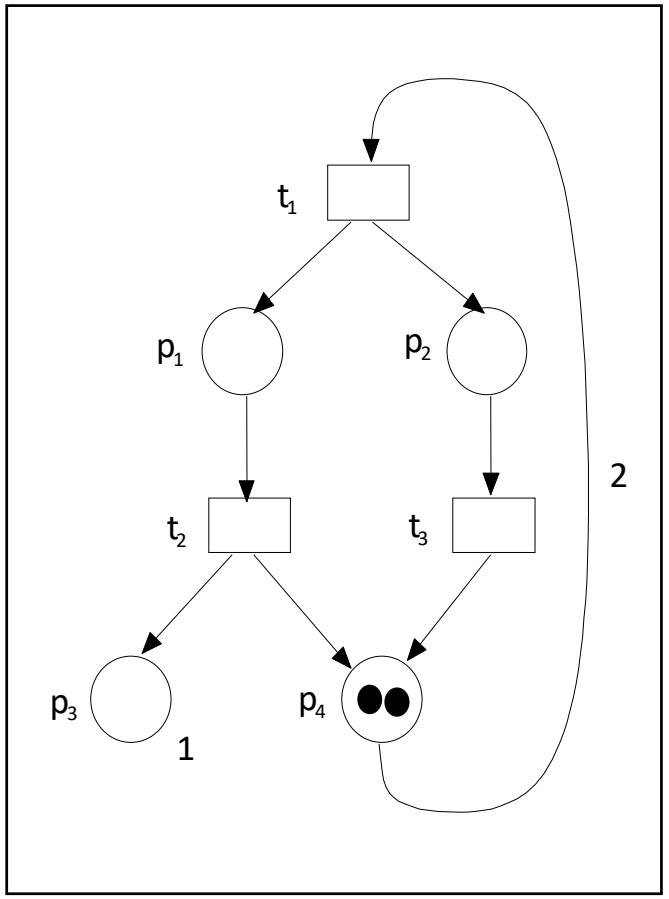


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

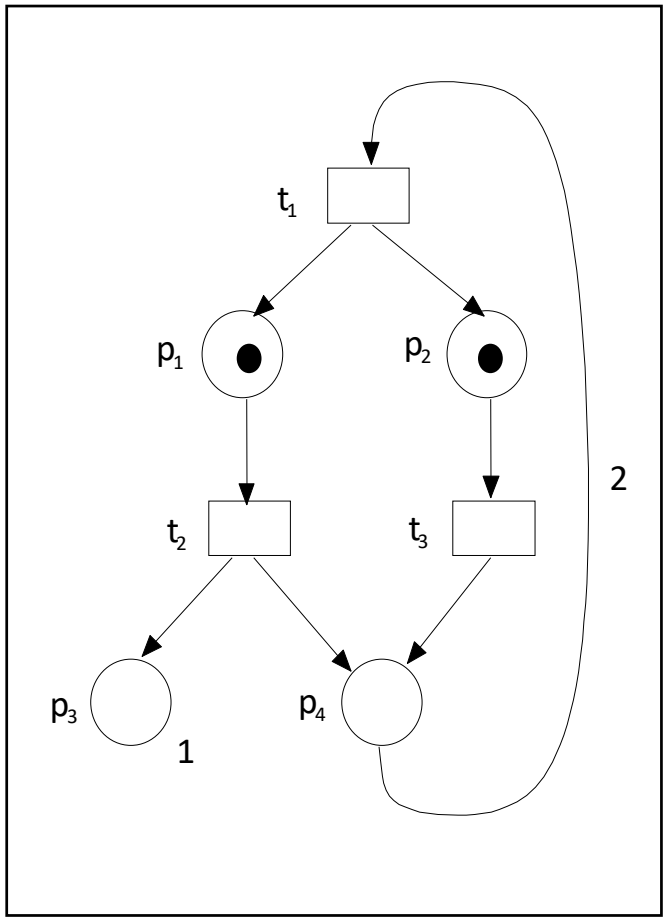


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1					

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

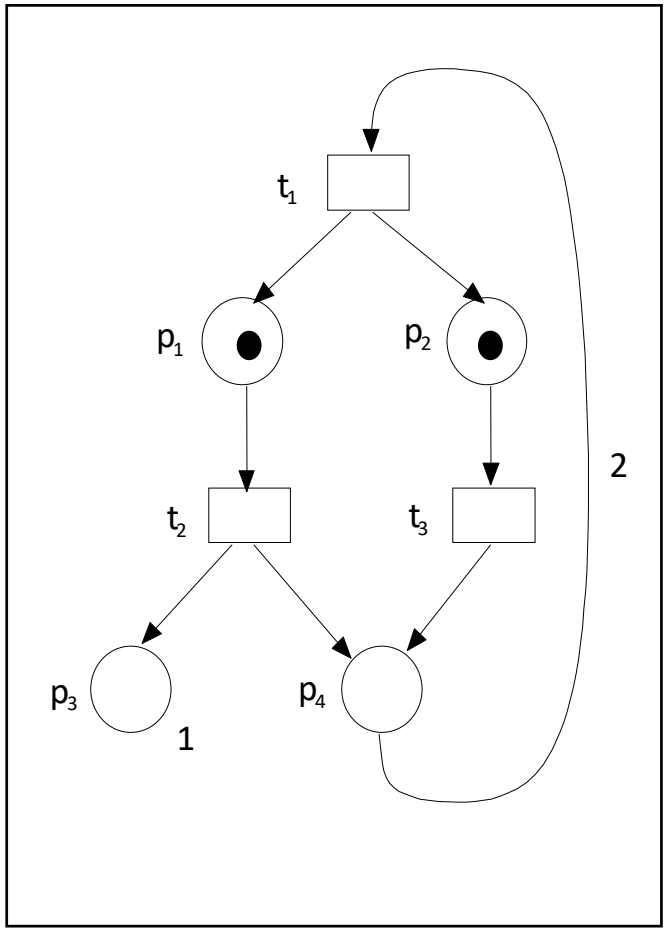


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1	1	1	0	0	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

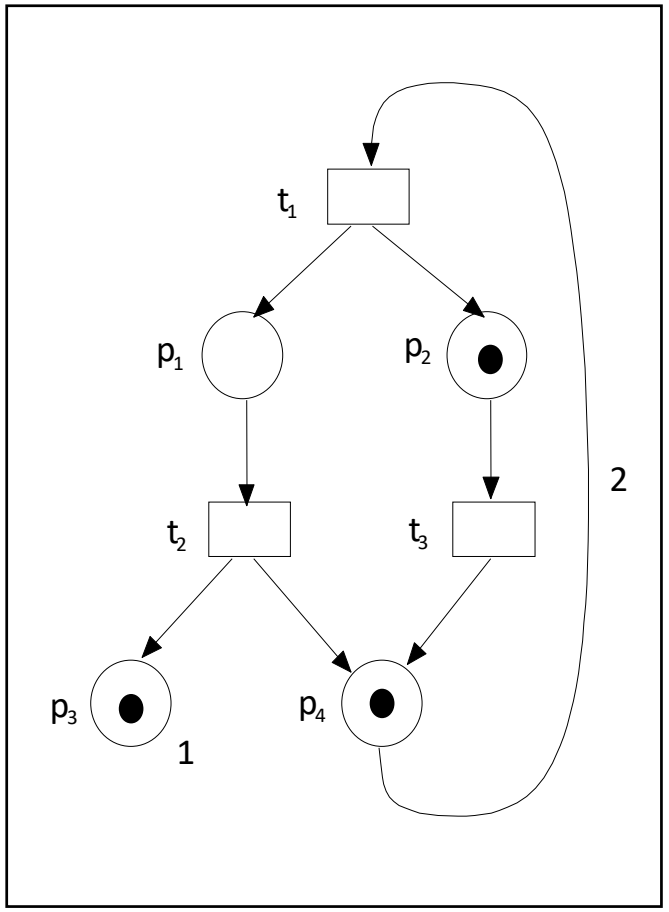


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2					
M_3					

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

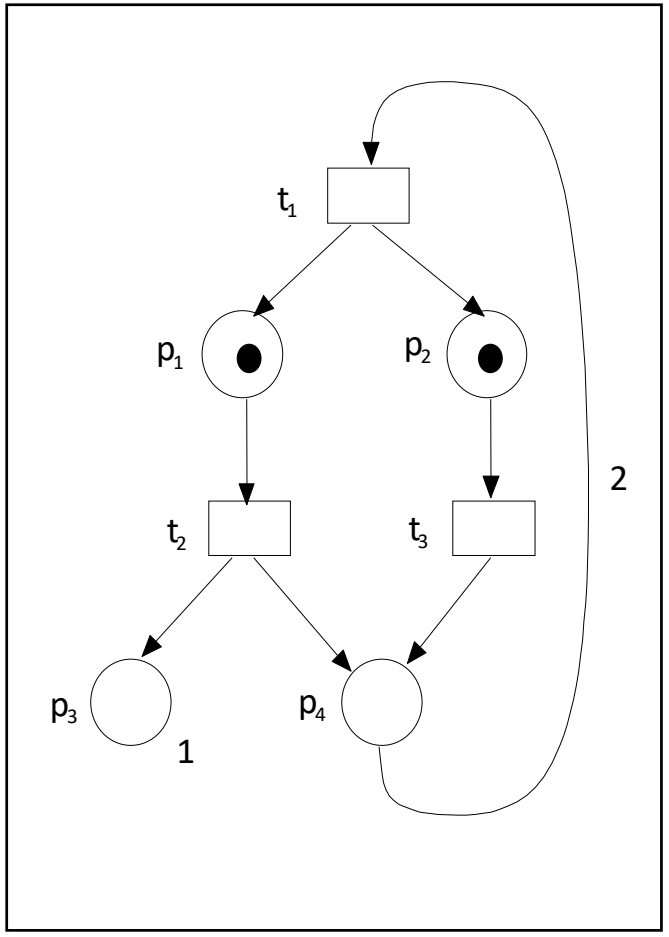


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2	0	1	1	1	
M_3					

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

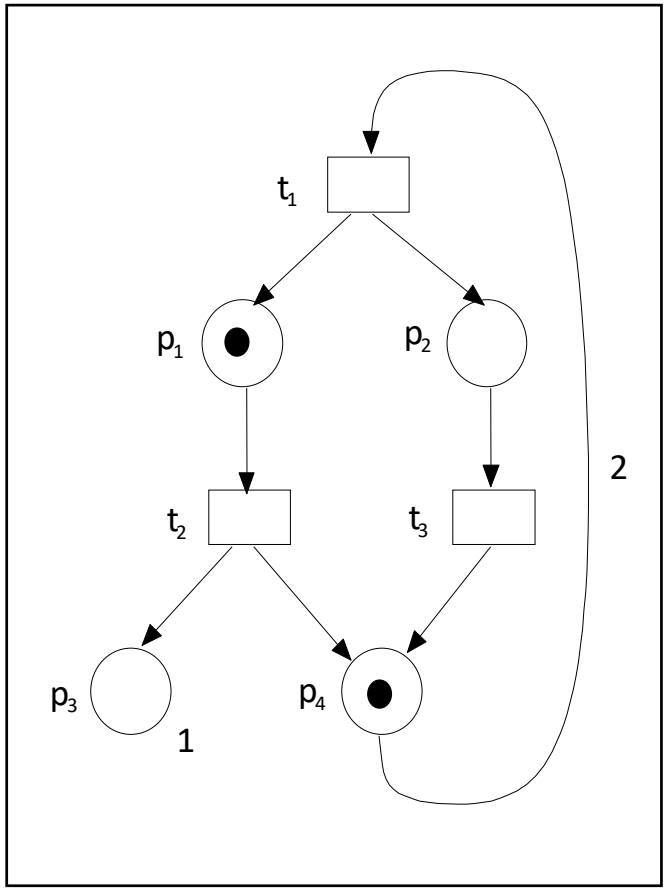


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2	0	1	1	1	
M_3					

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

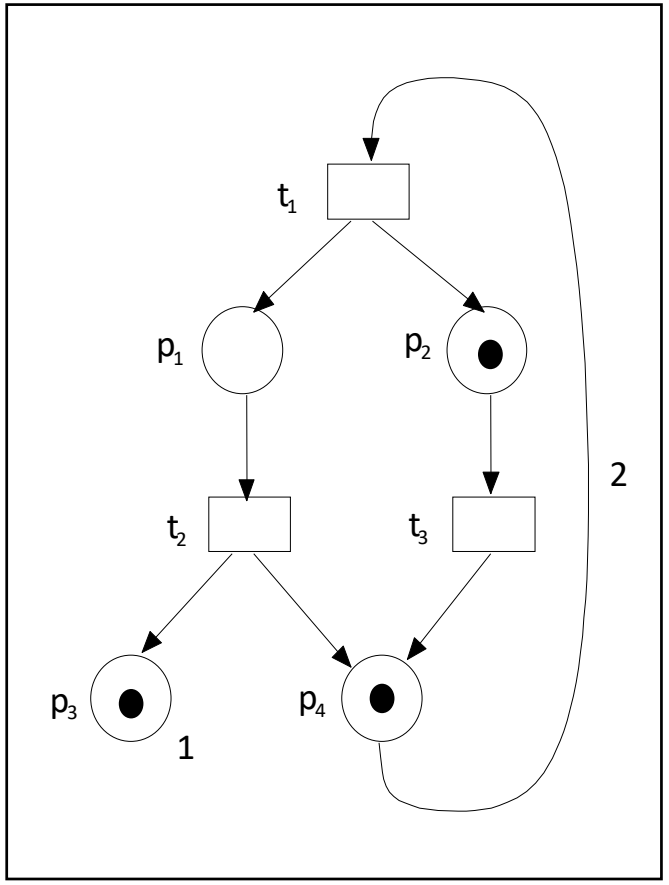


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2	0	1	1	1	
M_3	1	0	0	1	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

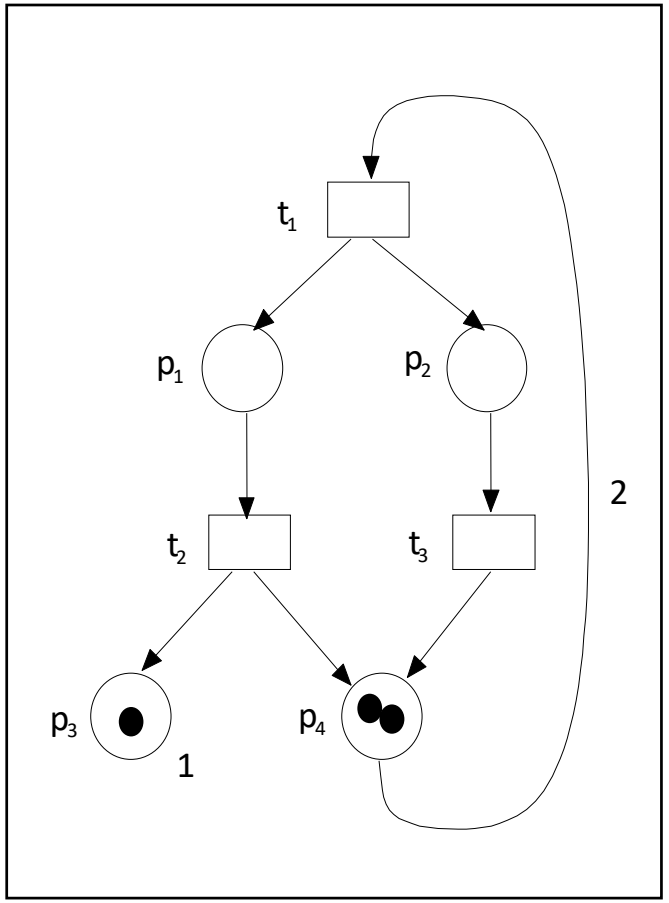


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1	1	1	0	0	$t2 \rightarrow M_2$ $t3 \rightarrow M_3$
M_2	0	1	1	1	$t3 \rightarrow M_4$
M_3	1	0	0	1	
M_4					

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

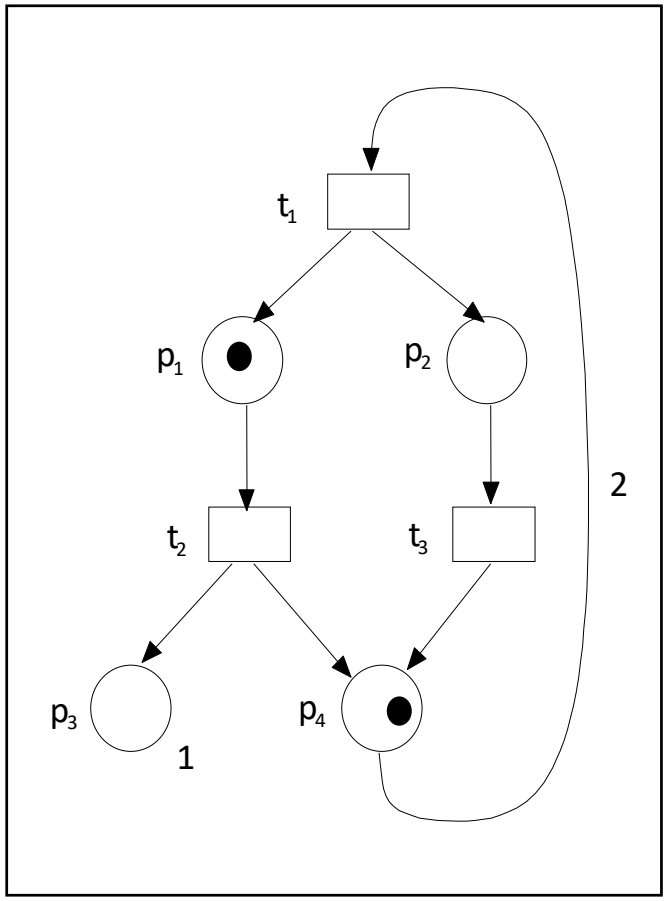


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1	1	1	0	0	$t2 \rightarrow M_2$ $t3 \rightarrow M_3$
M_2	0	1	1	1	$t3 \rightarrow M_4$
M_3	1	0	0	1	
M_4	0	0	1	2	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

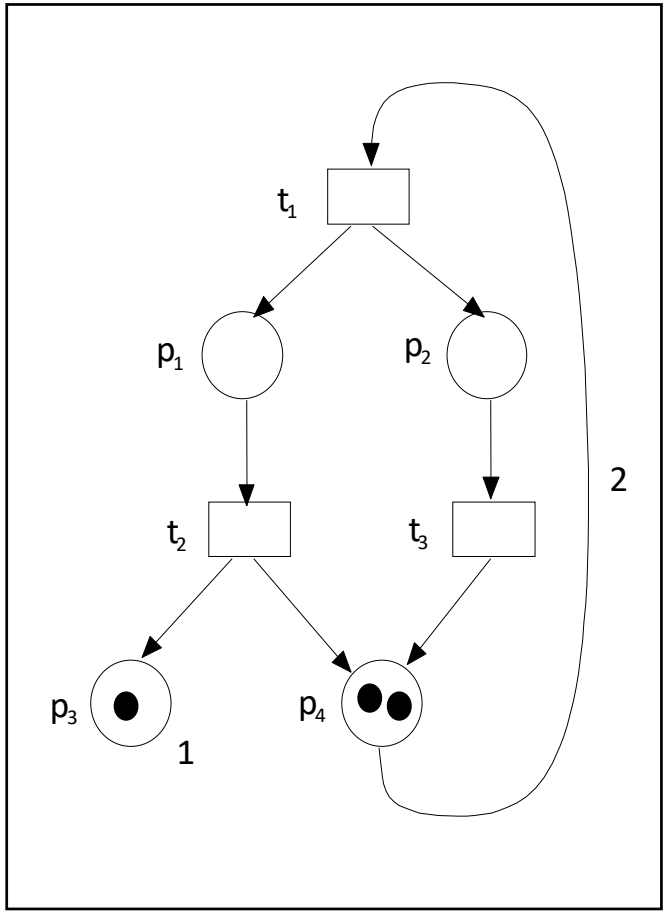


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2	0	1	1	1	t3 → M_4
M_3	1	0	0	1	t2 → M_4
M_4	0	0	1	2	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

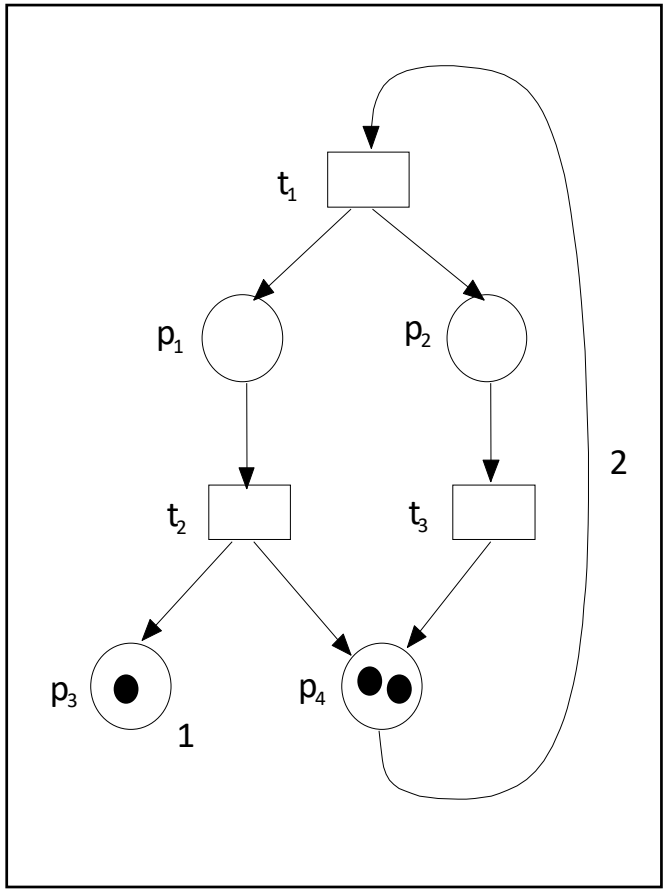


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1	1	1	0	0	$t2 \rightarrow M_2$ $t3 \rightarrow M_3$
M_2	0	1	1	1	$t3 \rightarrow M_4$
M_3	1	0	0	1	$t2 \rightarrow M_4$
M_4	0	0	1	2	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

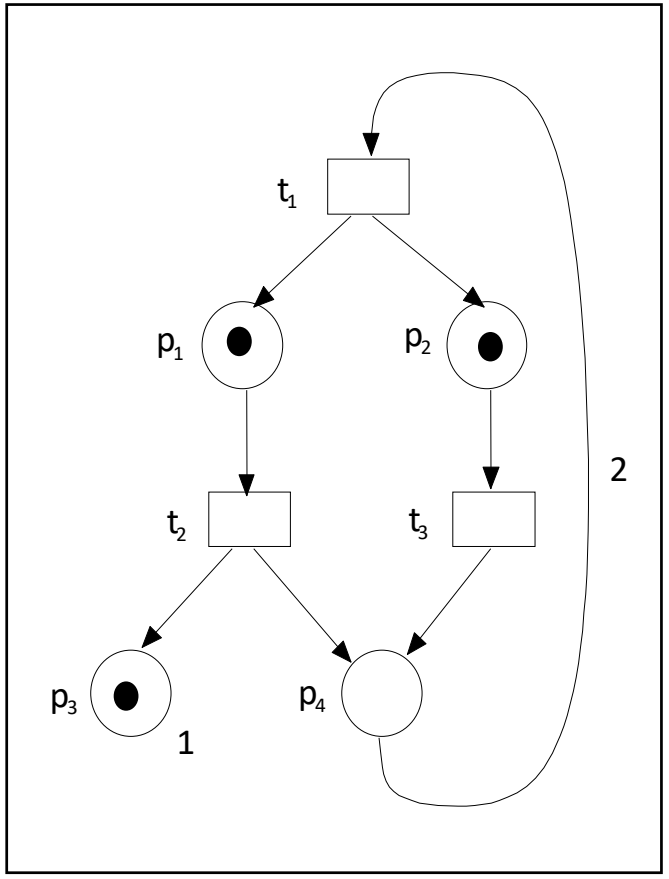


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1	1	1	0	0	$t2 \rightarrow M_2$ $t3 \rightarrow M_3$
M_2	0	1	1	1	$t3 \rightarrow M_4$
M_3	1	0	0	1	$t2 \rightarrow M_4$
M_4	0	0	1	2	$t1 \rightarrow M_5$
M_5					

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

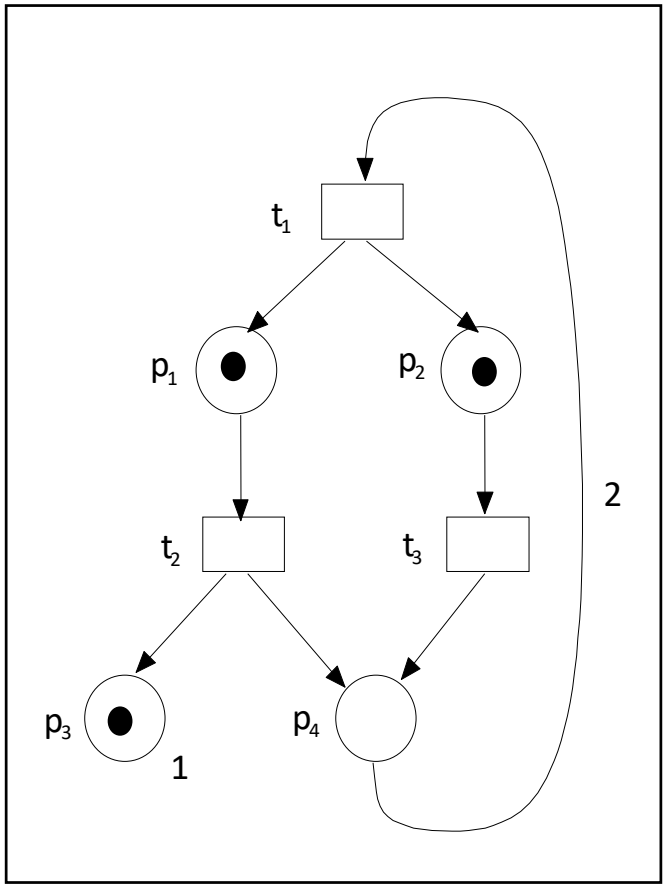


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1	1	1	0	0	$t2 \rightarrow M_2$ $t3 \rightarrow M_3$
M_2	0	1	1	1	$t3 \rightarrow M_4$
M_3	1	0	0	1	$t2 \rightarrow M_4$
M_4	0	0	1	2	$t1 \rightarrow M_5$
M_5	1	1	1	0	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

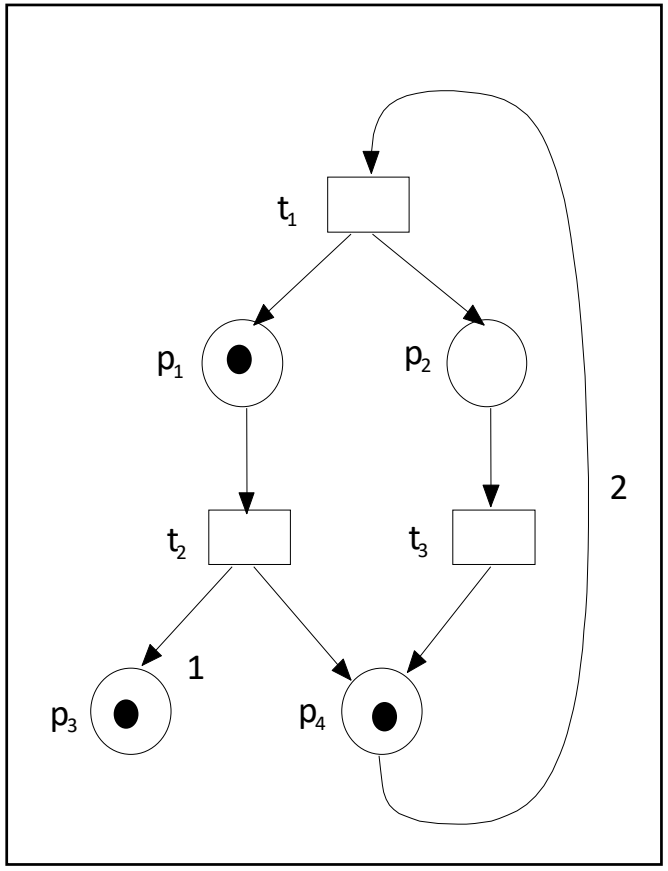


Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2	0	1	1	1	t3 → M_4
M_3	1	0	0	1	t2 → M_4
M_4	0	0	1	2	t1 → M_5
M_5	1	1	1	0	t3 → M_6
M_6					

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2	0	1	1	1	t3 → M_4
M_3	1	0	0	1	t2 → M_4
M_4	0	0	1	2	t1 → M_5
M_5	1	1	1	0	t3 → M_6
M_6	1	0	1	1	

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

Reachability Graph

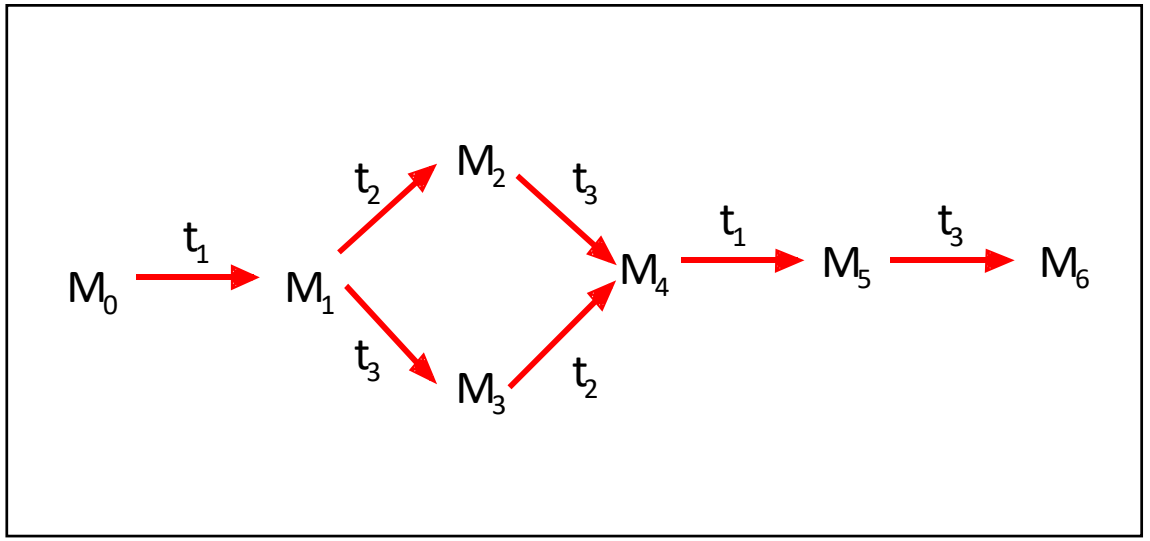
- The reachability graph of a Petri net is the part of the transition system reachable from the initial state in graph-like notation.
- The reachability graph can be calculated as follows:
  - Let X be the set containing just the initial state and let Y be the empty set.
  - Take an element x of X and add this to Y. Calculate all states reachable for x by firing some enabled transition. Each successor state that is not in Y is added to X.
  - If X is empty stop, otherwise goto 2.

DEFINITION 2.7 (Reachability Graph): The reachability graph G(Y) of a P/T-Net Y is defined as follows:

$$G(Y) := \{(M_1, t, M_2) \in [M_0\rangle \times T \times [M_0\rangle \mid M_1 \xrightarrow{t} M_2\}$$

Place/Transition Nets

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	t1 → M_1
M_1	1	1	0	0	t2 → M_2 t3 → M_3
M_2	0	1	1	1	t3 → M_4
M_3	1	0	0	1	t2 → M_4
M_4	0	0	1	2	t1 → M_5
M_5	1	1	1	0	t3 → M_6
M_6	1	0	1	1	



Reachability Graph

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets

DEFINITION 2.8 (Boundedness): Let Y = (P, T, F, C, W, M_0) be a P/T-Net and $k \in \mathbb{N}$ a natural number. Then:

- **Y is denoted as *bounded* if $\forall M \in [M_0\rangle, p \in P : M(p) \leq k$**

Liveness[1]

- $[M_0\rangle$ all reachable markings from M_0
- $L(M_0)$ all possible firing sequences from M_0

A petri net is L_k-Live if all transitions are L_k-Live with k = 0,1,2,3,4

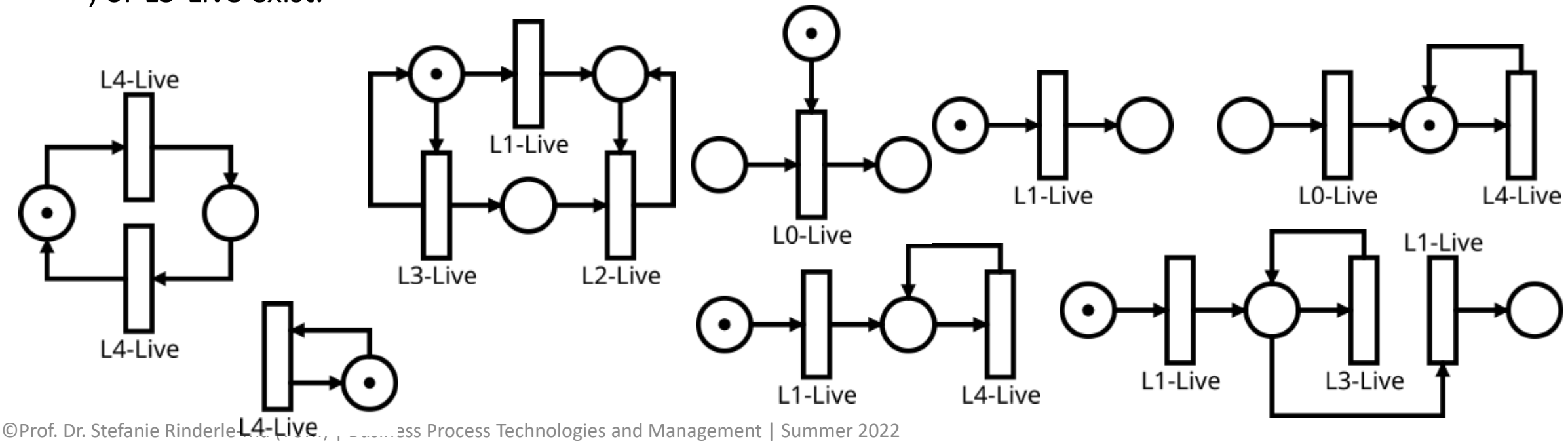
A transition is

- L0-Live: the transition can never fire in any $L(M_0)$
- L1-Live: the transition can fire at least once for a sequence in $L(M_0)$
- L2-Live: the transition can fire a finite number of times for a sequence in $L(M_0)$
- L3-Live: the transition can fire an infinite number of times for a sequence in $L(M_0)$
- L4-Live: the transition can fire an infinite number of times for $[M_0\rangle$

[1] Murata, Tadao. "Petri nets: Properties, analysis and applications." Proceedings of the IEEE 77.4 (1989): 541-580. http://www2.ing.unipi.it/~a009435/issw/extra/murata.pdf

Liveness

- A petri net in M_0 is live or strong-live: all transitions are L4-Live
- A petri net in M_0 is weak-live or quasi-live: at least one transition is L4-Live, transitions which are L0-, L1-, L2-, or L3-Live exist.
- A petri net in M_0 has a livelock: at least one transition is L3-Live, transitions which are L0-, L1-, L2-, or L3-Live exist.



Place/Transition Nets

- **Different kinds of states:**
  - **Initial state**: Initial distribution of tokens.
  - **Reachable state**: Reachable from initial state.
  - **Final state** (also referred to as “dead states”): No transition is enabled.
  - **Home state** (also referred to as home marking): It is always possible to return (i.e., it is reachable from any reachable state).
- *How to recognize these states in the reachability graph?*

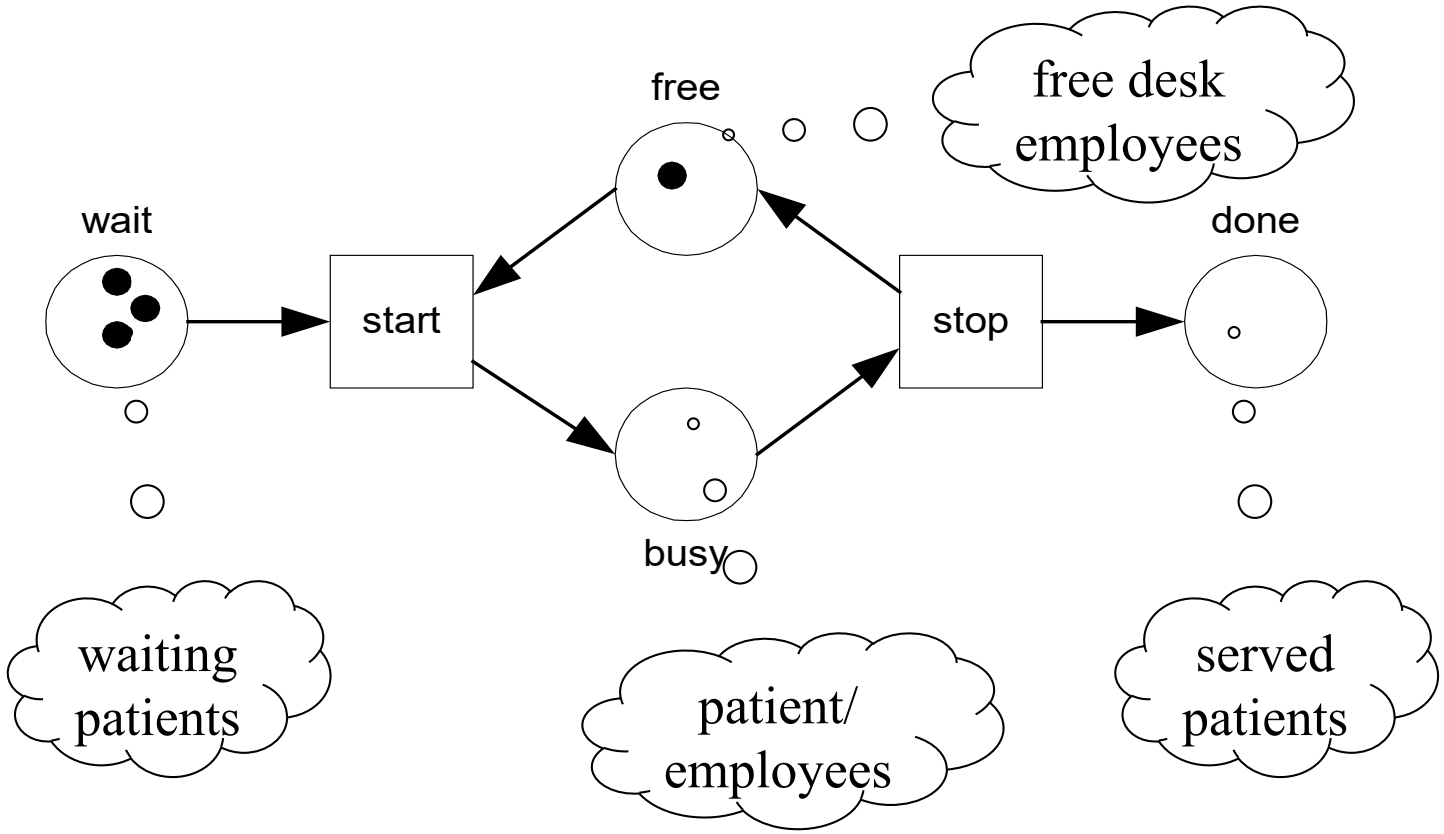
High-Level Petri Nets

- Limitations of classical Petri Nets:
  - Too large and complex
  - Takes too much time to model a given situation
  - It is not possible to handle time and data

- Petri nets extended with:
  - Color (i.e., data)
  - Time
  - Hierarchy

Colored Petri Nets (CPN)

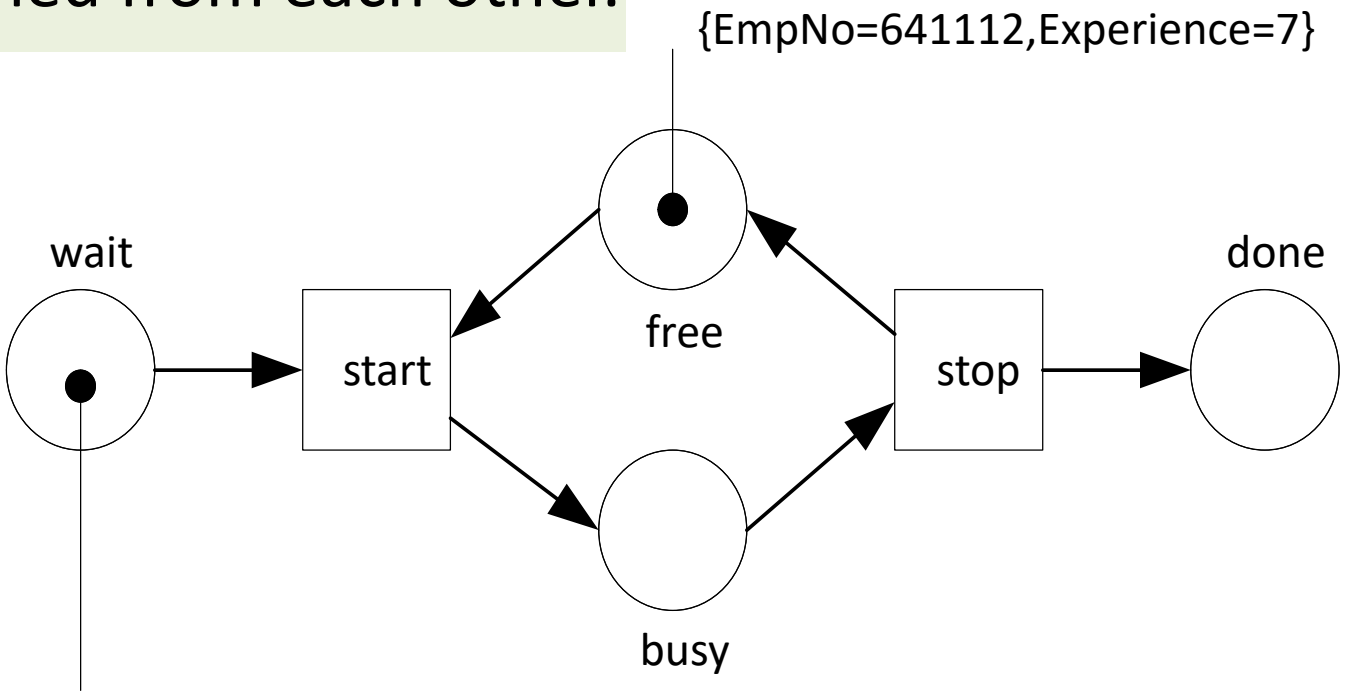
In classical Petri Nets tokens cannot be distinguished



Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Colored Petri Nets (CPN)

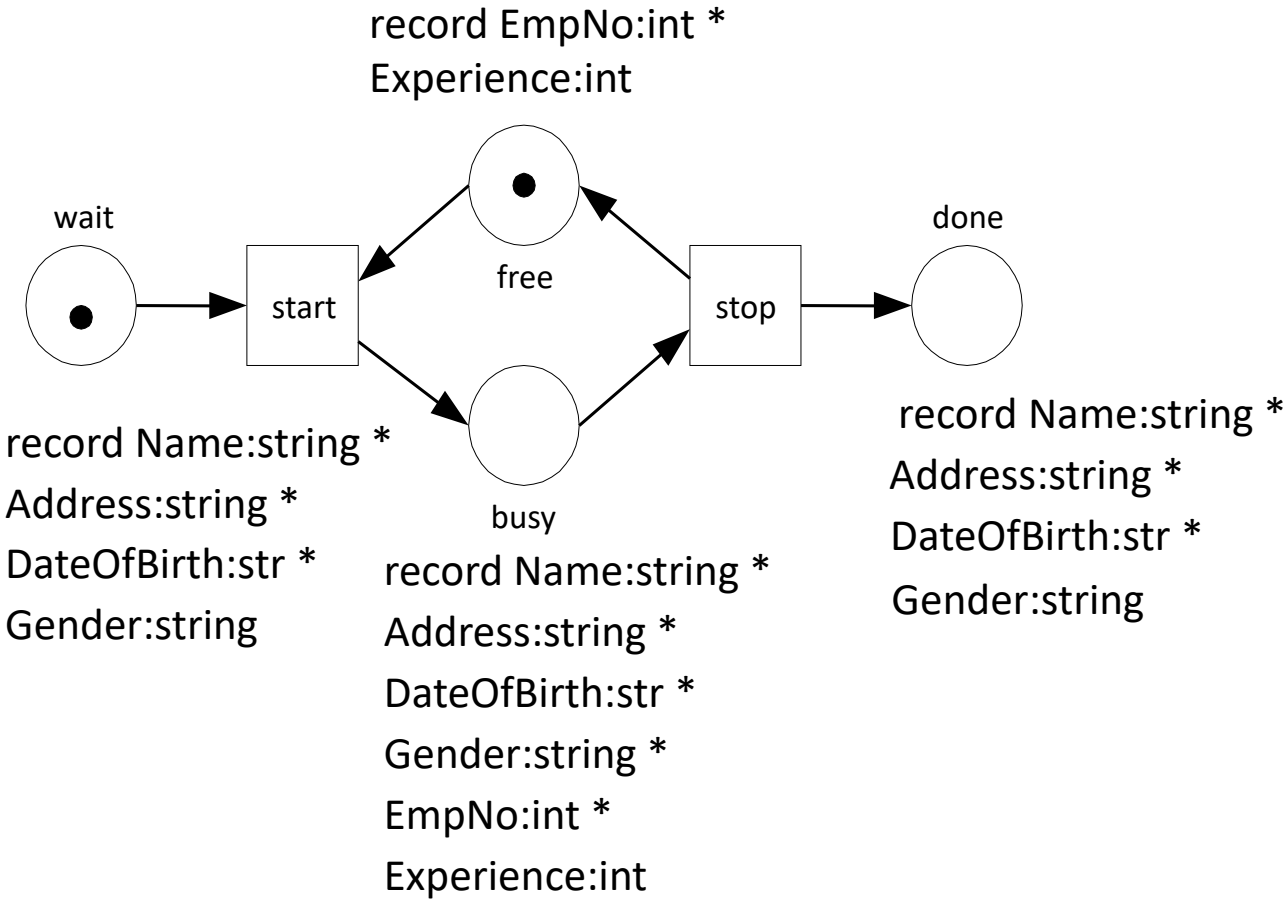
Each token is provided with a **value** or **color** and can therefore be distinguished from each other.



Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Colored Petri Nets

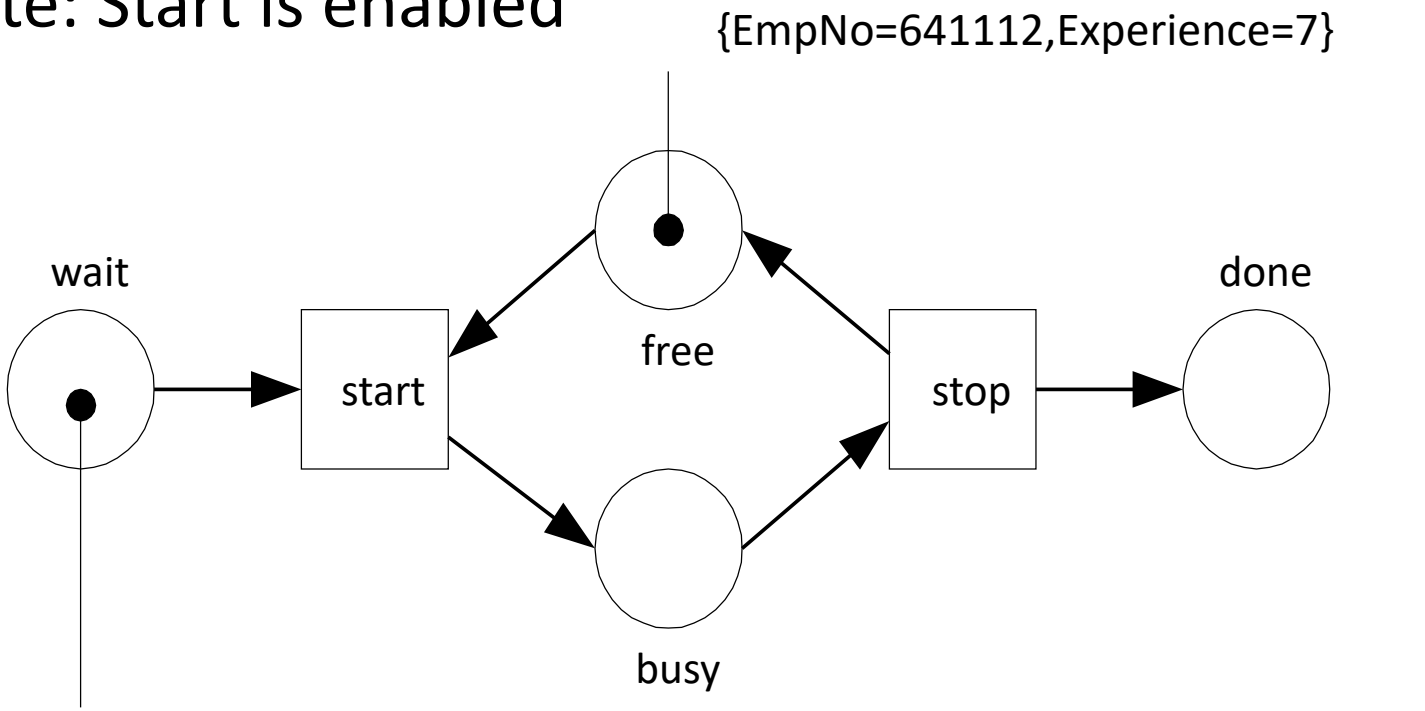
- Places are typed
- also referred to as color sets



Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Colored Petri Nets

Initial State: Start is enabled

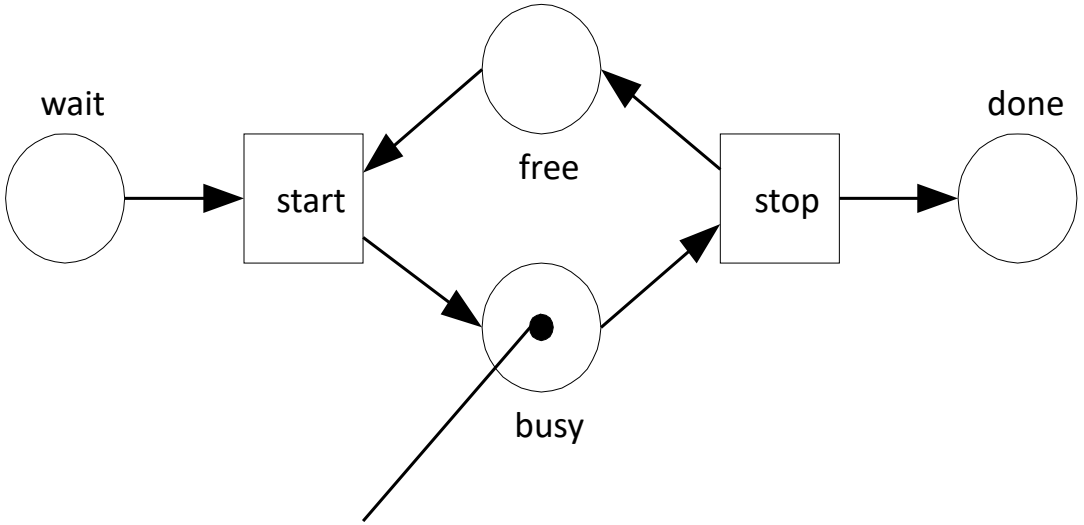


Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Colored Petri Nets

Transition start fired

New value is created by simply merging the two records

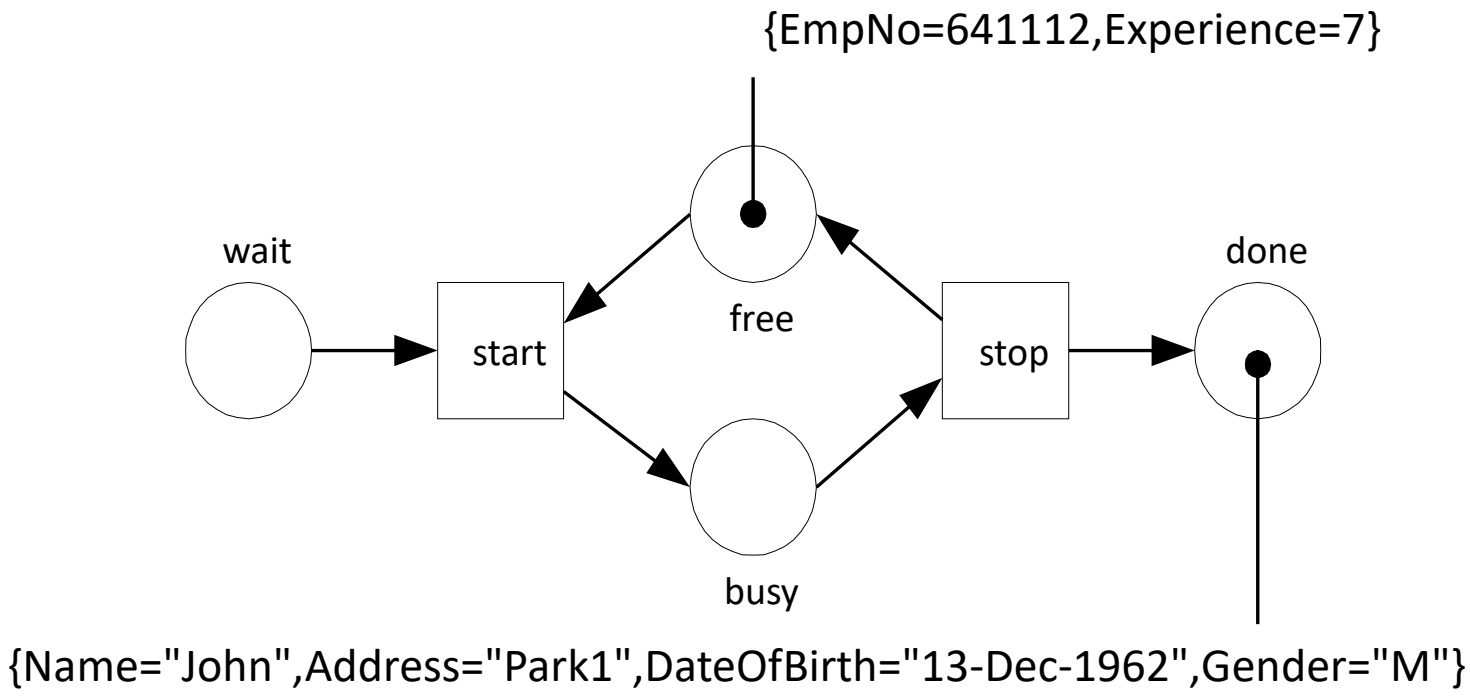


{Name="John",Address="Park1",DateOfBirth="13-Dec-1962",Gender="M",EmpNo=641112,Experience=7}

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Colored Petri Nets

- Transition stop fired
- New values are created by simply splitting the two records in two parts



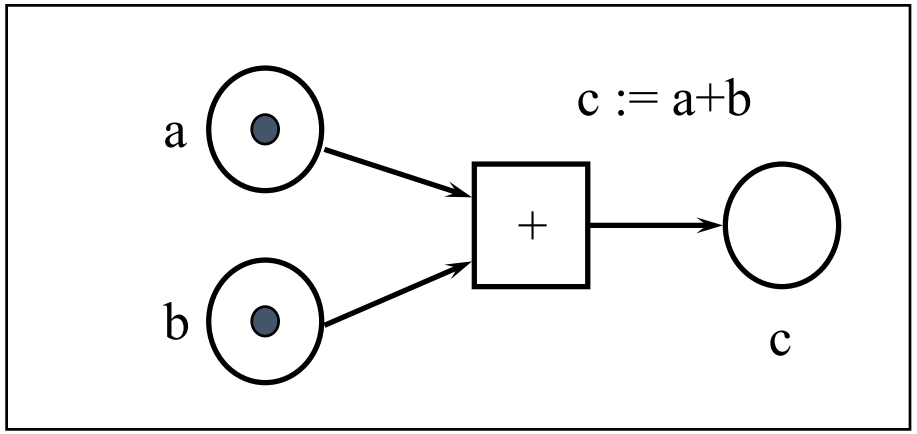
Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

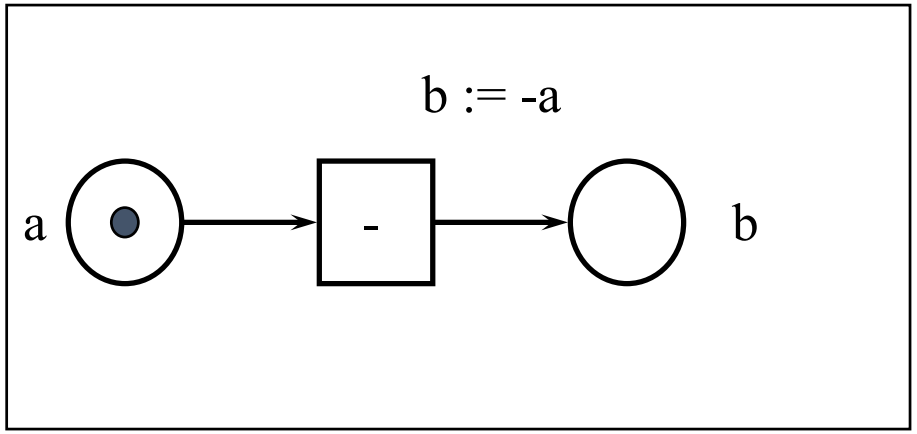
Colored Petri Nets

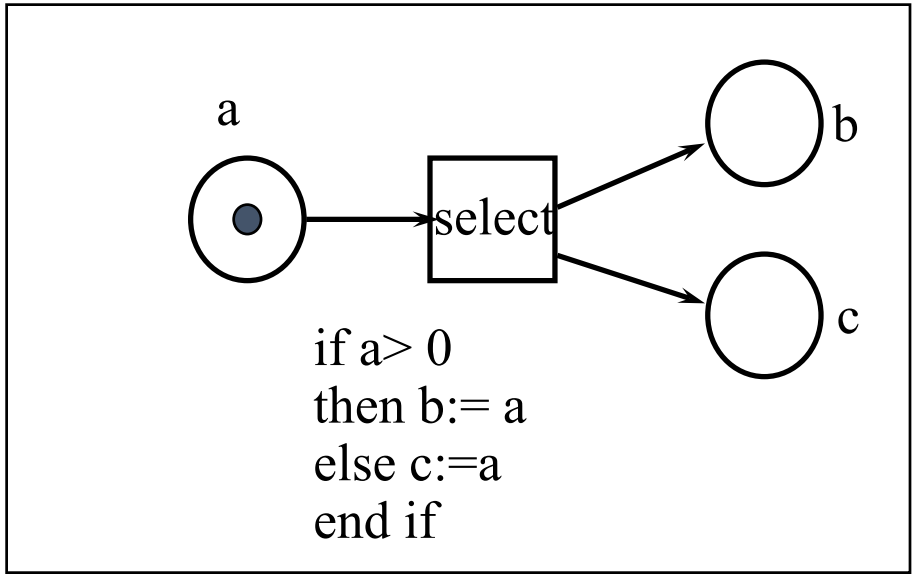
- Conditions for the values of the tokens to be consumed can be set
- A transition is then only enabled when there is at least one token at each input place and the preconditions have been met
- Each transition has an (in)formal specification which specifies:
  - the number of tokens to be produced,
  - the values of these tokens,
  - and (optionally) a precondition.

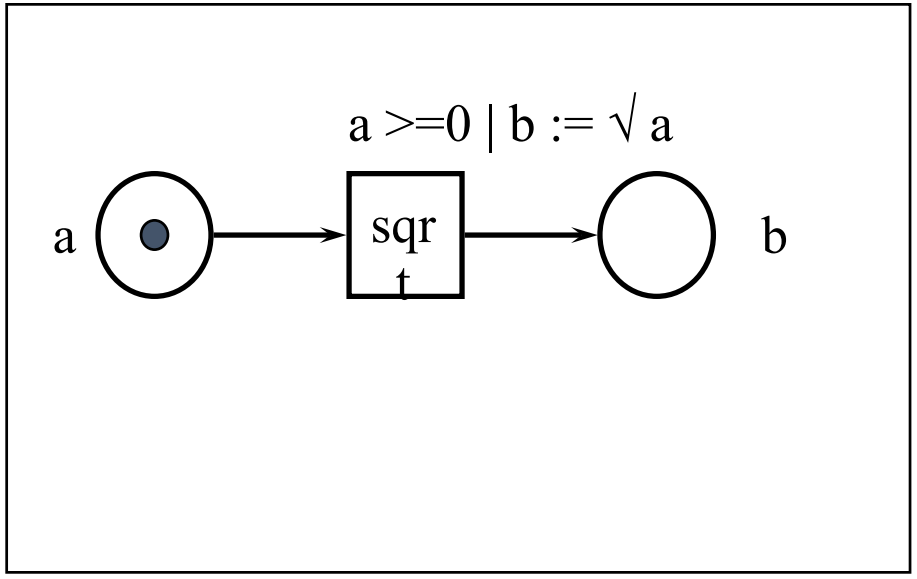
Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Colored Petri Nets







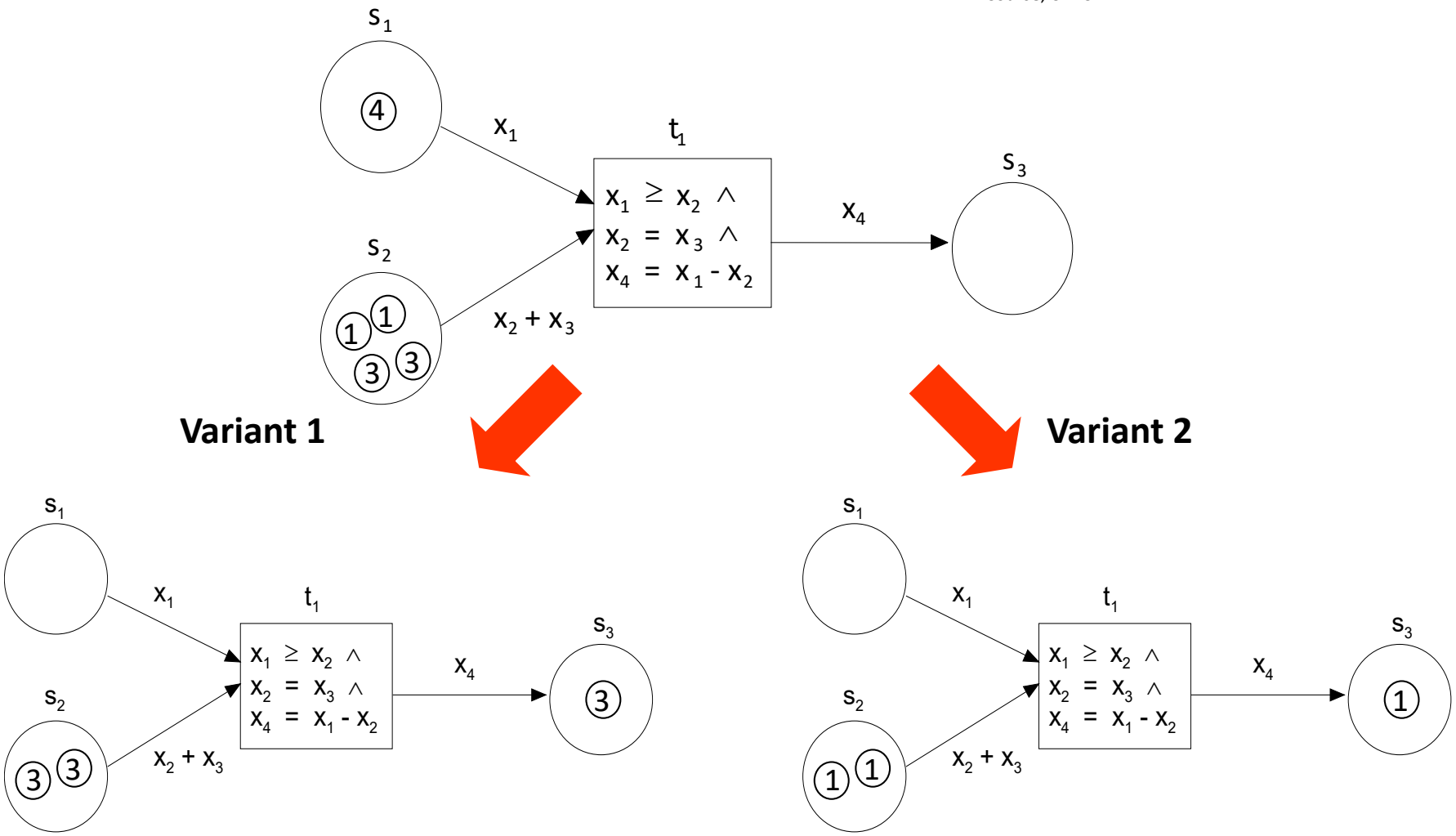


Color Extension exercise: calculate √ |a+b| using these building blocks

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Predicate/Transition Nets

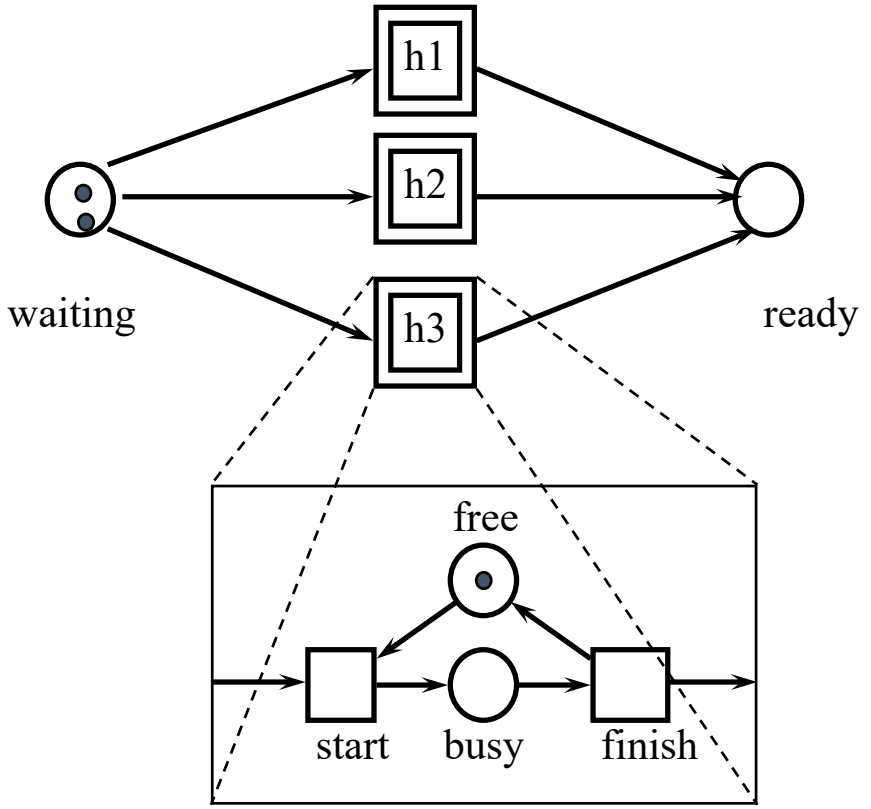
Slide based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm



Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

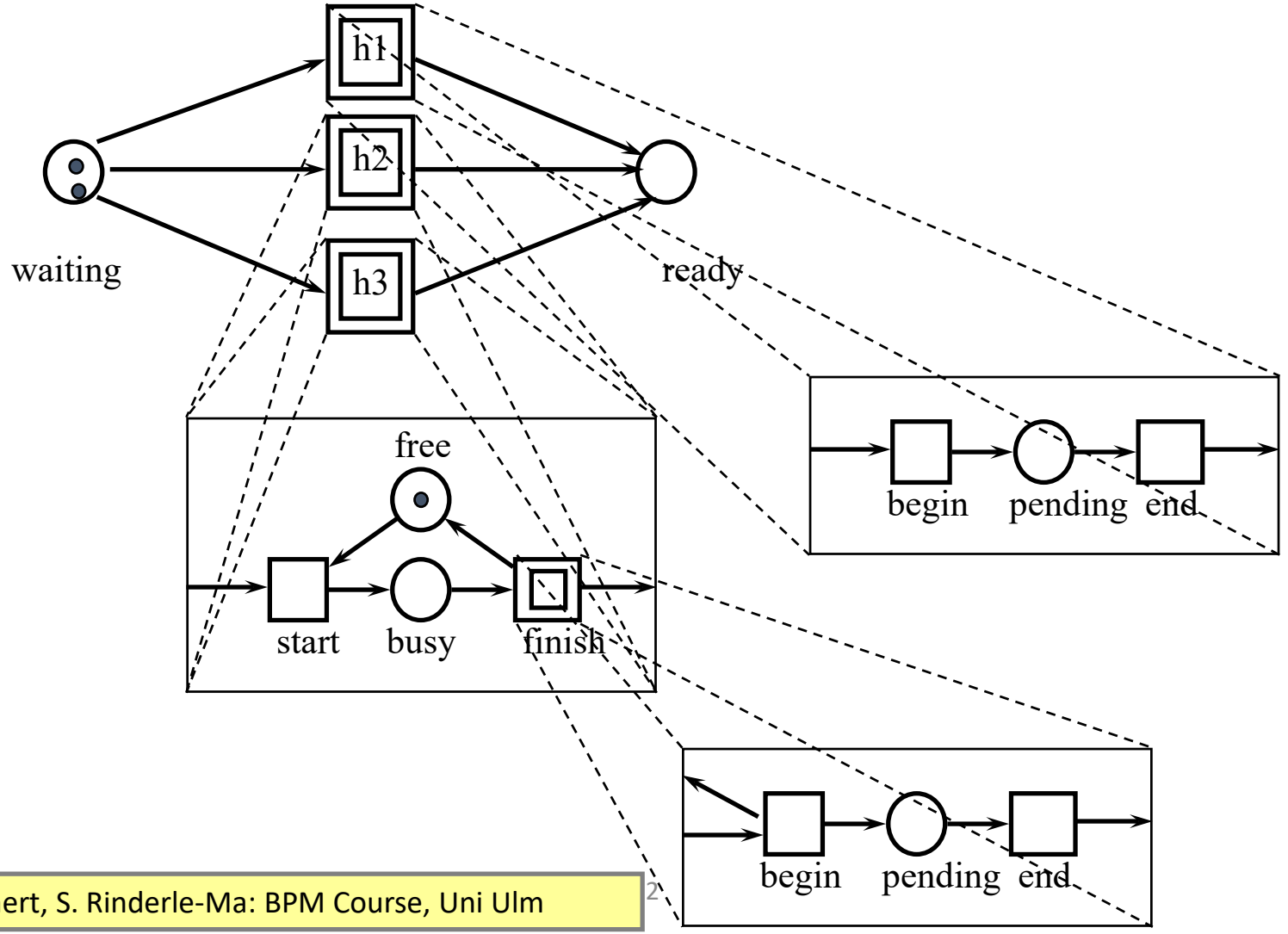
Hierarchy Extension

- A mechanism to structure complex Petri nets
- A subnet is a net composed out of places, transitions and subnets.



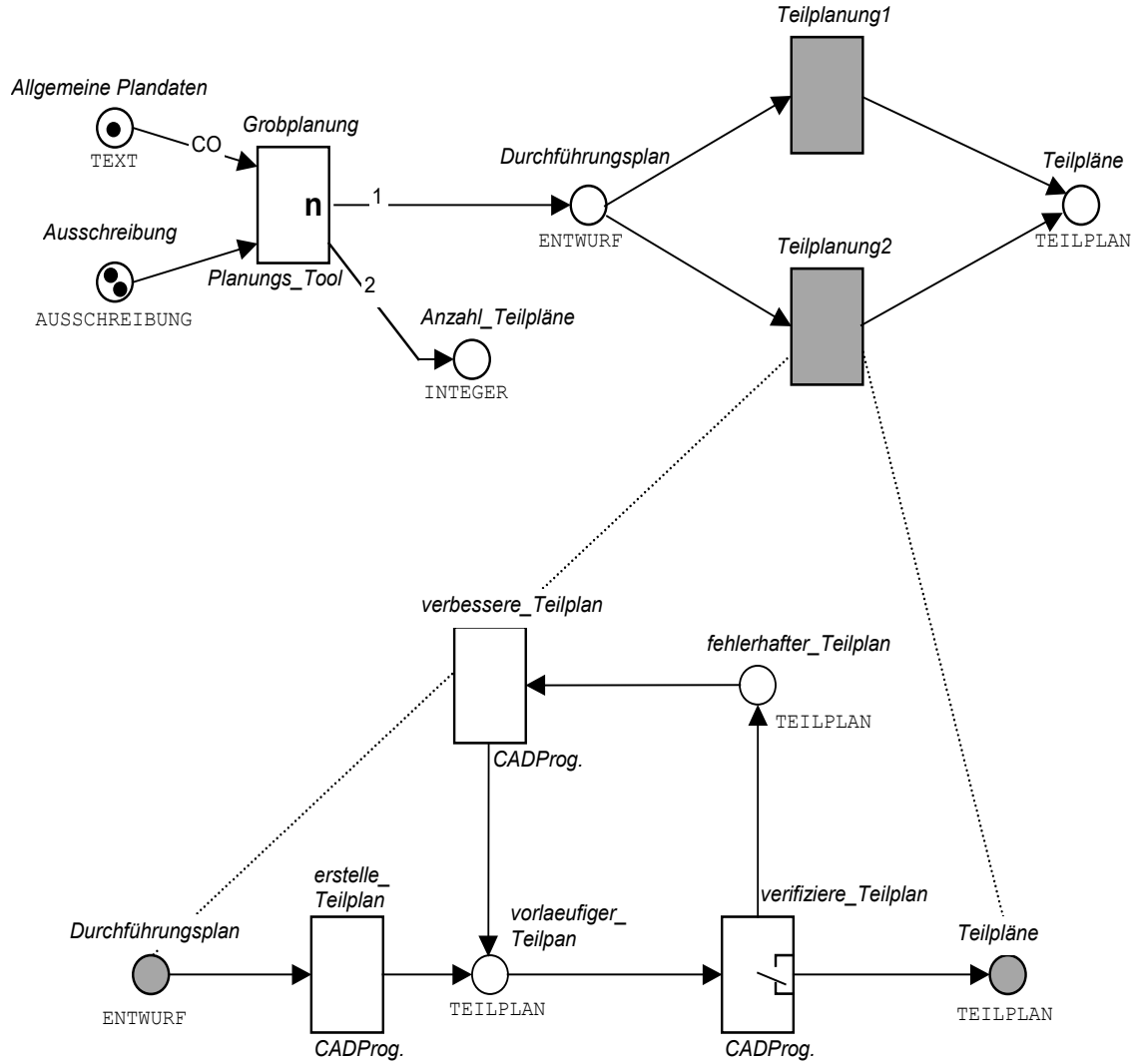
Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Hierarchy Extension



Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

FunSoft Nets



Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Workflow Nets

- Why Workflow Nets?
- Our objective: Modelling processes / workflows
- Classical Petri Nets are
  - Not expressive enough on the one hand side: Tokens should carry information
  - Too expressive at the other hand side: Possibility to model incorrect workflows
- Plus need for
  - Formal semantics
  - Graphical representation
  - Analysis of process properties
  - Tool independence

Partly based on: Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures. Springer 2019

Workflow Nets

DEFINITION 2.9 (Workflow Net): A Petri net PN = (P, T, F) is called Workflow Net (WF Net for short) if and only if the following conditions holds:

- There is a distinguished place i $\in$ P (called initial place) that has no incoming edge, i.e., $\bullet i = \varnothing$
- There is a distinguished place o $\in$ P (called final place) that has no outgoing edge, i.e., $o\bullet = \varnothing$
- Every place and every transition is located on a path from the initial place to the final place

- *Consequences*:
  - i is the only place without incoming edges
  - o is the only place without outcoming edges
- WHY?

Partly based on: Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures. Springer 2019

Workflow Nets

- Example: Sequence “First A, then B“

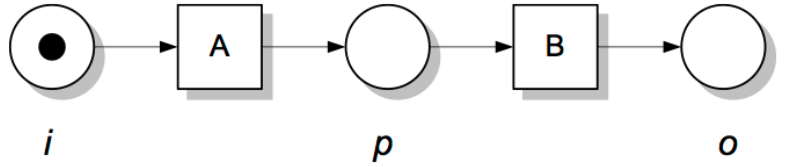


Fig. 4.42. Sequence pattern in workflow net

- Example: Parallelism “B and C can be processed in arbitrary order”

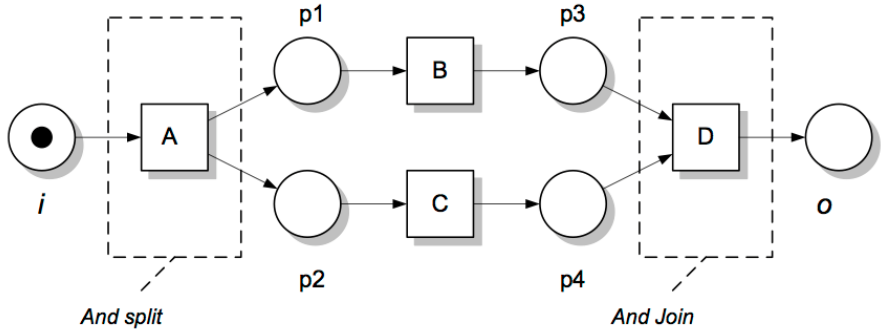


Fig. 4.43. *And split* and *and join* patterns in workflow nets

Workflow Nets

- Example: Conditional Paths “Either A or B” → implicit choice (depending on whether A or B fires first)

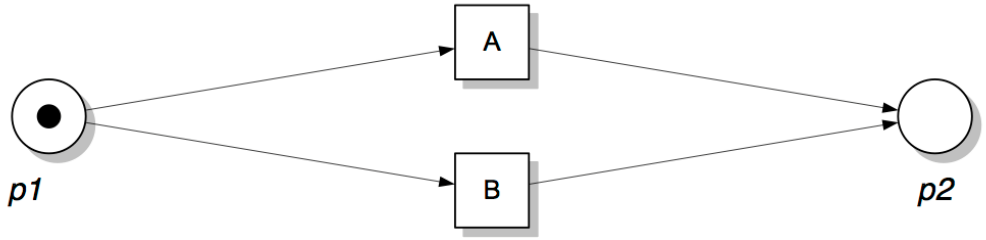


Fig. 4.44. *Implicit exclusive or split* also known as *deferred choice*

- Explicit choice by using a decision transition → Transition implements decision rule
- Exclusive Or: one and only one condition evaluates to True
- Use of default branches to avoid that workflow gets stuck

Workflow Nets

Workflow Nets [Aals07] provide syntactical and semantical extensions in order to allow for more compact models (syntactic sugaring)

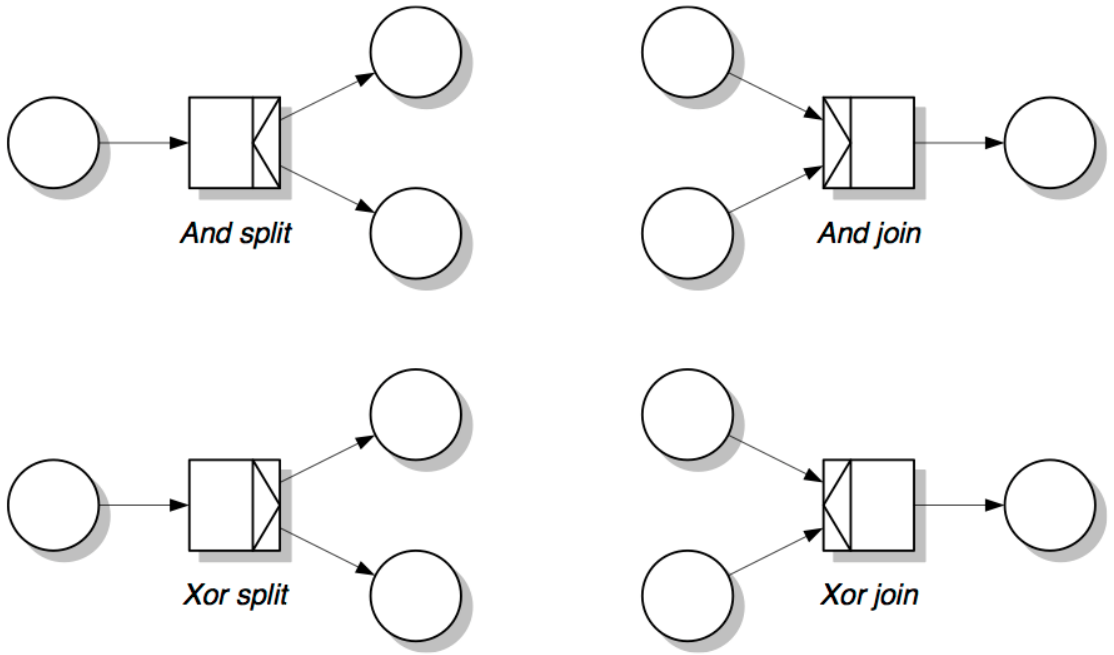
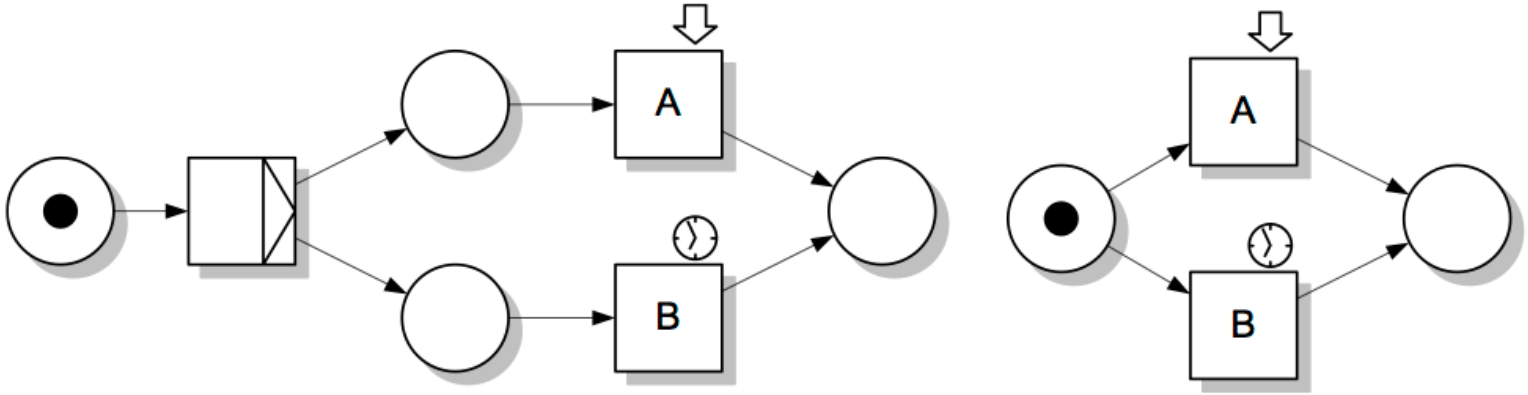


Fig. 4.46. Syntactic sugaring of transitions in workflow nets

Workflow Nets

Example: Explicit choice of conditional branch (by user or based on transition predicates)



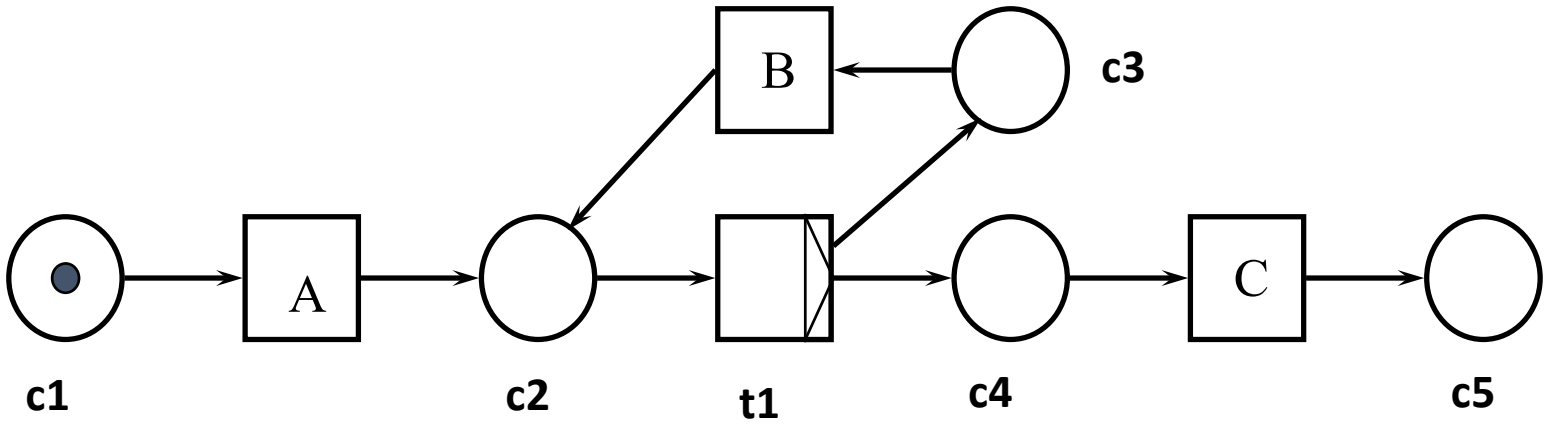
(a) *Explicit xor split* does not enable A and B concurrently

(b) *Implicit xor split* enables A and B concurrently

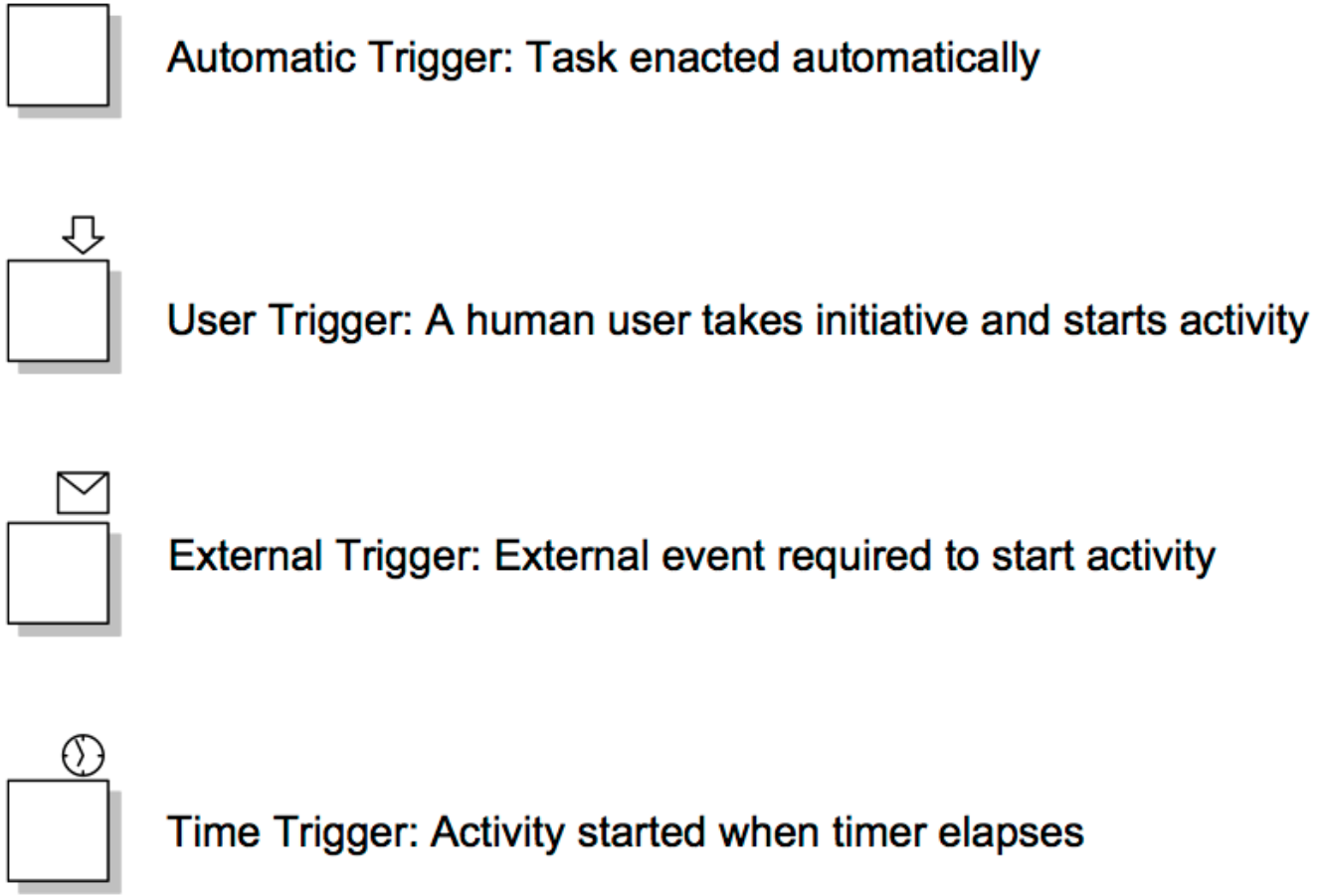
Fig. 4.49. Sample workflow nets illustrate the difference between *explicit xor split* (a) and *implicit xor split* (b)

Workflow Nets

Example: Modeling loop backs: *B may be fired multiple times!*



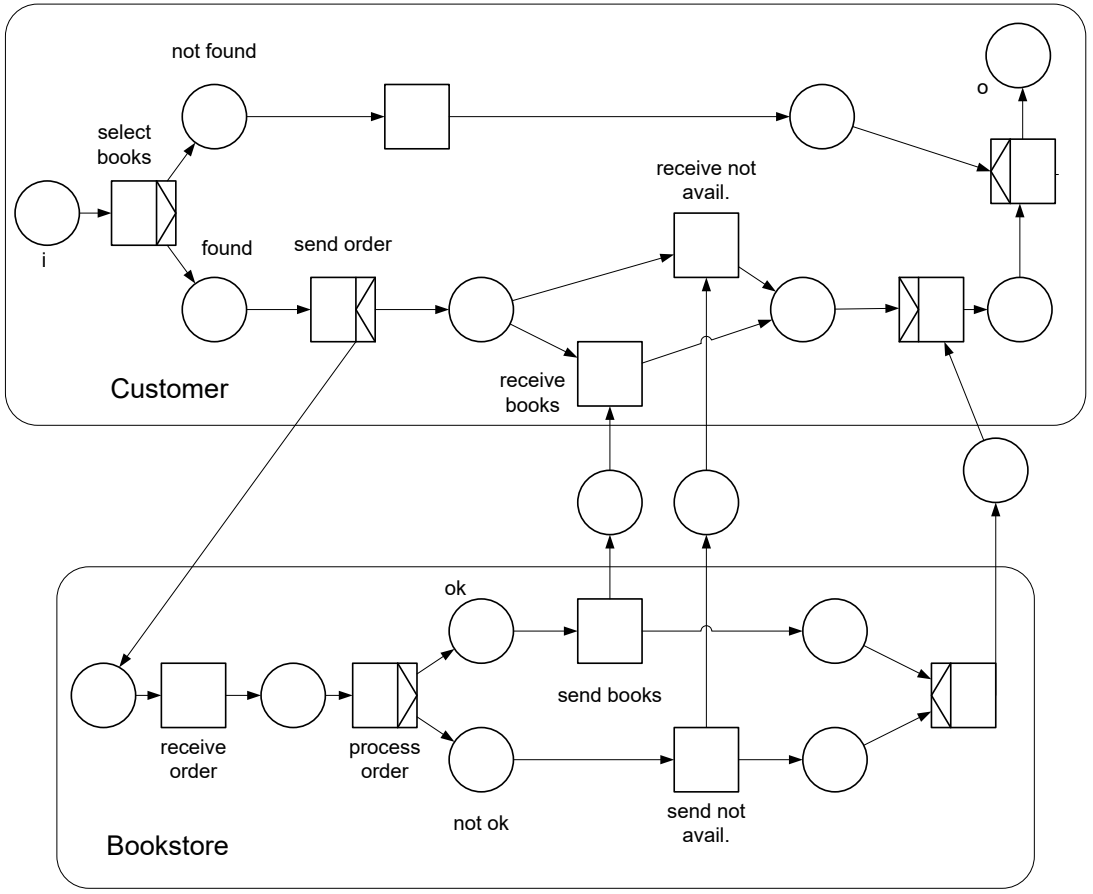
Workflow Nets



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2012, 2007

Fig. 4.47. Triggers in workflow nets

Workflow Nets



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.52. Sample workflow net involving multiple parties

Workflow Nets

Representation of different workflow instances by token colors

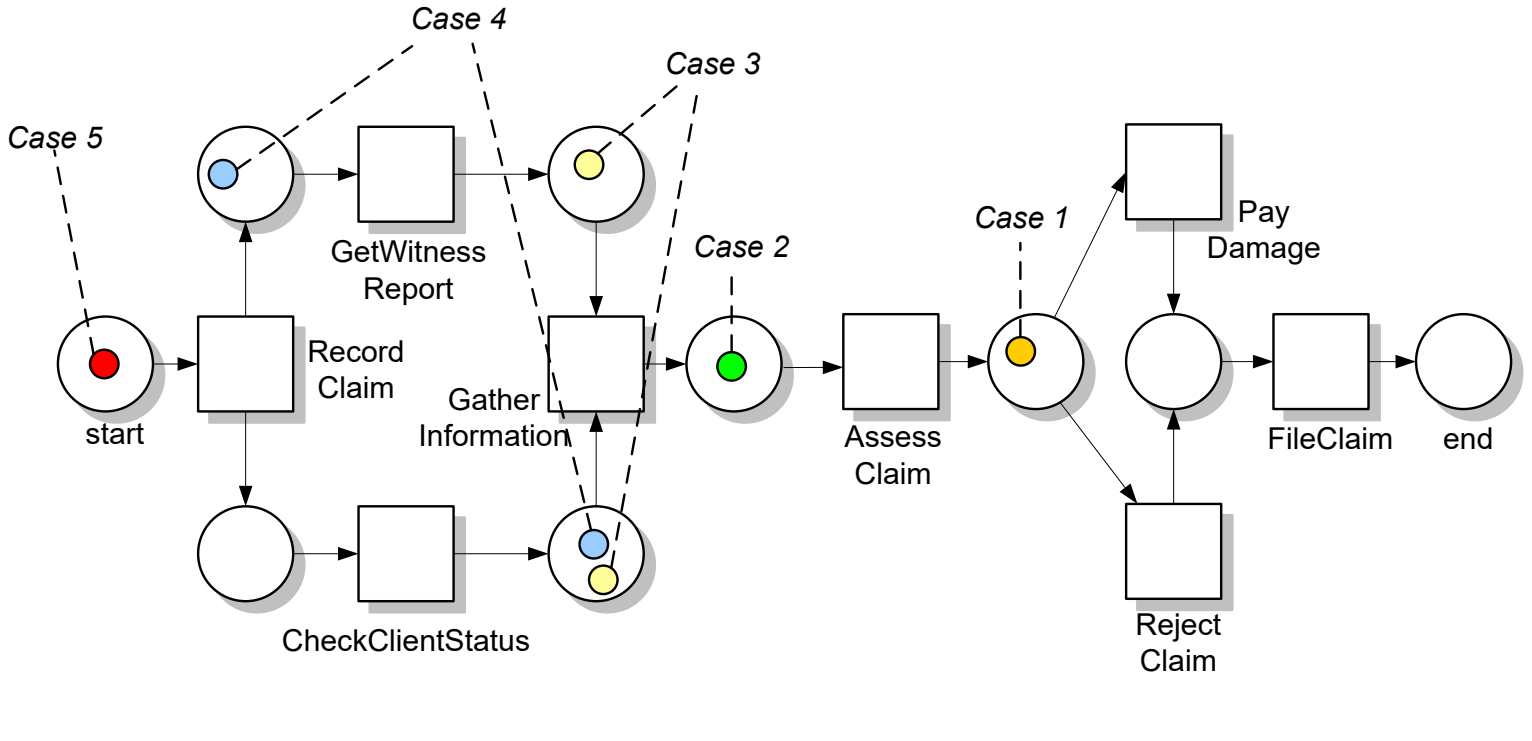


Fig 4.53. Sample workflow net with coloured tokens representing process instances

Workflow Nets

- First of all note the following important distinctions:
  - **task:** A logical step which may be executed for many cases.
  - **work item** = task + case -> A logical step which may be executed for a specific case.
  - **activity** = task + case + (resource) + (trigger) -> The actual execution of a task for a specific case.
- Work items and activities are task instances.
- Workflow Nets abstract from data flow and temporal issues
- Extended Workflow Nets → YAWL (Yet Another Workflow Language): support of Workflow Patterns (later)

Soundness Notions and Analysis

- Problem: Workflow design is often erroneous
- What is the error in the following example workflow model?
- What is cheaper?
  - Detecting the error before deploying the workflow …
  - … or when it is already running?

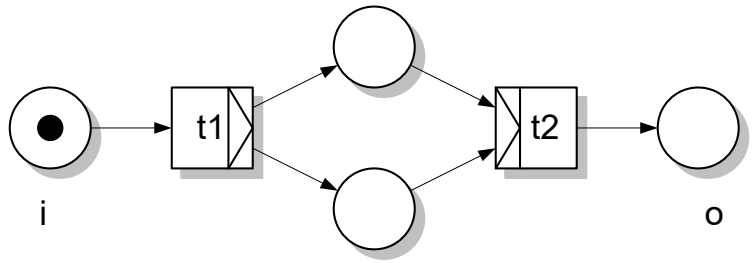


Fig 6.4. Workflow net with deadlock

Soundness Notions and Analysis

DEFINITION 2.10 (Structural Soundness) A process model is structurally sound if the following conditions hold:

- **∃ initial node (without incoming edges)**
- **∃ final node (without outcoming edges)**
- **Each node is on a path from inital node to final node**

- Criterion takes only structure of a process into account (no marking information yet)
- Independent of modeling notation / formalism
- Trivially fulfilled for WF Nets (fulfilled by definition)

Soundness Notions and Analysis

- The following Petri Net is obviously structurally not sound
- What are the consequences?

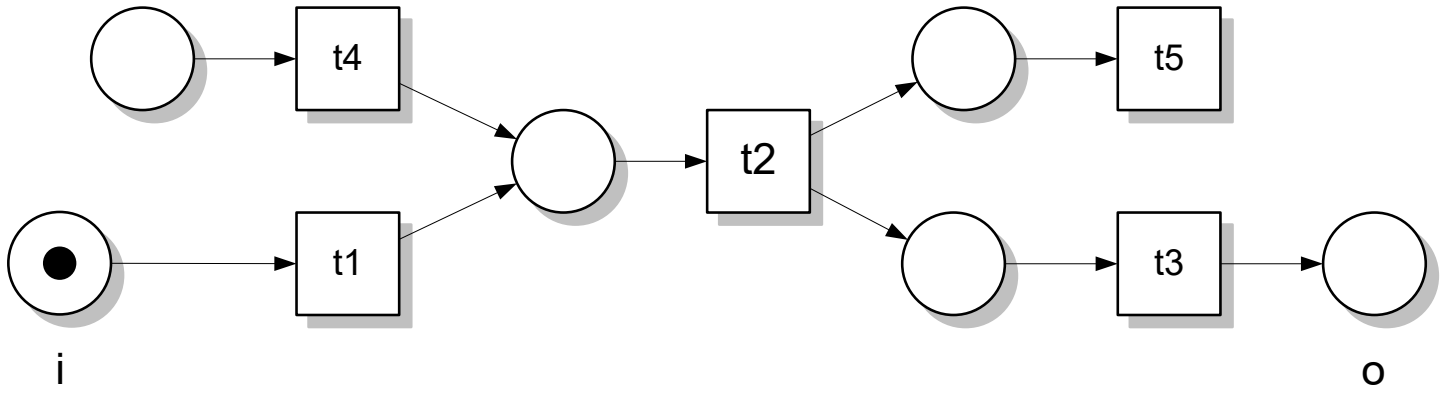


Fig 6.3. Petri net with dangling places and dangling transitions

Soundness Notions and Analysis

- Let us go back to the motivating example
- The problem here is not a structural one, it is behavorial one
- Specifically, at runtime, the workflow instance will run into a deadlock
- A deadlock is a reachable state where no transition is activated
- How can we determine deadlocks?

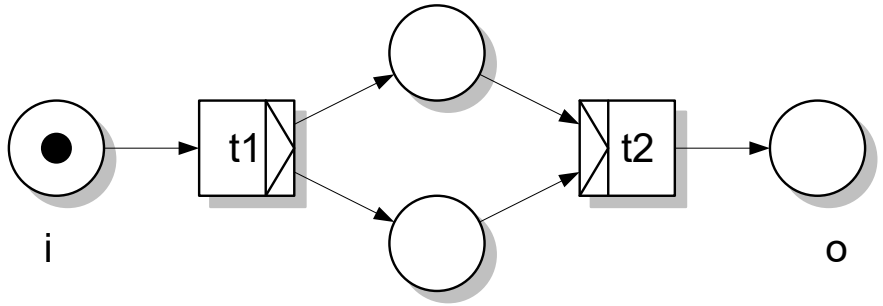


Fig 6.4. Workflow net with deadlock

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Soundness Notions and Analysis

- In addition to deadlocks, the workflow might run into a livelock
- Livelock: a subset of the workflow activities is trapped in an infinite loop
- Possible reasons:
  - Loop backward condition always evaluates to true
  - Workflow model design as in the following example:

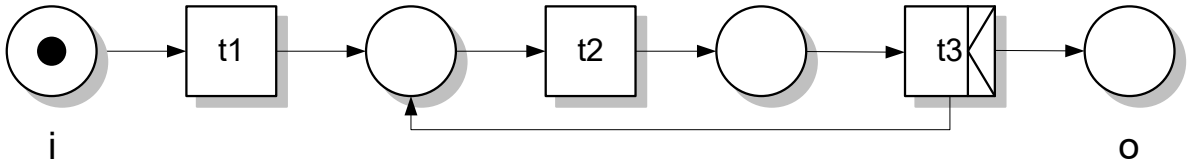
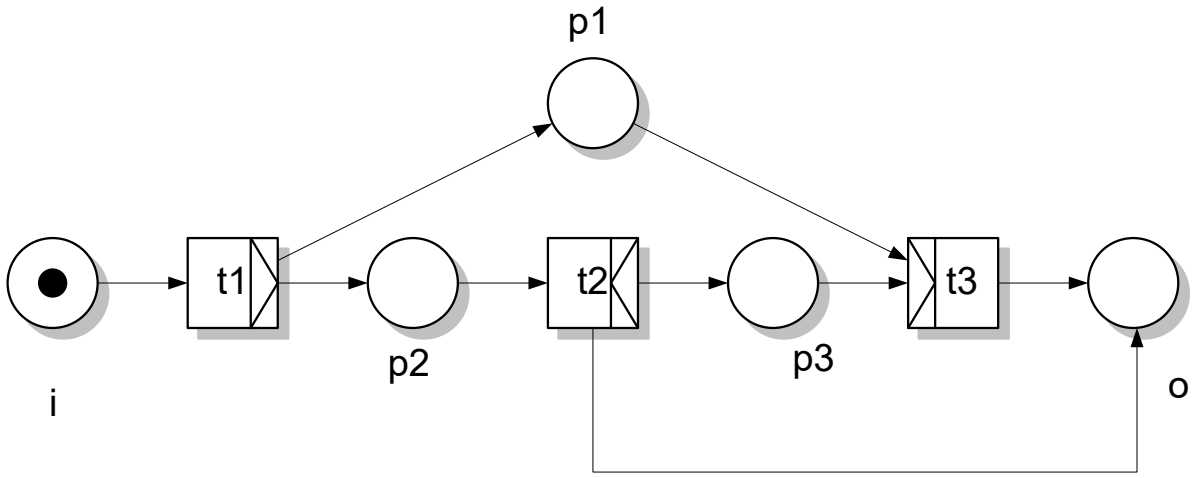


Fig 6.5. Workflow net with livelock

Soundness Notions and Analysis

- What are the problems with the following WF Net?



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 6.6. Workflow net with deadlock/remaining tokens

Soundness Notions and Analysis

DEFINITION 2.11 (Useful Notions) Let PN = (P, T, F) be a WF Net, $i \in P$ be its initial place, $o \in P$ its final place, and M, M‘ markings. Then we define the following notions:

- **M_i is the state (marking) in which there is exactly one token in place i and no token in any other place $p \in P$**
- **M_f is the state (marking) in which there is exactly one token in place o and no token in any other place $p \in P$**
- **$M \geq M' \Leftrightarrow M(p) \geq M'(p)\ \forall\ p \in P$**
- **$M > M' \Leftrightarrow M \geq M' \wedge \exists\ p \in P$ with $M(p) > M'(p)$**
- **(PN, M_i) is called a workflow system based on WF Net PN with intial marking M_i**

Soundness Notions and Analysis

DEFINITION 2.12 (Soundness) A workflow system (PN, M_i) with PN = (P, T, F) is sound $\Leftrightarrow$ the following conditions hold:

- **For every marking M reachable from M_i there exists a firing sequence leading from M to M_f, formally:**

$$\forall M\,(M_i \xrightarrow{*} M) \Rightarrow (M \xrightarrow{*} M_f)$$

- **M_f is the only marking reachable from M_i with at least one token in place o, formally:**

$$\forall M(M_i \xrightarrow{*} M \wedge M \geq M_f) \Rightarrow (M = M_f)$$

- **There are no dead transitions for PN in M_i, formally:**

$$(\forall t \in T)\exists M, M' : i \xrightarrow{*} M \xrightarrow{t} M'$$

Soundness Notions and Analysis

- Soundness can be verified based on the reachability graph of the WF Net
- Is the following WF Net sound?

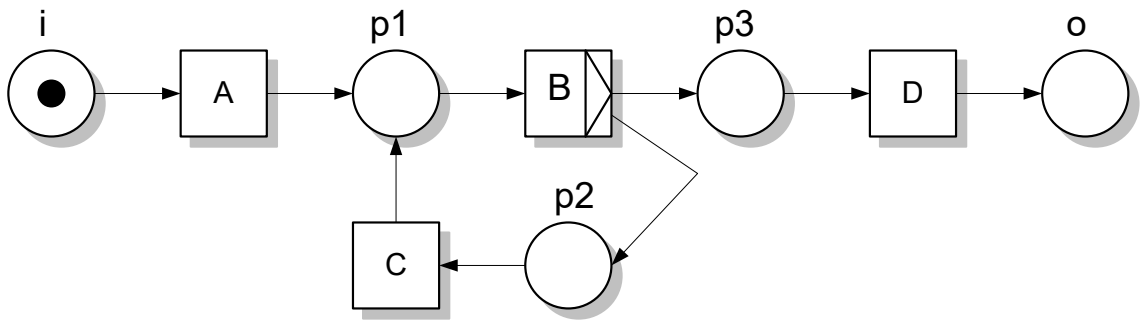


Fig 6.8. Workflow net with loop

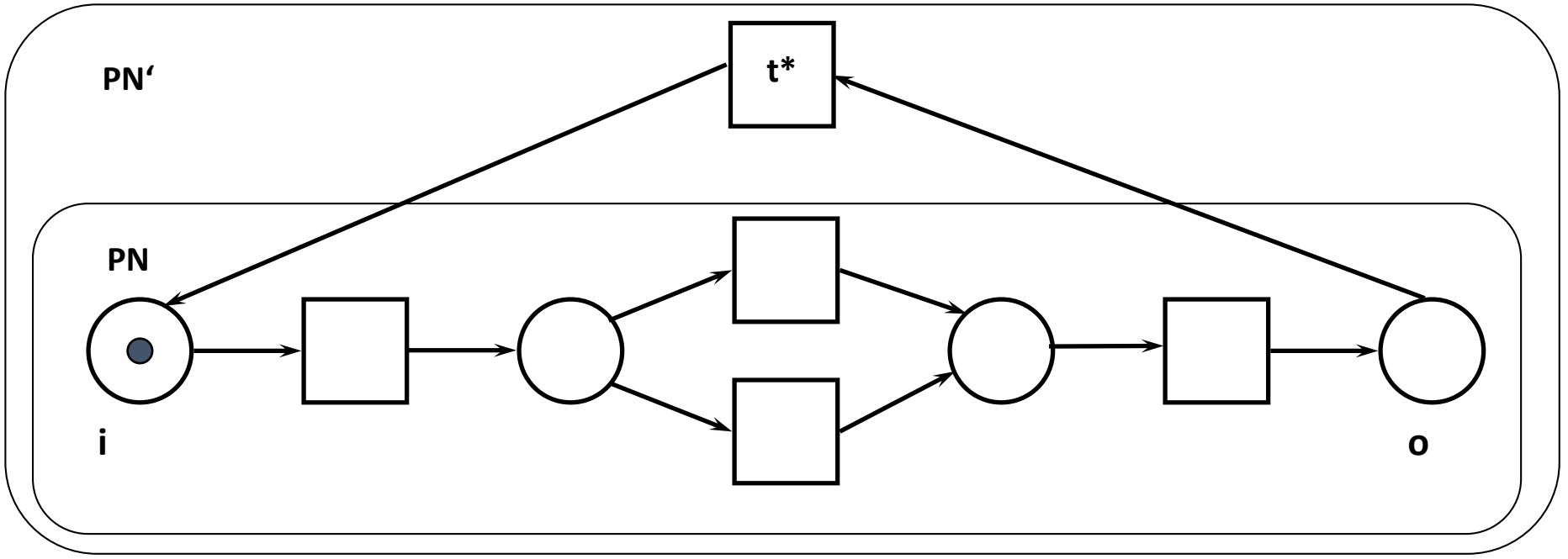
M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Soundness Notions and Analysis

- Computing the reachability graph works for simple models
- Support by analysis tools (e.g., Woflan http://www.win.tue.nl/woflan/doku.php)
- Due to state explosion runtime behavior is exponential
- Idea: show that if certain properties hold for a WF Net, this WF Net is sound AND the properties can be checked efficiently
- The first and third soundness conditions imply some property such as liveness
- The second soundness condition implies some property such as boundedness
- However, these properties are not 100% fitting on WF Nets
- Trick: Use extension of WF Nets

Soundness Notions and Analysis

- Example WF Net PN is obviously sound, bounded, but not live
- How to extend PN such that the extended Petri Net becomes live?
- Introduce artificial transition t* connecting places o and i



Soundness Notions and Analysis

- Liveness and boundedness can be checked in polynomial time for free-choice nets
- Within a free choice net the sets of input places of two transitions are either disjoint or identical

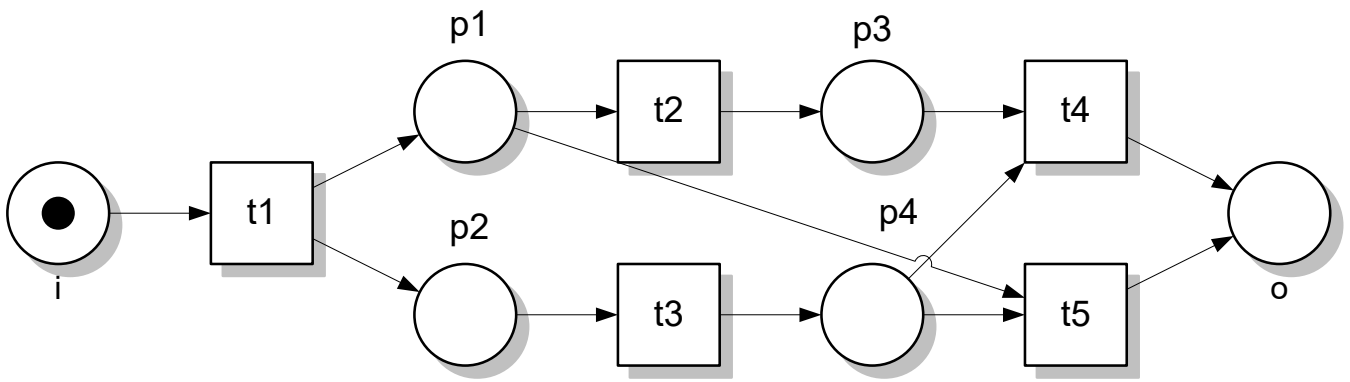


Fig 6.11. Non-free-choice workflow net

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Soundness Notions and Analysis

Soundness is desirable for many workflow applications, because it guarantees the following properties

- ***P1 (Termination):*** every instance starting in the initial state finally reaches the final state → no instance „gets lost“ somewhere during the the workflow execution
- ***P2 (Proper Termination):*** the final state is the only state reachable from the intial state where there is a token in the final places → no tokens are somewhere left in the workflow when it is already finished
- ***P3 (Do dead transitions):*** Each transition can constribute to at least one workflow instances
- ***P4 (Transition participation)*** – (implied: P1 $\wedge$ P3 $\Rightarrow$ P4): for each transition t there exists a firing sequence from i to o in which t participates:

$$(\forall t \in T)\exists M, M' : (M_i \xrightarrow{*} M \xrightarrow{t} M' \xrightarrow{*} M_f)$$

Soundness Notions and Analysis

- However, for some application scenarios soundness is too strict
  - Scenario 1: Semiformal business process modeling (e.g., using EPCs → Chapter 1)
    - Rather modeling of „best practices“ than of correct models
    - Goal: model should reflect necessary execution paths
    - However, it might also reflect non-correct ones
  - Scenario 2: In the context of **interorganizational workflows** (**workflow choregraphies**)
    - Not executed within one enterprise (inhouse)
    - but executed among different partners, mostly based on message exchange (e.g., supplier → producer → customer)
    - Practical assumption: services are used which provide necessary functionality (and even more functionality than that) and the „excessive“ functionality does not have to be used

Soundness Notions and Analysis

- This leads to ***relaxed*** soundness notions (i.e., notions where we abstain from certain properties P1 – P4)
- For Scenario 1 the notion of ***relaxed soundness*** has been developed: *For each transition t there is a workflow instance execution which starts in the initial state, executes t, and terminates in the final state.*

DEFINITION 2.13 (Sound firing sequence) Let S = (PN, M_i) be a workflow system. Let σ, σ‘ be firing sequences and let M, M‘ be markings. σ is a sound firing sequences if

$$M_i \xrightarrow{\sigma} M \wedge \exists \sigma' : M \xrightarrow{\sigma'} M_f$$

DEFINITION 2.14 (Relaxed Soundness): A workflow system S = (PN, M_i) is relaxed sound ⇔

$$\forall t \in T \exists M, M' : (M_i \xrightarrow{*} M \xrightarrow{t} M' \xrightarrow{*} M_f)$$

Soundness Notions and Analysis

- For Scenario 2 the ***weak soundness*** notion has been developed
- Weak soundness requires termination (P1) and proper termination (P2)
- However, it does not require that every transition participates in the workflow execution

DEFINITION 2.15 (Weak Soundness) A workflow system (PN, M_i) with PN = (P, T, F) is weak sound ⇔ the following conditions hold:

- **For every marking M reachable from M_i there exists a firing sequence leading from M to M_f, formally:** $\forall M\,(M_i \xrightarrow{*} M) \Rightarrow (M \xrightarrow{*} M_f)$
- **M_f is the only marking reachable from M_i with at least one token in place o, formally:**

$$\forall M(M_i \xrightarrow{*} M \wedge M \geq M_f) \Rightarrow (M = M_f)$$

Soundness Notions and Analysis

- *Soundness* is the strictest correctness notion requiring properties P1, P2, P3 (and implies P4)
- *Weak soundness* allows for dead transitions, but guarantees proper termination (P1, P2)
- *Relaxed Soundness* only guarantees that every transition participates in at least one workflow instance (P4), but allows for deadlocks
- *Lazy soundness* has been developed for certain workflow patterns which are neither sound nor weak sound nor relaxed sound
- Lazy soundness allows for tokens to be left within the workflow instance even if there is a token within the final place
- The transitions possibly activated by the left tokens are called lazy activities
- They might be fired even after the termination of the workflow instance
- Properties P1 – P4 cannot be cleary verified for or against lazy soundness → further investigations necessary

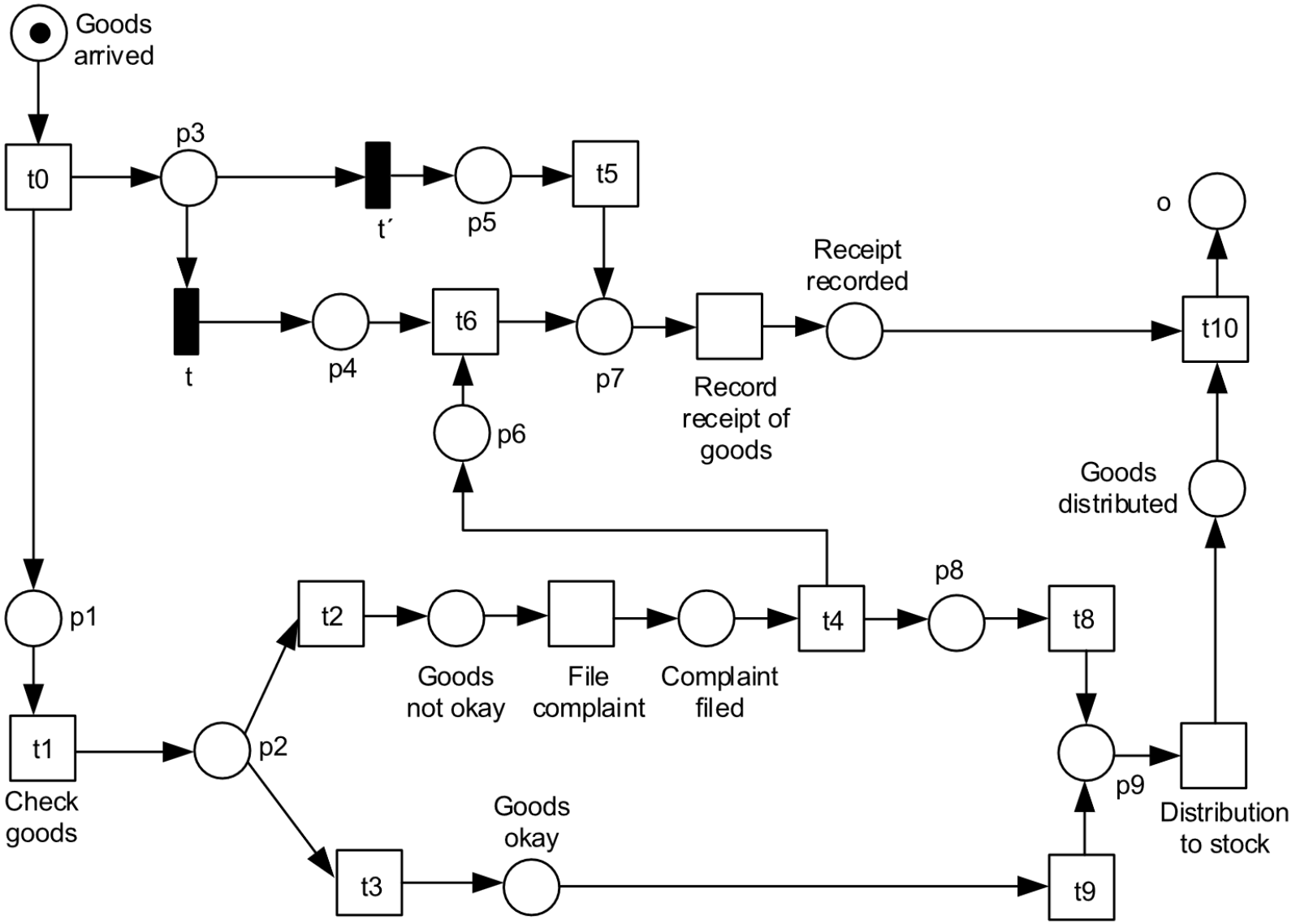


Fig. 7.19. Relaxed sound workflow net

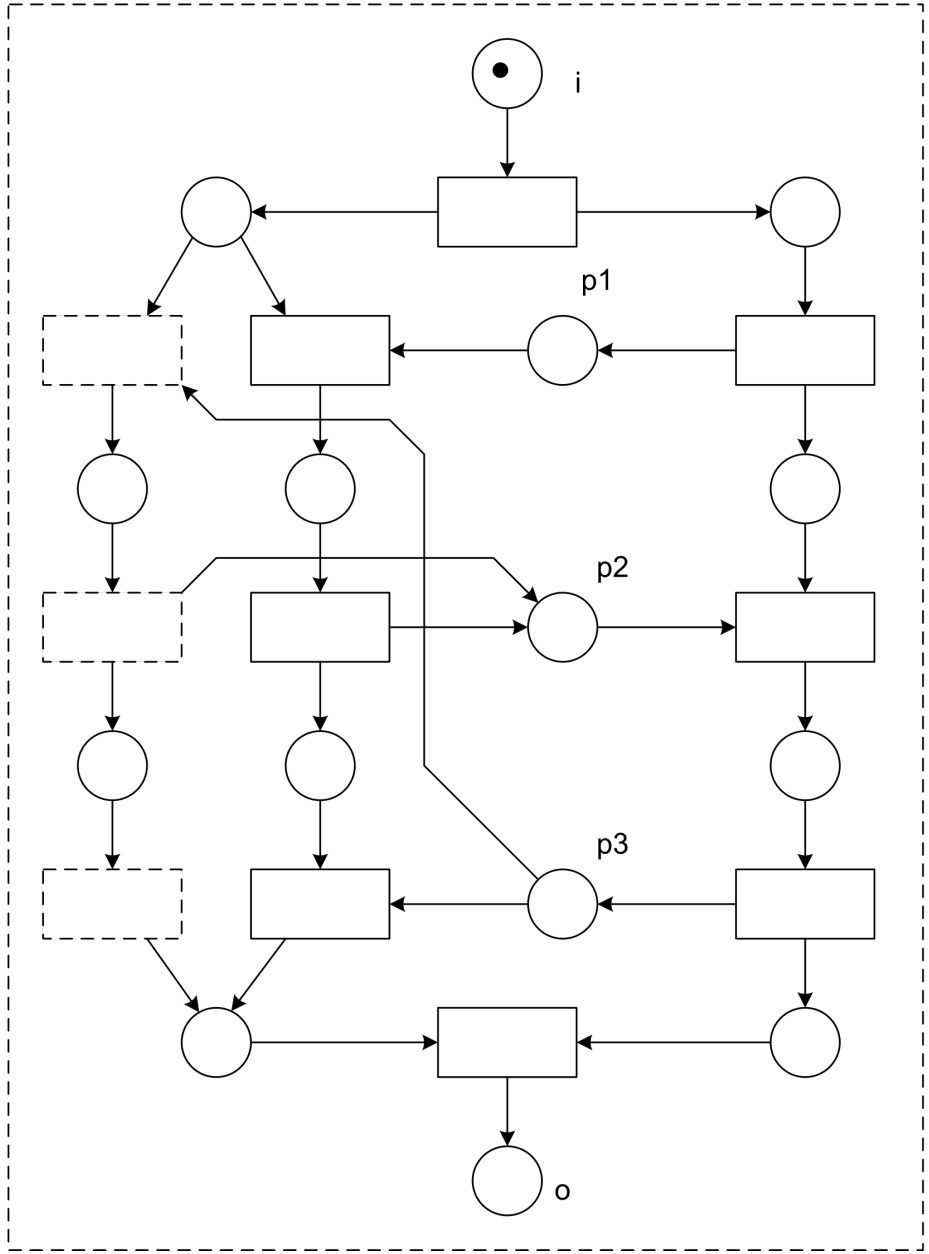


Fig. 7.24. Example of weak sound workflow net

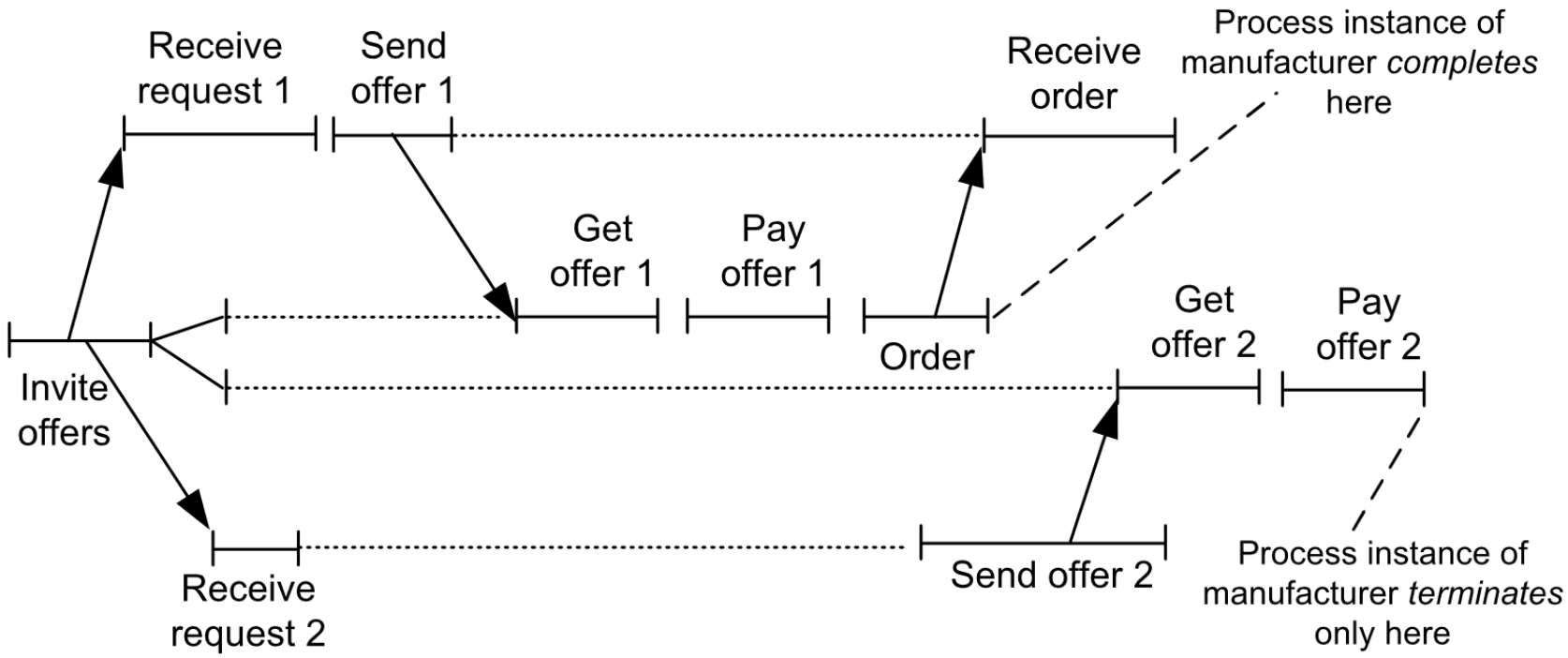
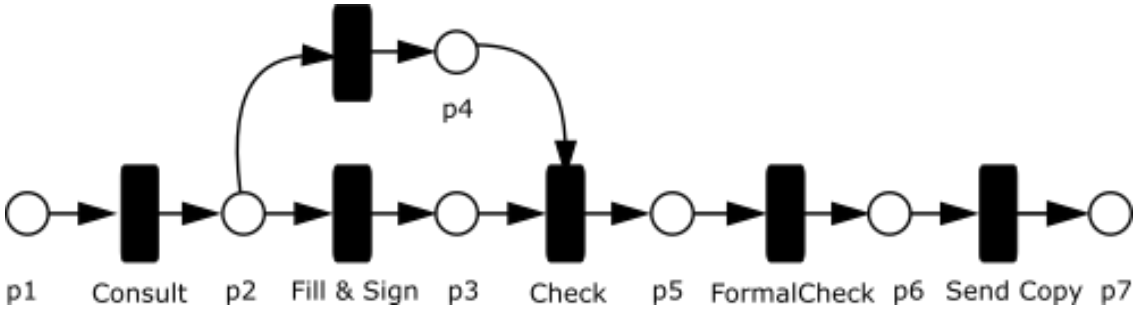


Fig. 7.28. Process instance, illustrating lazy soundness

Tool Support

- CPN-Tools: https://cpntools.org/
- Woflan: http://is.tm.tue.nl/research/woflan/
- YAWL (Open source)
  - YAWL: Yet Another Workflow Language
  - https://yawlfoundation.github.io/
  - Modeling and enactment of workflows based on high-level (Workflow) Nets
  - Omitting places allows for more compact visualization

Verification with Woflan

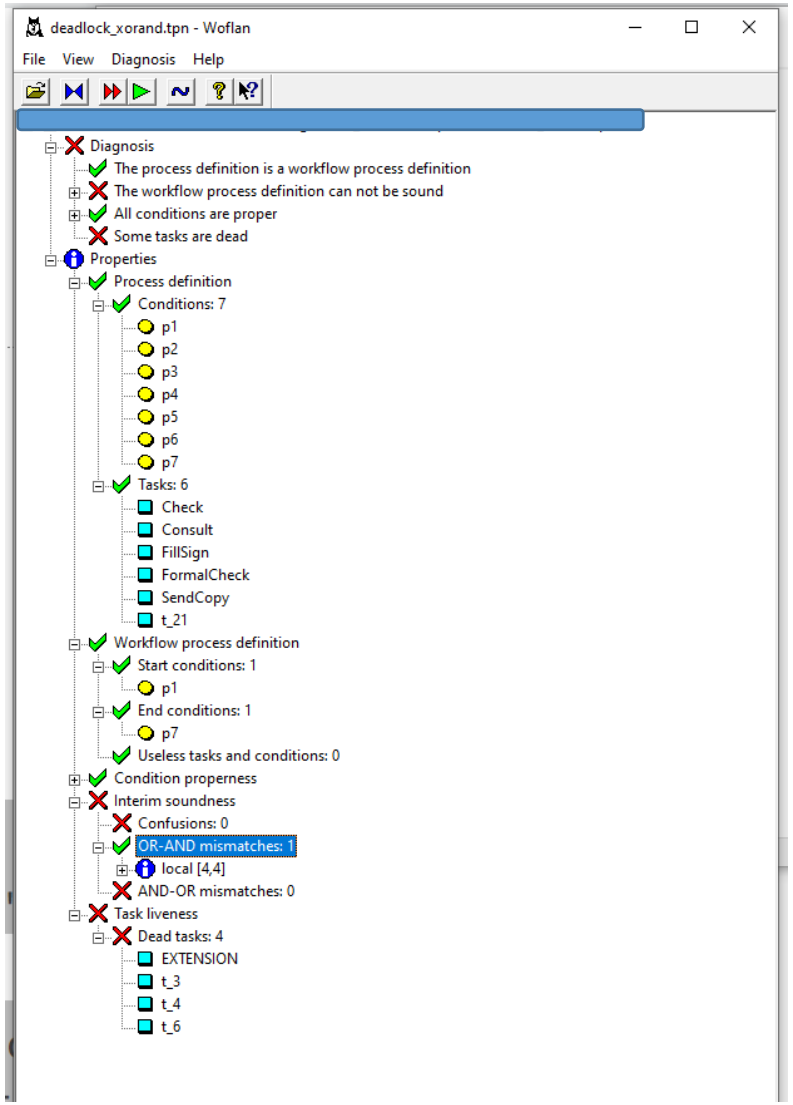


.tpn Format

```
place "p1";
place "p2";
place "p3";
place "p4";
place "p5";
place "p6";
place "p7";
trans "t_1"~"Consult" in "p1" out "p2" ;
trans "t_2"~"FillSign" in "p2" out "p3" ;
trans "t_21" in "p2" out "p4" ;
trans "t_3"~"Check" in "p3" "p4" out "p5" ;
trans "t_4"~"FormalCheck" in "p5" out "p6" ;
trans "t_6"~"SendCopy" in "p6" out "p7" ;
```

Verification with Woflan

```
place "p1";
place "p2";
place "p3";
place "p4";
place "p5";
place "p6";
place "p7";
trans "t_1"~"Consult" in "p1" out "p2" ;
trans "t_2"~"FillSign" in "p2" out "p3" ;
trans "t_21" in "p2" out "p4" ;
trans "t_3"~"Check" in "p3" "p4" out "p5" ;
trans "t_4"~"FormalCheck" in "p5" out "p6" ;
trans "t_6"~"SendCopy" in "p6" out "p7" ;
```



RPST

Refined Process Structure Trees are a formal construct that helps to (1) verify that a workflow graph is executable, (2) can serve as the basis for executing (interpreting) the graph, or (3) can serve as the basis for easily transforming the graph to an imperative programming language (such as python).

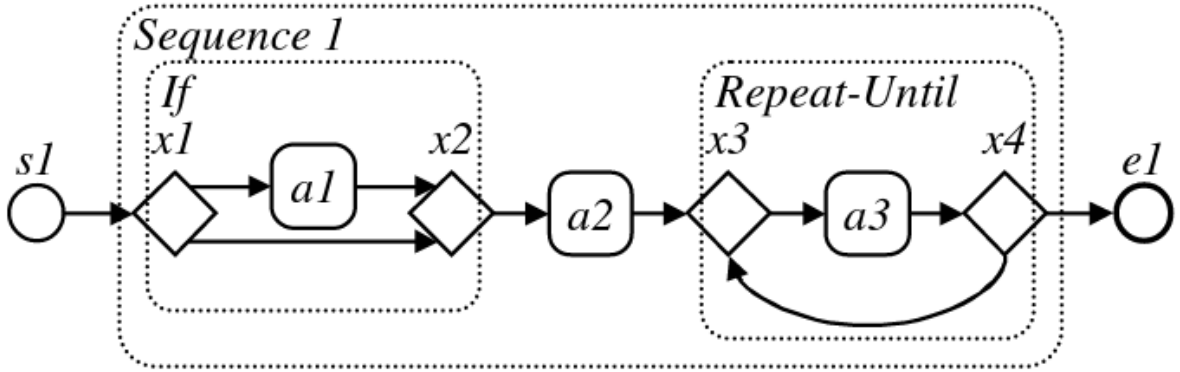
All graphs and figures that explain the RPST and its concepts have been taken from the literature below.

Literature

- Vanhatalo, Jussi, Hagen Völzer, and Jana Koehler. "The refined process structure tree." International Conference on Business Process Management. Springer, Berlin, Heidelberg, 2008.
- Sadiq, M. E. Orlowska, Analyzing process models using graph reduction techniques., Inf. Syst. 25 (2) (2000) 117–134.

RPST

- Worflow Graphs: BPMN, EPKs, ... are saved as nodes (tasks) and edges that connect them.
- Process Execution Engines, like most compilers or virtual machines (e.g. java, gcc, python) internally rely on Abstract Syntax Trees (AST). A formal programming language, e.g. BPMN, is translated to a parse tree for analysis (find syntax errors) and execution.
- RPST is an AST, that is geared towards the rather minimal grammar workflow graphs often represent (many branches, no assignments, very simple conditions).
- RPSTs allow to decompose the graph into logical blocks (cmp. structural sound petri nets) with one single starting point, and one single end point.
- The leaves of the tree are always tasks, branches represent logical constructs such as XOR, OR, PARALLEL, ...



RPST

Workflow graphs - Two Terminal Graphs (TTG) - i.e. structural sound petri nets

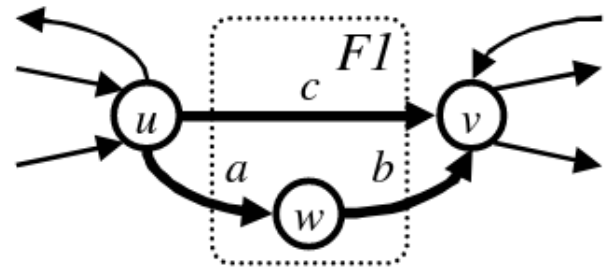
- one source node (without incoming edges)
- one sink node (without outcoming edges)
- each node is on a path from source node to sink node

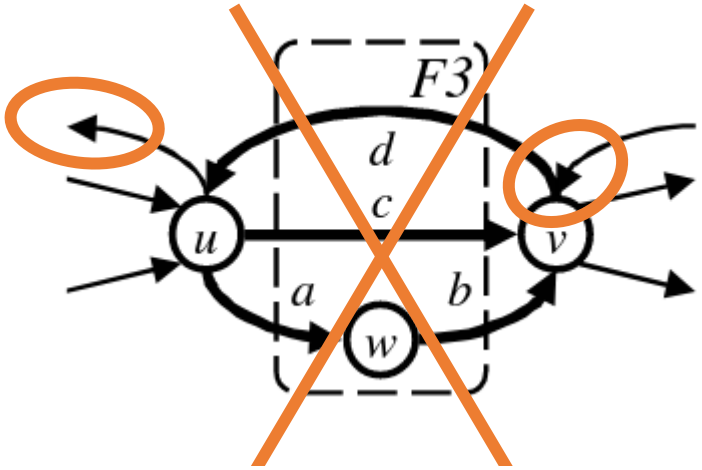
TTG: G = (V, E, M)

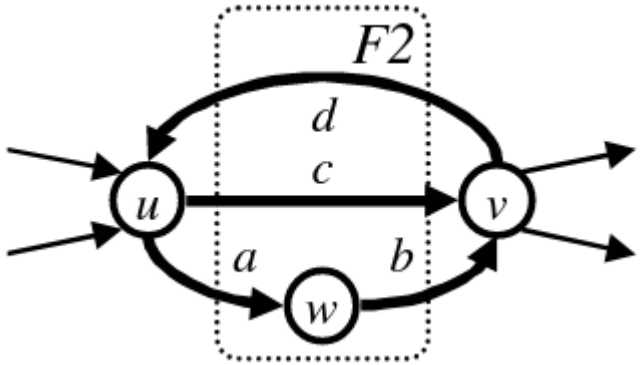
- V is the set of nodes (e.g. u, v, w)
- E the set of edges (e.g. a,b, c)
- M pairs of ordered/unordered nodes connected by edges

F = Fragment - a subset of edges E

- one source/entry boundary node s - all outgoing edges of s are in F **OR** no incoming edges of s are in F
- one sink/exit boundary node t - all incoming edges of t are in F **OR** no outgoing edges of t are in F
- t and s have to be connected through the edges in F.
  All edges in F have to be in at least one path from s to t. For example:
  - F1 = {a,b,c}, boundary nodes u,v
  - F2 = {a,b,c,d}, boundary nodes u,v; ab is a path from s to t that connects through w.
  - F3 = {a,b,c,d) - WRONG



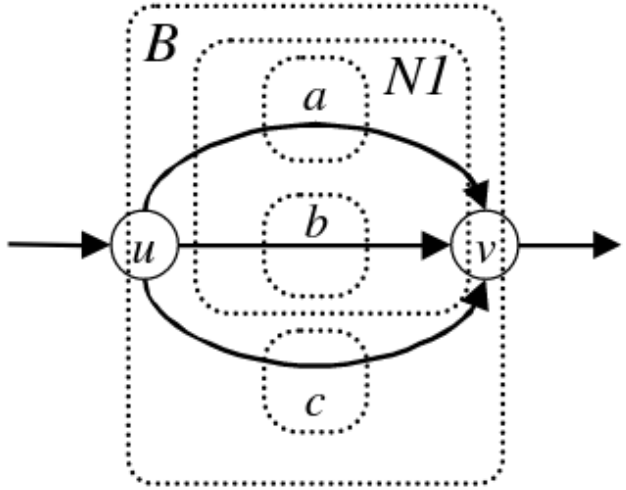




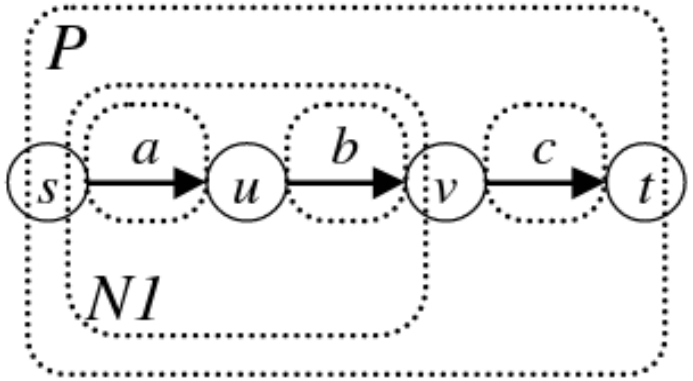
RPST

Decomposition of G = (V, E, M) - build a set of fragments ε

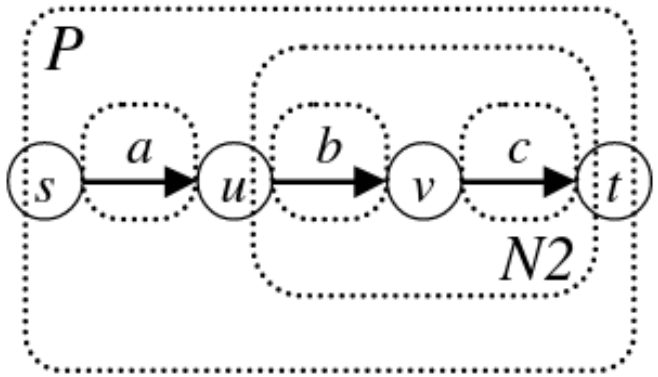
- Rule 1: E ϵ ε - each edge is part of the fragment set
- Rule 2: F1 ∩ F2 = ∅ or F1 ⊆ F2 or F2 ⊆ F1 - fragments are either disjoint or part of each other. They are not allowed to partially overlap.
- Decompositions should be maximal - i.e. Rule 1 must be fulfilled and no more fragments exist that fulfill Rule 2.
- Different decompositions can exist.



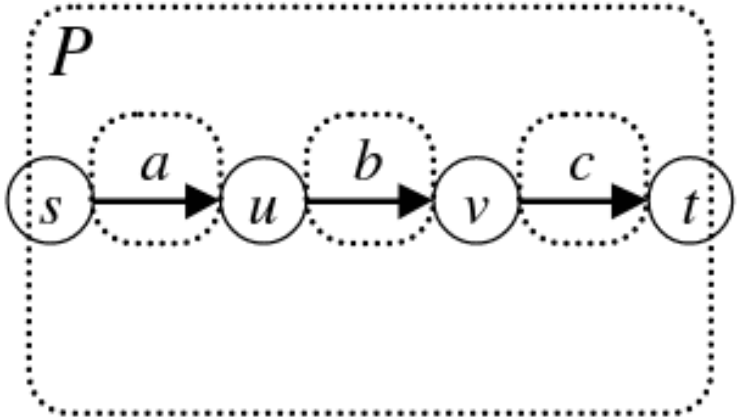
Maximal Decomposition



Maximal Decomposition



Alternative Maximal Decomposition

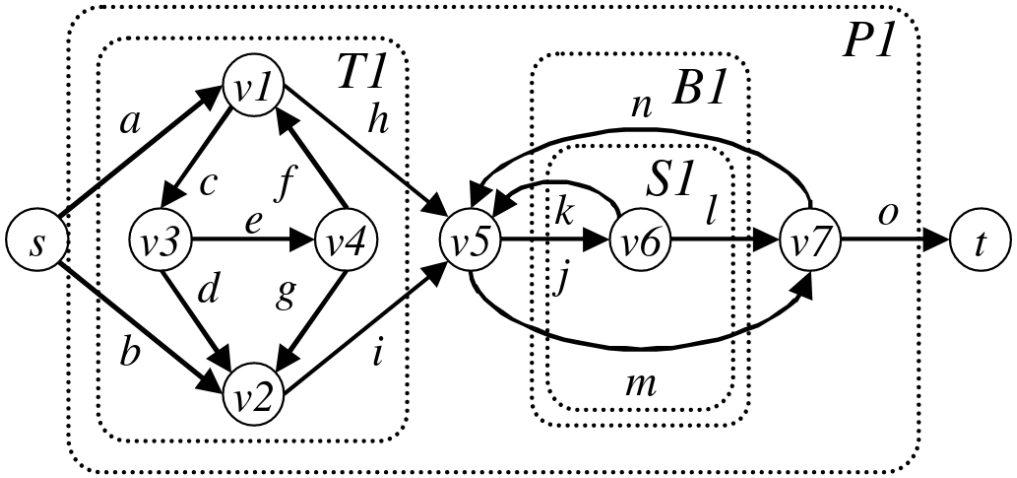


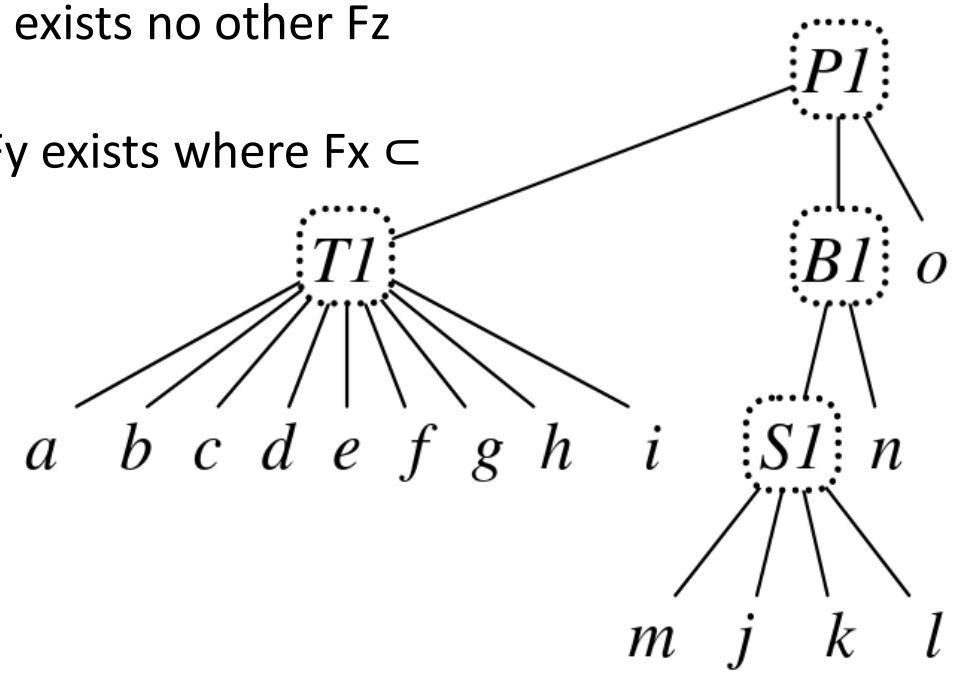
Not-yet-maximal Decomposition

RPST

Final Step - RPST

- One maximal decomposition.
- Nodes are leaves.
- Each fragment Fx has a parent Fy if Fx ⊂ Fy and there exists no other Fz where |Fz \ Fx| < |Fy \ Fx|
- A fragment Fx is the root of the RPST if no fragment Fy exists where Fx ⊂ Fy





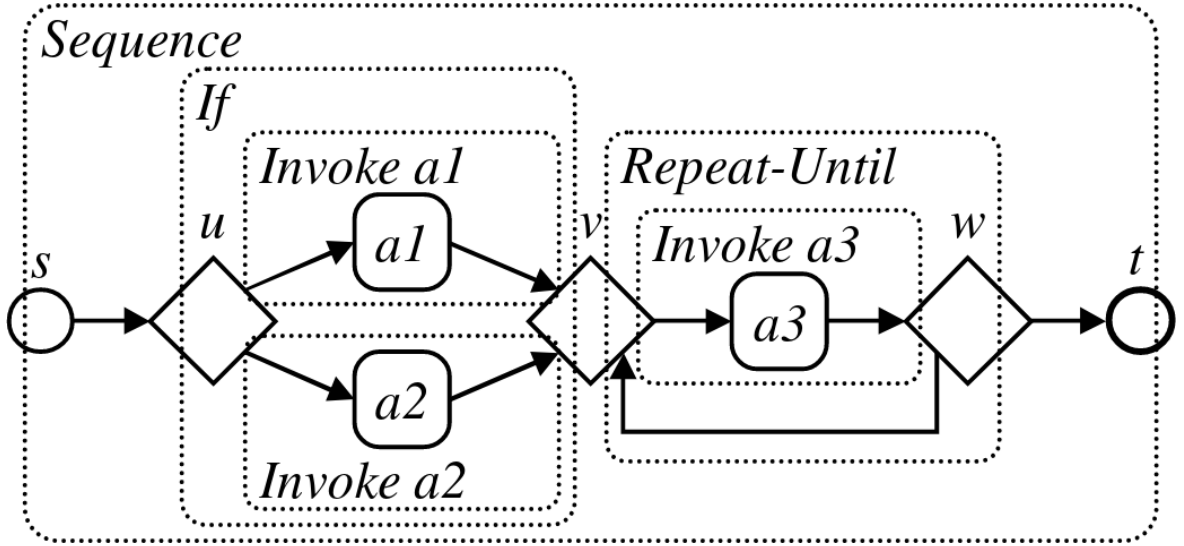
RPST

- Application - IBM WebSphere Business Process Suite: support different languages such as BPEL and BPMN. All formats are transformed to an RPST. RPST is used for control-flow analysis (different branches of the tree can be analysed in parallel) an refactoring (find identical branches).

- Application - cpee.org: an XML representation of the RPST is the native format of the BPMN modeler. When a BPMN process model is instantiated, the RPST is translate to executable code, compiled and executed on a target machine.

- Possible Applications: efficient difference and merge algorithms (see Theses page on https://bpm.in.tum.de).

- Possible Applications: every technique known from compiler theory, such as data-flow analysis or code optimization.



RPST

Further reading

- Please study [Vanhatalo, Jussi, Hagen Völzer, and Jana Koehler 2008] or [Sadiq, M. E. Orlowska 2000] for a linear time algorithm to automatically extract an RPST from a workflow graph.

Summary and Outlook

- Petri Nets provide the basis for many studies on workflow properties
  - Formal foundations
  - Formal semantics
  - Analysis functions
- For workflow verification often it is proposed to map other formalisms to Petri Nets → then use standard Petri Nets analysis tools
- Common workflow patterns are described by using Petri Nets (http://www.workflowpatterns.com/)
- However, as mentioned, important aspects are abstracted from
  - Data flow
  - Temporal restrictions
  - Organizational aspects
- Might be overcome by use of high-level Petri Nets